

Team Notebook

Ateneo de Manila University

The 100 Last Minute Bugs That Really, Really, Really, Really, Really Love You

June 16, 2026

Contents

1	Boilerplate	3
1.1	Template	3
1.2	Debug	3
1.3	Generator-Checker	3
2	Data Structures	3
2.1	Sparse Table	3
2.2	Square Root Decomposition	4
2.3	Segment Tree Monoids	4
2.4	Segment Tree	5
2.5	2D Segment Tree	5
2.6	Dynamic Segment Tree	6
2.7	Persistent Segment Tree	7
2.8	Kth Smallest Persistent Segment Tree	8
2.9	Segment Tree Beats	8
3	Geometry	9
3.1	Point	9
3.2	Area of a Polygon	9
3.3	Intersection	9
3.4	Get Length of Segment Union	10
3.5	Get Intersecting Segments	10
3.6	Get Minimum Enclosing Circle	10
3.7	Get Nearest Pair of Points	10
3.8	Li-Chao Tree	10
4	Graphs	11
4.1	Topological Sort	11
4.2	Cycle Find DFS	11
4.3	Disjoint Set Union/UnionFind	11
4.4	Shortest Paths	11
4.4.1	Dijkstra	11
4.4.2	Bellman-Ford	12
4.4.3	Floyd-Warshall	12
4.5	Trees	12
4.5.1	Lowest Common Ancestor	12
4.5.2	Heavy Light Decomposition	12
4.5.3	Centroid Decomposition	12
4.6	Connectivity and Components	13
4.6.1	Bridge-Finding Algorithm	13
4.6.2	Block-Cut Tree	13

4.6.3	Cutpoints-Bridges	13
4.6.4	Kosaraju	13
4.6.5	2SAT	13
4.6.6	Flows and Bipartite Matching	14
4.6.7	Dinic	14
4.6.8	Ford-Fulkerson Directed	14
4.6.9	Ford-Fulkerson Undirected	14
4.6.10	Kuhn	15
4.6.11	Min Cost Max Flow	15

5	Math	15
5.1	Binary and Ternary Search	15
5.2	Modulo Integer	16
5.3	Matrix	17
5.4	Modulo Pow	20
5.5	Modulo Combinatorics	20
5.6	Dynamic MEX	20
5.7	Extended Euclidean GCD	20
5.8	Sieve of Eratosthenes	20
5.9	Linear Sieve	20
5.10	Segmented Sieve	21
5.11	Is Prime Miller-Rabin	21
5.12	Get One Factor Brent's Pollard-Rho	21
5.13	Get Prime Factors	21
5.14	Get All Factors	21
5.15	Euler's Totient Function	22
5.16	Number of Divisors	22
5.17	Sum of Divisors	22
5.18	Inverse Modulo	22
5.19	Chinese Remainder Theorem	22
5.20	Garner	22
5.21	Modulo Log	22
5.22	Primitive Root	22
5.23	Modulo Root	23
5.24	Floor Sum	23
5.25	Montgomery Multiplication	23
5.26	Poly FFT	23
5.27	Moly NTT	24
5.28	Fast Is Square	26
5.29	Josephus Problem	26
5.30	Simpson Integration	26
5.31	Solve Diophantine Equation	26
5.32	Solve Modulo Equation	26
6	Misc	27
6.1	Custom Hash	27
6.2	Fast Convolutions	27
6.3	Kadane's Algorithm	27
6.4	Longest Common Subsequence	27
6.5	Longest Common Substring	28
6.6	Longest Increasing Subsequence	28
6.7	Random	28
6.8	Z-Algorithm	28

7	Formulas	28
7.1	Physics	28
7.2	Markov Chains	28
7.3	Burnside's Lemma	28
7.4	Bézout's identity	28
7.5	Misc	29
7.5.1	Determinants and PM	29
7.5.2	BEST Theorem	29
7.5.3	Primitive Roots	29
7.5.4	Sum of primes	29
7.5.5	Floor	29
7.5.6	Large Primes	29
7.6	Other Combinatorics Stuff	30
7.7	The Twelfefold Way	30

```
// S:  $O(n * \log(n))$ , U: NA, Q:  $O(1)$  or  $O(\log(n))$ , M:  $O(n * \log(n))$ 
template<typename T, typename F>
struct SparseTable {
    int n;
```

```

static constexpr inline S map(F f, const S &a) {
    if (a.len == 0) { return a; }
    S res;
    res.And = ((a.And & f.a) & ~f.b) | (~a.Or & f.a) & f.b;
    res.Or = ((a.Or & f.a) & ~f.b) | (~a.And & f.a) & f.b;
    res.Xor = (a.Xor & f.a) ^ (a.len & 1 ? f.b : 0);
    res.len = a.len;
    return res;
}

static constexpr inline F cmp(F f, F g) { return {g.a & f.a, (g.b &
f.a) ^ f.b}; }

/*
void andUpdate(lng l, lng r, lng x) { update(l, r, {x, 0}); }
void orUpdate(lng l, lng r, lng x) { update(l, r, {x, x}); }
void xorUpdate(lng l, lng r, lng x) { update(l, r, {~0LL, x}); }
void setUpdate(lng l, lng r, lng x) { update(l, r, {0, x}); }
*/
};

template<bool opt = false>
struct MonGcd {
    struct S { lng Gcd, Lcm, len; };
    struct F {
        lng a, b;
        bool operator==(const F&) const = default;
    };

    static constexpr inline S idS() {
        if constexpr (opt) { return {0, 1, 0}; }
        return {0, 0, 0};
    }

    static constexpr inline F idF() { return {0, 1}; }
    static constexpr inline S defR(lng l, lng r) {
        if constexpr (opt) { return {0, 1, r - 1 + 1}; }
        return {0, 0, r - 1 + 1};
    }

    static constexpr inline S init(lng i, lng x) {
        if constexpr (opt) { return {x, x, 1}; }
        return {x, 0, 1};
    }

    static constexpr inline S ope(const S &a, const S &b) {
        if (a.len == 0) { return b; }
        if (b.len == 0) { return a; }
        S res;
        res.Gcd = gcd(a.Gcd, b.Gcd);
        if constexpr (opt) { res.Lcm = lcm(a.Lcm, b.Lcm); }
        res.len = a.len + b.len;
        return res;
    }

    static constexpr inline S map(F f, const S &a) {
        if (a.len == 0) { return a; }
        S res;
        res.Gcd = lcm(gcd(a.Gcd, f.a), f.b);
        if constexpr (opt) { res.Lcm = lcm(gcd(a.Lcm, f.a), f.b); }
        res.len = a.len;
        return res;
    }

    static constexpr inline F cmp(F f, F g) { return {gcd(g.a, f.a),
lcm(gcd(g.b, f.a), f.b)}; }

/*
void gcdUpdate(lng l, lng r, lng x) { update(l, r, {x, 1}); }
void lcmUpdate(lng l, lng r, lng x) { update(l, r, {0, x}); }
void setUpdate(lng l, lng r, lng x) { update(l, r, {1, x}); }
*/
};

struct MonSar {
    struct S {
        lng sum, max_prf, max_suf, max_sum, min_prf, min_suf, min_sum,
len;

```

```

    void rev() { std::swap(max_prf, max_suf); std::swap(min_prf, min_suf); }
};
struct F {
    bool to_set; lng set_upd;
    bool operator==(const F&) const = default;
};

static constexpr inline S idS() { return {0, -INF64, -INF64, -INF64, INF64, INF64, INF64, 0}; }
static constexpr inline F idF() { return {false, 0}; }
static constexpr inline S defR(lng l, lng r) { return {0, 0, 0, 0, 0, 0, 0, r - l + 1}; }
static constexpr inline S init(lng i, lng x) { return {x, x, x, x, x, x, x, 1}; }

static constexpr inline S ope(const S &a, const S &b) {
    if (a.len == 0) { return b; }
    if (b.len == 0) { return a; }
    S res;
    res.sum = a.sum + b.sum;
    res.max_prf = max(a.max_prf, a.sum + b.max_prf);
    res.max_suf = max(b.max_suf, b.sum + a.max_suf);
    res.max_sum = max({a.max_sum, b.max_sum, a.max_suf + b.max_prf});
    res.min_prf = min(a.min_prf, a.sum + b.min_prf);
    res.min_suf = min(b.min_suf, b.sum + a.min_suf);
    res.min_sum = min({a.min_sum, b.min_sum, a.min_suf + b.min_prf});
    res.len = a.len + b.len;
    return res;
}

static constexpr inline S map(F f, const S &a) {
    if (!f.to_set || a.len == 0) { return a; }
    S res = a;
    res.sum = f.set_upd * a.len;
    res.max_prf = res.max_suf = res.max_sum = f.set_upd > 0 ? res.sum : f.set_upd;
    res.min_prf = res.min_suf = res.min_sum = f.set_upd < 0 ? res.sum : f.set_upd;
    return res;
}

static constexpr inline F cmp(F f, F g) { return f.to_set ? f : g; }
}

struct MonSet {
    struct S { /* states */ lng len; };
    struct F {
        bool to_set; lng set_upd;
        bool operator==(const F&) const = default;
    };

    static constexpr inline S idS() { return {0}; }
    static constexpr inline F idF() { return {false, 0}; }
    static constexpr inline S defR(lng l, lng r) { return {r - l + 1}; }
    static constexpr inline S init(lng i, lng x) { return {1}; }

    static constexpr inline S ope(const S &a, const S &b) {
        if (a.len == 0) { return b; }
        if (b.len == 0) { return a; }
        S res;
        // ope states
        res.len = a.len + b.len;
        return res;
    }

    static constexpr inline S map(F f, const S &a) {
        if (!f.to_set || a.len == 0) { return a; }
        S res = a;
        // map states
        return res;
    }
}

```

```

static constexpr inline F cmp(F f, F g) { return f.to_set ? f : g; }
}

2.4 Segment Tree

// S: O(n), U: O(log(n)), Q: O(log(n)), M: O(n)
template<typename Mon>
struct SegTree {
    using S = typename Mon::S;
    using F = typename Mon::F;

    int n;
    vector<S> tree;
    vector<F> lazy;

    template<typename T>
    SegTree(std::span<const T> &a) : n(a.size()),
        tree(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1))),
            Mon::idS()),
        lazy(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1))),
            Mon::idF()) {
        if (n > 0) { build(1, 0, n - 1, a); }
    }

    template<typename T>
    void build(int i, int l, int r, std::span<const T> &a) {
        if (l == r) { tree[i] = Mon::init(l, a[l]); return; }
        int md = std::midpoint(l, r);
        build(2 * i, l, md, a);
        build(2 * i + 1, md + 1, r, a);
        tree[i] = Mon::ope(tree[2 * i], tree[2 * i + 1]);
    }

    inline void apply(int i, F f) {
        tree[i] = Mon::map(f, tree[i]);
        lazy[i] = Mon::cmp(f, lazy[i]);
    }

    inline void push(int i) {
        if (lazy[i] == Mon::idF()) { return; }
        apply(2 * i, lazy[i]);
        apply(2 * i + 1, lazy[i]);
        lazy[i] = Mon::idF();
    }

    void update(int i, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) { apply(i, f); return; }
        push(i);
        int md = std::midpoint(l, r);
        if (ql <= md) { _update(2 * i, l, md, ql, qr, f); }
        if (qr > md) { _update(2 * i + 1, md + 1, r, ql, qr, f); }
        tree[i] = Mon::ope(tree[2 * i], tree[2 * i + 1]);
    }

    void update(int l, int r, F f) {
        l = max(l, 0); r = min(r, n - 1);
        if (l <= r) { _update(1, 0, n - 1, l, r, f); }
    }

    template<class G>
    int _maxR(int i, int l, int r, int ql, G &g, S &acc) {
        if (r < ql) { return -1; }
        if (ql <= l) {
            S nac = Mon::ope(acc, tree[i]);
            if (g(nac)) { acc = nac; return -1; }
        }
        if (l == r) { return l - 1; }
        push(i);
        int md = std::midpoint(l, r);
        int res = _maxR(2 * i, l, md, ql, g, acc);
        if (res != -1) { return res; }
        return _maxR(2 * i + 1, md + 1, r, ql, g, acc);
    }

    template<class G>
    int maxR(int l, G g) { // Returns max r in [l, n - 1] where
        g(query(l, r)) is true.
    }
}

```

```

    l = max(l, 0);
    if (l > n - 1) { return l - 1; }
    S acc = Mon::idS();
    int res = _maxR(1, 0, n - 1, l, g, acc);
    return res == -1 ? n - 1 : res;
}

template<class G>
int _minL(int i, int l, int r, int qr, G &g, S &acc) {
    if (l > qr) { return -1; }
    if (r <= qr) {
        S nac = Mon::ope(tree[i], acc);
        if (g(nac)) { acc = nac; return -1; }
    }
    if (l == r) { return l + 1; }
    push(i);
    int md = std::midpoint(l, r);
    int res = _minL(2 * i + 1, md + 1, r, qr, g, acc);
    if (res != -1) { return res; }
    return _minL(2 * i, l, md, qr, g, acc);
}

template<class G>
int minL(int r, G g) { // Returns min l in [0, r] where g(query(l,
    r)) is true.
    r = min(r, n - 1);
    if (r < 0) { return 0; }
    S acc = Mon::idS();
    int res = _minL(1, 0, n - 1, r, g, acc);
    return res == -1 ? 0 : res;
}

S _query(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i]; }
    push(i);
    int md = std::midpoint(l, r);
    if (qr <= md) { return _query(2 * i, l, md, ql, qr); }
    if (ql > md) { return _query(2 * i + 1, md + 1, r, ql, qr); }
    return Mon::ope(_query(2 * i, l, md, ql, qr), _query(2 * i + 1,
        md + 1, r, ql, qr));
}

S query(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return Mon::idS(); }
    return _query(1, 0, n - 1, l, r);
}

S queryAll() { return tree[1]; }
}

```

2.5 2D Segment Tree

```

// S: O(n^2), U: O(log(n)^2), Q: O(log(n)^2), M: O(n^2)
template<typename Mon>
struct SegTree2D {
    using S = typename Mon::S;
    using F = typename Mon::F;

    int n, m;
    vector<vector<S>> tree;

    template<typename T>
    SegTree2D(const vector<vector<T>> &a) : n(a.size()), m(a.empty() ?
        0 : a[0].size()) {
        if (n == 0 || m == 0) { return; }
        tree.assign(4 * n, vector<S>(4 * m, Mon::idS()));
        build(1, 0, n - 1, a);
    }

    template<typename T>
    void _build(int i, int l1, int r1, int j, int l2, int r2, const
        vector<vector<T>> &a) {
        if (l2 == r2) {
            if (l1 == r1) { tree[i][j] = Mon::init(l2, a[l1][l2]); }
            else { tree[i][j] = Mon::ope(tree[2 * i][j],
                tree[2 * i + 1][j]); }
        }
    }
}

```

```

    return;
}
int md = 12 + (r2 - 12) / 2;
_build(i, l1, r1, 2 * j, 12, md, a);
_build(i, l1, r1, 2 * j + 1, md + 1, r2, a);
tree[i][j] = Mon::ope(tree[i][2 * j], tree[i][2 * j + 1]);
}
template<typename T>
void build(int i, int l1, int r1, const vector<vector<T>> &a) {
    if (l1 != r1) {
        int md = l1 + (r1 - l1) / 2;
        build(2 * i, l1, md, a);
        build(2 * i + 1, md + 1, r1, a);
    }
    _build(i, l1, r1, 1, 0, m - 1, a);
}

void _update_j(int i, int l1, int r1, int j, int l2, int r2, int qj, F f) {
    if (l2 == r2) {
        if (l1 == r1) { tree[i][j] = Mon::map(f, tree[i][j]); }
        else { tree[i][j] = Mon::ope(tree[2 * i][j], tree[2 * i + 1][j]); }
        return;
    }
    int md = 12 + (r2 - 12) / 2;
    if (qj <= md) { _update_j(i, l1, r1, 2 * j, 12, md, qj, f); }
    else { _update_j(i, l1, r1, 2 * j + 1, md + 1, r2, qj, f); }
    if (l1 == r1) { tree[i][j] = Mon::ope(tree[i][2 * j], tree[i][2 * j + 1]); }
    else { tree[i][j] = Mon::ope(tree[2 * i][j], tree[2 * i + 1][j]); }
}

void _update_i(int i, int l1, int r1, int qi, int qj, F f) {
    if (l1 != r1) {
        int md = l1 + (r1 - l1) / 2;
        if (qi <= md) { _update_i(2 * i, l1, md, qi, qj, f); }
        else { _update_i(2 * i + 1, md + 1, r1, qi, qj, f); }
    }
    _update_j(i, l1, r1, 1, 0, m - 1, qj, f);
}

void update(int i, int j, F f) {
    if (i >= 0 && i < n && j >= 0 && j < m) { _update_i(1, 0, n - 1, i, j, f); }
}

S _query_j(int i, int j, int l2, int r2, int qj1, int qj2) {
    if (qj1 <= 12 && r2 <= qj2) { return tree[i][j]; }
    int md = 12 + (r2 - 12) / 2;
    if (qj2 <= md) { return _query_j(i, 2 * j, 12, md, qj1, qj2); }
    if (qj1 > md) { return _query_j(i, 2 * j + 1, md + 1, r2, qj1, qj2); }
    return Mon::ope(_query_j(i, 2 * j, 12, md, qj1, qj2), _query_j(i, 2 * j + 1, md + 1, r2, qj1, qj2));
}

S _query_i(int i, int l1, int r1, int qi1, int qj1, int qi2, int qj2) {
    if (qi1 <= l1 && r1 <= qi2) { return _query_j(i, 1, 0, m - 1, qj1, qj2); }
    int md = l1 + (r1 - l1) / 2;
    if (qi2 <= md) { return _query_i(2 * i, l1, md, qi1, qj1, qi2, qj2); }
    if (qi1 > md) { return _query_i(2 * i + 1, md + 1, r1, qi1, qj1, qi2, qj2); }
    return Mon::ope(_query_i(2 * i, l1, md, qi1, qj1, qi2, qj2), _query_i(2 * i + 1, md + 1, r1, qi1, qj1, qi2, qj2));
}

```

```

}
S query(int i1, int j1, int i2, int j2) {
    i1 = max(i1, 0); i2 = min(i2, n - 1);
    j1 = max(j1, 0); j2 = min(j2, m - 1);
    if (i1 > i2 || j1 > j2) { return Mon::idS(); }
    return _query_i(1, 0, n - 1, i1, j1, i2, j2);
}

S queryAll() {
    if (n == 0 || m == 0) { return Mon::idS(); }
    return tree[1][1];
}

};

2.6 Dynamic Segment Tree

// S: O(q * log(n)), U: O(log(n)), Q: O(log(n)), M: O(q * log(n))
template<typename Mon>
struct DynSegTree {
    using S = typename Mon::S;
    using F = typename Mon::F;

    struct Node {
        S val; F lzy; int lc = -1, rc = -1;
        Node(lng l, lng r) : val(Mon::defR(l, r)), lzy(Mon::idF()) {}
    };
    lng n;
    int root = -1;
    vector<Node> tree;

    DynSegTree(lng N) : n(N) {
        tree.reserve(500'000);
        if (n > 0) { root = createNode(0, n - 1); }
    }
    template<typename T>
    DynSegTree(std::span<const T> &a) : n(a.size()) {
        tree.reserve(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1LL))));
        if (n > 0) { root = build(0, n - 1, a); }
    }
    template<typename T>
    int build(lng l, lng r, std::span<const T> &a) {
        int cur = createNode(l, r);
        if (l == r) { tree[cur].val = Mon::init(l, a[l]); return cur; }
        lng md = std::midpoint(l, r);
        int nlc = build(l, md, a); tree[cur].lc = nlc;
        int nrc = build(md + 1, r, a); tree[cur].rc = nrc;
        tree[cur].val = Mon::ope(tree[nlc].val, tree[nrc].val);
        return cur;
    }
    int createNode(lng l, lng r) {
        tree.emplace_back(l, r);
        return tree.size() - 1;
    }

    inline void apply(int i, F f) {
        tree[i].val = Mon::map(f, tree[i].val);
        tree[i].lzy = Mon::cmp(f, tree[i].lzy);
    }

    inline void push(int i, lng l, lng r) {
        if (tree[i].lzy == Mon::idF()) { return; }
        lng md = std::midpoint(l, r);
        if (tree[i].lc == -1) { int nlc = createNode(l, md); tree[i].lc = nlc; }
        if (tree[i].rc == -1) { int nrc = createNode(md + 1, r); tree[i].rc = nrc; }
        apply(tree[i].lc, tree[i].lzy);
        apply(tree[i].rc, tree[i].lzy);
        tree[i].lzy = Mon::idF();
    }

    void _update(int i, lng l, lng r, lng ql, lng qr, F f) {
        if (ql <= l && r <= qr) { apply(i, f); return; }
        push(i, l, r);
        lng md = std::midpoint(l, r);

```

```

        if (ql <= md) {
            if (tree[i].lc == -1) { int nlc = createNode(l, md); tree[i].lc = nlc; }
            _update(tree[i].lc, l, md, ql, qr, f);
        }
        if (qr > md) {
            if (tree[i].rc == -1) { int nrc = createNode(md + 1, r); tree[i].rc = nrc; }
            _update(tree[i].rc, md + 1, r, ql, qr, f);
        }
        S l_val = tree[i].lc != -1 ? tree[tree[i].lc].val : Mon::defR(l, md);
        S r_val = tree[i].rc != -1 ? tree[tree[i].rc].val : Mon::defR(md + 1, r);
        tree[i].val = Mon::ope(l_val, r_val);
    }
    void update(lng l, lng r, F f) {
        l = max(l, 0LL); r = min(r, n - 1);
        if (l <= r) { _update(root, 0, n - 1, l, r, f); }
    }

    template<class G>
    lng _maxR(int i, lng l, lng r, lng ql, G &g, S &acc) {
        if (r < ql) { return -1; }
        if (ql <= l) {
            S nac = Mon::ope(acc, i == -1 ? Mon::defR(l, r) : tree[i].val);
            if (g(nac)) { acc = nac; return -1; }
        }
        if (l == r) { return l - 1; }
        if (i != -1) { push(i, l, r); }
        lng md = std::midpoint(l, r);
        lng res = _maxR(i == -1 ? -1 : tree[i].lc, l, md, ql, g, acc);
        if (res != -1) { return res; }
        return _maxR(i == -1 ? -1 : tree[i].rc, md + 1, r, ql, g, acc);
    }
    template<class G>
    lng maxR(lng l, G g) { // Returns max r in [l, n - 1] where g(query(l, r)) is true.
        if (l < 0 || l > n - 1) { return l - 1; }
        S acc = Mon::idS();
        lng res = _maxR(root, 0, n - 1, l, g, acc);
        return res == -1 ? n - 1 : res;
    }

    template<class G>
    lng _minL(int i, lng l, lng r, lng qr, G &g, S &acc) {
        if (l > qr) { return -1; }
        if (r <= qr) {
            S nac = Mon::ope(i == -1 ? Mon::defR(l, r) : tree[i].val, acc);
            if (g(nac)) { acc = nac; return -1; }
        }
        if (l == r) { return l + 1; }
        if (i != -1) { push(i, l, r); }
        lng md = std::midpoint(l, r);
        lng res = _minL(i == -1 ? -1 : tree[i].rc, md + 1, r, qr, g, acc);
        if (res != -1) { return res; }
        return _minL(i == -1 ? -1 : tree[i].lc, l, md, qr, g, acc);
    }
    template<class G>
    lng minL(lng r, G g) { // Returns min l in [0, r] where g(query(l, r)) is true.
        if (r < 0 || r > n - 1) { return r + 1; }
        S acc = Mon::idS();
        lng res = _minL(root, 0, n - 1, r, g, acc);
        return res == -1 ? 0 : res;
    }

    S _query(int i, lng l, lng r, lng ql, lng qr) {
        if (i == -1) {

```



```

lng _maxR(int i, lng l, lng r, lng ql, G &g, S &acc, F tac) {
    if (r < ql) { return -1; }
    if (ql <= l) {
        S val = i == -1 ? Mon::defR(l, r) : tree[i].val; val =
            Mon::map(tac, val);
        S nac = Mon::ope(acc, val);
        if (g(nac)) { acc = nac; return -1; }
    }
    if (l == r) { return l - 1; }
    tac = i == -1 ? tac : Mon::cmp(tac, tree[i].lzy);
    lng md = std::midpoint(l, r);
    lng res = _maxR(i == -1 ? -1 : tree[i].lc, l, md, ql, g, acc,
        tac);
    if (res != -1) { return res; }
    return _maxR(i == -1 ? -1 : tree[i].rc, md + 1, r, ql, g, acc,
        tac);
}

template<class G>
lng maxR(int ver, lng l, G g) { // Returns max r in [l, n - 1]
    where g(query(l, r)) is true.
    if (l < 0 || l > n - 1) { return l - 1; }
    S acc = Mon::idS();
    lng res = _maxR(roots[ver], 0, n - 1, l, g, acc, Mon::idF());
    return res == -1 ? n - 1 : res;
}

template<class G>
lng _minL(int i, lng l, lng r, lng qr, G &g, S &acc, F tac) {
    if (l > qr) { return -1; }
    if (r <= qr) {
        S val = i == -1 ? Mon::defR(l, r) : tree[i].val; val =
            Mon::map(tac, val);
        S nac = Mon::ope(val, acc);
        if (g(nac)) { acc = nac; return -1; }
    }
    if (l == r) { return l + 1; }
    tac = i == -1 ? tac : Mon::cmp(tac, tree[i].lzy);
    lng md = std::midpoint(l, r);
    lng res = _minL(i == -1 ? -1 : tree[i].rc, md + 1, r, qr, g,
        acc, tac);
    if (res != -1) { return res; }
    return _minL(i == -1 ? -1 : tree[i].lc, l, md, qr, g, acc,
        tac);
}

template<class G>
lng minL(int ver, lng r, G g) { // Returns min l in [0, r] where
    g(query(l, r)) is true.
    if (r < 0 || r > n - 1) { return r + 1; }
    S acc = Mon::idS();
    lng res = _minL(roots[ver], 0, n - 1, r, g, acc, Mon::idF());
    return res == -1 ? 0 : res;
}

S _query(int i, lng l, lng r, lng ql, lng qr, F acc) {
    if (i == -1) {
        if (max(l, ql) > min(r, qr)) { return Mon::idS(); }
        return Mon::map(acc, Mon::defR(max(l, ql), min(r, qr)));
    }
    if (ql <= l && r <= qr) { return Mon::map(acc, tree[i].val); }
    F nac = Mon::cmp(acc, tree[i].lzy);
    lng md = std::midpoint(l, r);
    if (qr <= md) { return _query(tree[i].lc, l, md, ql, qr,
        nac); }
    if (ql > md) { return _query(tree[i].rc, md + 1, r, ql, qr,
        nac); }
    return Mon::ope(_query(tree[i].lc, l, md, ql, qr, nac),
        _query(tree[i].rc, md + 1, r, ql, qr, nac));
}

S query(int ver, lng l, lng r) {
    l = max(l, 0LL); r = min(r, n - 1);
    if (l > r) { return Mon::idS(); }
    return _query(roots[ver], 0, n - 1, l, r, Mon::idF());
}

```

```

// S: O(q * log(n)), U: O(log(n)), Q: O(log(n)), M: O(q * log(n))
template<typename Mon>
struct PerSegTree {
    using S = typename Mon::S;
    using F = typename Mon::F;

    struct Node {
        S val; F lzy; int lc = -1, rc = -1;
        Node(lng l, lng r) : val(Mon::defR(l, r)), lzy(Mon::idF()) {}
    };
    lng n;
    vector<int> roots;
    vector<Node> tree;

    PerSegTree(lng N) : n(N) {
        tree.reserve(10'000'000);
        roots.push_back(cloneNode(-1, 0, n - 1));
    }

    template<typename T>
    PerSegTree(std::span<const T> &a) : n(a.size()) {
        tree.reserve(10'000'000);
        roots.push_back(build(0, n - 1, a));
    }

    template<typename T>
    int build(lng l, lng r, std::span<const T> &a) {
        int cur = cloneNode(-1, l, r);
        if (l == r) { tree[cur].val = Mon::init(l, a[l]); return cur; }
        lng md = std::midpoint(l, r);
        int nlc = build(l, md, a); tree[cur].lc = nlc;
        int nrc = build(md + 1, r, a); tree[cur].rc = nrc;
        tree[cur].val = Mon::ope(tree[nlc].val, tree[nrc].val);
        return cur;
    }

    int cloneNode(int i, lng l, lng r) {
        if (i == -1) { tree.emplace_back(l, r); }
        else { Node tmp = tree[i]; tree.push_back(tmp); }
        return tree.size() - 1;
    }

    inline void apply(int i, F f) {
        tree[i].val = Mon::map(f, tree[i].val);
        tree[i].lzy = Mon::cmp(f, tree[i].lzy);
    }

    inline void push(int i, lng l, lng r) {
        if (tree[i].lzy == Mon::idF()) { return; }
        lng md = std::midpoint(l, r);
        int nlc = cloneNode(tree[i].lc, l, md); tree[i].lc = nlc;
        int nrc = cloneNode(tree[i].rc, md + 1, r); tree[i].rc = nrc;
        apply(tree[i].lc, tree[i].lzy);
        apply(tree[i].rc, tree[i].lzy);
    }
};

```

2.8 Kth Smallest Persistent Segment Tree

```

    if (l_lrc == -1) { cnt -= segt->tree[l_lrc].val.cnt; }
    int r_lrc = r_i == -1 ? -1 : segt->tree[r_i].lrc;
    if (r_lrc != -1) { cnt += segt->tree[r_lrc].val.cnt; }
    int md = std::midpoint(l, r);
    if (cnt >= k) { return _kthMin(l_lrc, r_lrc, l, md, k); }
    int l_rc = l_i == -1 ? -1 : segt->tree[l_i].rc;
    int r_rc = r_i == -1 ? -1 : segt->tree[r_i].rc;
    return _kthMin(l_rc, r_rc, md + 1, r, k - cnt);
}

lng kthMin(int l, int r, int k) {
    assert(0 <= l && l <= r && r < n);
    assert(1 <= k && k <= r - l + 1);
    int ord = _kthMin(segt->roots[l], segt->roots[r + 1], 0,
        v.size() - 1, k);
    return v[ord];
}

lng _kthSum(int l_i, int r_i, int l, int r, int k) {
    if (k == 0) { return 0; }
    if (l == r) { return lng(k) * v[l]; }
    int l_cnt = 0; lng l_sum = 0;
    int l_lrc = l_i == -1 ? -1 : segt->tree[l_i].lrc;
    if (l_lrc != -1) {
        l_cnt -= segt->tree[l_lrc].val.cnt;
        l_sum -= segt->tree[l_lrc].val.sum;
    }
    int r_lrc = r_i == -1 ? -1 : segt->tree[r_i].lrc;
    if (r_lrc != -1) {
        l_cnt += segt->tree[r_lrc].val.cnt;
        l_sum += segt->tree[r_lrc].val.sum;
    }
    int md = std::midpoint(l, r);
    if (l_cnt >= k) { return _kthSum(l_lrc, r_lrc, l, md, k); }
    else {
        int l_rc = l_i == -1 ? -1 : segt->tree[l_i].rc;
        int r_rc = r_i == -1 ? -1 : segt->tree[r_i].rc;
        return l_sum + _kthSum(l_rc, r_rc, md + 1, r, k - l_cnt);
    }
}

lng kthSum(int l, int r, int k) {
    assert(0 <= l && l <= r && r < n);
    assert(0 <= k && k <= r - l + 1);
    return _kthSum(segt->roots[l], segt->roots[r + 1], 0, v.size()
        - 1, k);
}

lng maxLeq(int l, int r, lng x) {
    assert(0 <= l && l <= r && r < n);
    int cnt = _cntLeq(l, r, x);
    if (cnt == 0) { return -INF64; }
    return kthMin(l, r, cnt);
}

lng minGeq(int l, int r, lng x) {
    assert(0 <= l && l <= r && r < n);
    int cnt = _cntLeq(l, r, x - 1);
    if (cnt == r - l + 1) { return INF64; }
    return kthMin(l, r, cnt + 1);
}
}

};

2.9 Segment Tree Beats

// S: O(n), U: O(log(n)^2), Q: O(log(n)), M: O(n)
struct SegTreeBeats {
    struct Node {
        lng sum;
        lng mx1, mx2; int mxc;
        lng mn1, mn2; int mnc;
        bool to_set; lng add_upd, set_upd;
    };
    int n;
    vector<Node> tree;

    template<typename T>

```

2.9 Segment Tree Beats

```
// S: O(n), U: O(log(n)^2), Q: O(log(n)), M: O(n)
struct SegTreeBeats {
    struct Node {
        lng sum;
        lng mx1, mx2; int mxc;
        lng mn1, mn2; int mnc;
        bool to_set; lng add_upd, set_upd;
    };
    int n;
    vector<Node> tree;

    template<typename T>
```

```

SegTreeBeats<std::span<const T> &a>: n(a.size()),
tree(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1)))) {
    if (n > 0) { build(1, 0, n - 1, a); }
}

template<typename T>
void build(int i, int l, int r, std::span<const T> &a) {
    if (l == r) {
        tree[i].sum = a[l];
        tree[i].mx1 = a[l]; tree[i].mx2 = -INF64; tree[i].mxc = 1;
        tree[i].mn1 = a[l]; tree[i].mn2 = INF64; tree[i].mnc = 1;
        tree[i].to_set = false; tree[i].add_upd = 0;
        tree[i].set_upd = 0;
        return;
    }
    int md = std::midpoint(l, r);
    build(2 * i, l, md, a);
    build(2 * i + 1, md + 1, r, a);
    pull(i);
    tree[i].to_set = false; tree[i].add_upd = 0; tree[i].set_upd = 0;
}

inline void pull(int i) {
    int lc = 2 * i, rc = 2 * i + 1;
    tree[i].sum = tree[lc].sum + tree[rc].sum;
    if (tree[lc].mx1 == tree[rc].mx1) {
        tree[i].mx1 = tree[lc].mx1;
        tree[i].mx2 = max(tree[lc].mx2, tree[rc].mx2);
        tree[i].mxc = tree[lc].mxc + tree[rc].mxc;
    } else if (tree[lc].mx1 > tree[rc].mx1) {
        tree[i].mx1 = tree[lc].mx1;
        tree[i].mx2 = max(tree[lc].mx2, tree[rc].mx1);
        tree[i].mxc = tree[lc].mxc;
    } else {
        tree[i].mx1 = tree[rc].mx1;
        tree[i].mx2 = max(tree[lc].mx1, tree[rc].mx2);
        tree[i].mxc = tree[rc].mxc;
    }
    if (tree[lc].mn1 == tree[rc].mn1) {
        tree[i].mn1 = tree[lc].mn1;
        tree[i].mn2 = min(tree[lc].mn2, tree[rc].mn2);
        tree[i].mnc = tree[lc].mnc + tree[rc].mnc;
    } else if (tree[lc].mn1 < tree[rc].mn1) {
        tree[i].mn1 = tree[lc].mn1;
        tree[i].mn2 = min(tree[lc].mn2, tree[rc].mn1);
        tree[i].mnc = tree[lc].mnc;
    } else {
        tree[i].mn1 = tree[rc].mn1;
        tree[i].mn2 = min(tree[lc].mn1, tree[rc].mn2);
        tree[i].mnc = tree[rc].mnc;
    }
}

inline void addApply(int i, int len, lng x) {
    tree[i].sum += x * len;
    tree[i].mx1 += x; if (tree[i].mx2 != -INF64) { tree[i].mx2 += x; }
    tree[i].mn1 += x; if (tree[i].mn2 != INF64) { tree[i].mn2 += x; }
    if (tree[i].to_set) { tree[i].set_upd += x; }
    else { tree[i].add_upd += x; }
}

inline void chmaxApply(int i, lng x) {
    if (x <= tree[i].mn1) { return; }
    tree[i].sum += (x - tree[i].mn1) * tree[i].mnc;
    if (tree[i].mx1 == tree[i].mn1) { tree[i].mx1 = x; }
    if (tree[i].mx2 == tree[i].mn1) { tree[i].mx2 = x; }
    tree[i].mn1 = x;
}

inline void chminApply(int i, lng x) {
    if (x >= tree[i].mx1) { return; }
    tree[i].sum -= (tree[i].mx1 - x) * tree[i].mxc;
    if (tree[i].mn1 == tree[i].mx1) { tree[i].mn1 = x; }
}

```



```

    if (tree[i].mn2 == tree[i].mx1) { tree[i].mn2 = x; }
    tree[i].mx1 = x;
}

inline void setApply(int i, int len, lng x) {
    tree[i].sum = x * len;
    tree[i].mx1 = x; tree[i].mx2 = -INF64; tree[i].mxc = len;
    tree[i].mn1 = x; tree[i].mn2 = INF64; tree[i].mnc = len;
    tree[i].to_set = true; tree[i].add_upd = 0; tree[i].set_upd =
x;
}

inline void push(int i, int l, int r) {
    int md = std::midpoint(l, r);
    int lc = 2 * i, rc = 2 * i + 1;
    if (tree[i].to_set) {
        setApply(lc, md - l + 1, tree[i].set_upd);
        setApply(rc, r - md, tree[i].set_upd);
        tree[i].to_set = false;
    }
    if (tree[i].add_upd != 0) {
        addApply(lc, md - l + 1, tree[i].add_upd);
        addApply(rc, r - md, tree[i].add_upd);
        tree[i].add_upd = 0;
    }
    if (tree[i].mx1 < tree[lc].mx1) { chminApply(lc, tree[i].mx1); }
    if (tree[i].mx1 < tree[rc].mx1) { chminApply(rc, tree[i].mx1); }
    if (tree[i].mn1 > tree[lc].mn1) { chmaxApply(lc, tree[i].mn1); }
    if (tree[i].mn1 > tree[rc].mn1) { chmaxApply(rc, tree[i].mn1); }
}

void _addUpdate(int i, int l, int r, int ql, int qr, lng x) {
    if (ql <= l && r <= qr) { addApply(i, r - l + 1, x); return; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    if (ql <= md) { _addUpdate(2 * i, l, md, ql, qr, x); }
    if (qr > md) { _addUpdate(2 * i + 1, md + 1, r, ql, qr, x); }
    pull(i);
}

void _chmaxUpdate(int i, int l, int r, int ql, int qr, lng x) {
    if (x <= tree[i].mn1) { return; }
    if (ql <= l && r <= qr && x < tree[i].mn2) { chmaxApply(i, x);
return; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    if (ql <= md) { _chmaxUpdate(2 * i, l, md, ql, qr, x); }
    if (qr > md) { _chmaxUpdate(2 * i + 1, md + 1, r, ql, qr, x); }
    pull(i);
}

void _chminUpdate(int i, int l, int r, int ql, int qr, lng x) {
    if (x >= tree[i].mx1) { return; }
    if (ql <= l && r <= qr && x > tree[i].mx2) { chminApply(i, x);
return; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    if (ql <= md) { _chminUpdate(2 * i, l, md, ql, qr, x); }
    if (qr > md) { _chminUpdate(2 * i + 1, md + 1, r, ql, qr, x); }
    pull(i);
}

void _setUpdate(int i, int l, int r, int ql, int qr, lng x) {
    if (ql <= l && r <= qr) { setApply(i, r - l + 1, x); return; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    if (ql <= md) { _setUpdate(2 * i, l, md, ql, qr, x); }
    if (qr > md) { _setUpdate(2 * i + 1, md + 1, r, ql, qr, x); }
    pull(i);
}

```

```

void addUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _addUpdate(1, 0, n - 1, l, r, x); }
}

void chminUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _chminUpdate(1, 0, n - 1, l, r, x); }
}

void chmaxUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _chmaxUpdate(1, 0, n - 1, l, r, x); }
}

void setUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _setUpdate(1, 0, n - 1, l, r, x); }
}

lng _maxQuery(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i].mx1; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    lng res = -INF64;
    if (ql <= md) { res = max(res, _maxQuery(2 * i, l, md, ql, qr)); }
    if (qr > md) { res = max(res, _maxQuery(2 * i + 1, md + 1, r, ql, qr)); }
    return res;
}

lng _minQuery(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i].mn1; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    lng res = INF64;
    if (ql <= md) { res = min(res, _minQuery(2 * i, l, md, ql, qr)); }
    if (qr > md) { res = min(res, _minQuery(2 * i + 1, md + 1, r, ql, qr)); }
    return res;
}

lng _sumQuery(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i].sum; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    lng res = 0;
    if (ql <= md) { res += _sumQuery(2 * i, l, md, ql, qr); }
    if (qr > md) { res += _sumQuery(2 * i + 1, md + 1, r, ql, qr); }
    return res;
}

lng maxQuery(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return -INF64; }
    return _maxQuery(1, 0, n - 1, l, r);
}

lng minQuery(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return INF64; }
    return _minQuery(1, 0, n - 1, l, r);
}

lng sumQuery(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return 0; }
    return _sumQuery(1, 0, n - 1, l, r);
}

```

```
struct Point {
    T x, y;

    Point() : x(0), y(0) {}
    Point(T x, T y) : x(x), y(y) {}

    friend Point operator+(Point a, Point b) { a.x += b.x; a.y += b.y;
    return a; }
    friend Point operator-(Point a, Point b) { a.x -= b.x; a.y -= b.y;
    return a; }
    friend Point operator*(Point a, T b) { a.x *= b; a.y *= b; return
a; }
    friend Point operator*(T b, Point a) { a.x *= b; a.y *= b; return
a; }

    friend T dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
    friend T cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
};

using lng = Point<lng>;
using dpoint = Point<double>;

int sgn(lng x) { return x < 0 ? -1 : x > 0; }
```

3.2 Area of a Polygon

```
// T: O(n), M: O(1)
double polyArea(const vector<point> &ps) {
    double res = 0;
    for (int i = 0; i < ps.size(); i++) {
        point p1 = i > 0 ? ps[i - 1] : ps.back(), p2 = ps[i];
        res += (p1.x - p2.x) * (p1.y + p2.y);
    }
    return abs(res) / 2.0;
}
```

3.3 Intersection

```
// T: O(1), M: O(1)
bool isIntsc1D(lng a, lng b, lng c, lng d) {
    if (a > b) { std::swap(a, b); }
    if (c > d) { std::swap(c, d); }
    return max(a, c) <= min(b, d);
}

// T: O(1), M: O(1)
bool isIntscLinSeg(point a, point b, point c, point d) {
    if (cross(a - c, d - c) == 0 && cross(b - c, d - c) == 0) {
        return isIntsc1D(a.x, b.x, c.x, d.x) && isIntsc1D(a.y, b.y,
            c.y, d.y);}
    return sgn(cross(b - a, c - a)) != sgn(cross(b - a, d - a))
        && sgn(cross(d - c, a - c)) != sgn(cross(d - c, b - c));
}

// T: O(1), M: O(1)
pair<bool, dpoint> getIntscLineLine(double a1, double b1, double c1,
double a2, double b2, double c2) {
    double det = a1 * b2 - a2 * b1;
    if (abs(det) < 1e-9) { return {false, {}}; }
    return {true, {-(c1 * b2 - c2 * b1) / det, -(a1 * c2 - a2 * c1) /
        det}};
}

// T: O(1), M: O(1)
vector<dpoint> getIntscLineCircle(double r, double a, double b, double
c) { // ax + by + c = 0
    double d_sq = c * c / (a * a + b * b), x0 = -a * c / (a * a + b *
        b), y0 = -b * c / (a * a + b * b);
    if (d_sq > r * r + 1e-9) { return {}; }
    if (std::abs(d_sq - r * r) < 1e-9) { return {dpoint(x0, y0)}; }
    double d = std::sqrt((r * r - d_sq) / (a * a + b * b));
    return {dpoint(x0 + b * d, y0 - a * d), dpoint(x0 - b * d, y0 + a
        d)};
}
```

3 Geometry

3.1 Point

```
// T: O(1), M: O(1)
template<typename T>
```

```

return ps[i].x < ps[j].x || (ps[i].x == ps[j].x && ps[i].y <
ps[j].y);
});

auto mul = [&](lng a, lng b) -> lng { return (a - b) * (a - b); };
auto update = [&](int i, int j) -> void {
    lng d = mul(ps[i].x, ps[j].x) + mul(ps[i].y, ps[j].y);
    if (d < mn_d) { mn_d = d; res = {i, j}; }
};

auto solve = [&](auto &&solve, int l, int r) -> void {
    if (r - l <= 3) {
        for (int i = l; i < r; i++) {
            for (int j = i + 1; j < r; j++) {
                update(ids[i], ids[j]);
            }
        }
        std::sort(ids.begin() + l, ids.begin() + r,
            [&](int i, int j) { return ps[i].y < ps[j].y; });
        return;
    }

    int md = std::midpoint(l, r); lng md_x = ps[ids[md]].x;
    solve(solve, l, md);
    solve(solve, md, r);
    std::merge(ids.begin() + l, ids.begin() + md, ids.begin() + md,
        ids.begin() + r, tmp.begin(),
        [&](int i, int j) { return ps[i].y < ps[j].y; });
    std::copy(tmp.begin(), tmp.begin() + r - l, ids.begin() + l);

    int ti = 0;
    for (int i = l; i < r; i++) {
        if (mul(ps[ids[i]].x, md_x) < mn_d) {
            for (int j = ti - 1; j >= 0 && mul(ps[ids[i]].y,
                ps[tmp[j]].y) < mn_d; j--) {
                update(ids[i], tmp[j]);
            }
            tmp[ti++] = ids[i];
        }
    }
};

solve(solve, 0, n);
return res;
}

```

3.6 Get Minimum Enclosing Circle

```
// S: O(n), U: NA, Q: O(1), M: O(n)
struct WeizlMEC {
    using cdb = std::complex<double>;
    using mec = std::variant<array<cdb, 2>, array<cdb, 3>>;
    static inline std::mt19937_64
    rng{std::chrono::steady_clock::now().time_since_epoch().count()};

    double indicator(const mec &c, cdb p) {
        return std::visit([&](auto &&c) -> double {
            cdb a = c[0], b = c[1], i0 = (b - p) * std::conj(a - p);
            if constexpr (std::tuple_size_v<std::decay_t<decltype(c)>>
                == 2) {
                return std::real(i0);
            } else {
                cdb i2 = (a - c[2]) * std::conj(b - c[2]), i1 = i0 *
                i2;
                return std::imag(i2) < 0 ? -std::imag(i1) :
                std::imag(i1);
            }
        }, c);
    }
}
```

```
bool isInside(const mec &c, cdb p) { return indicator(c, p) <= 0; }

mec getMEC(vector<cdb> ps) {
    if (ps.empty()) { return array<cdb, 2>{0, 0}; }
    if (ps.size() == 1) { return array<cdb, 2>{ps[0], ps[0]}; }

    std::shuffle(ps.begin(), ps.end(), rng);
    mec c = array<cdb, 2>{ps[0], ps[1]};
    for (int i = 0; i < ps.size(); i++) {
        if (!isInside(c, ps[i])) { c = array<cdb, 2>{ps[i], ps[0]}; }
        for (int j = 0; j < i; j++) {
            if (!isInside(c, ps[j])) { c = array<cdb, 2>{ps[i], ps[j]}; }
            for (int k = 0; k < j; k++) {
                if (!isInside(c, ps[k])) { c = array<cdb, 3>{ps[i], ps[j],
                    ps[k]}; }
            }
        }
    }
    return c;
};
```

3.8 Li-Chao Tree

```
// S: O(n), U: O(log(n)) or O(log(n)^2), Q: O(log(n)), M: O(n * log(n))
struct LiChaoTree {
    struct Line {
        lng m, b;
        lng eval(lng x) const { return m * x + b; }
    };
    struct Node {
        Line mx_l, mn_l; int lc = -1, rc = -1;
        Node(Line Mx_l, Line Mn_l) : mx_l(Mx_l), mn_l(Mn_l) {}
    };
    lng mn_x, mx_x;
    int root = -1;
    vector<Node> tree;

    LiChaoTree(lng Mn_x, lng Mx_x) : mn_x(Mn_x), mx_x(Mx_x) {
        tree.reserve(2'500'000); }
    int createNode() {
        tree.emplace_back(Line{0, -INF64}, Line{0, INF64});
        return tree.size() - 1;
    }

    int _addLineMax(int i, lng l, lng r, Line y) {
        if (i == -1) { i = createNode(); }
        lng md = std::midpoint(l, r);
        bool l_bet = y.eval(l) > tree[i].mx_l.eval(l);
        bool m_bet = y.eval(md) > tree[i].mx_l.eval(md);
        if (m_bet) { std::swap(tree[i].mx_l, y); }
        if (l == r) { return i; }
    }
};
```

```

    if (l_bet != m_bet) { int nlc = _addLineMax(tree[i].lc, l,
md, y); tree[i].lc = nlc; }
    else { int nrc = _addLineMax(tree[i].rc, md + 1,
r, y); tree[i].rc = nrc; }
    return i;
}
int _addLineMin(int i, lng l, lng r, Line y) {
    if (i == -1) { i = createNode(); }
    lng md = std::midpoint(l, r);
    bool l_bet = y.eval(l) < tree[i].mn_l.eval(l);
    bool m_bet = y.eval(md) < tree[i].mn_l.eval(md);
    if (m_bet) { std::swap(tree[i].mn_l, y); }
    if (l == r) { return i; }
    if (l_bet != m_bet) { int nlc = _addLineMin(tree[i].lc, l,
md, y); tree[i].lc = nlc; }
    else { int nrc = _addLineMin(tree[i].rc, md + 1,
r, y); tree[i].rc = nrc; }
    return i;
}
int _addSegmMax(int i, lng l, lng r, lng ql, lng qr, Line y) {
    if (ql <= l && r <= qr) { return _addLineMax(i, l, r, y); }
    if (i == -1) { i = createNode(); }
    lng md = std::midpoint(l, r);
    if (ql <= md) { int nlc = _addSegmMax(tree[i].lc, l,
ql, qr, y); tree[i].lc = nlc; }
    if (qr > md) { int nrc = _addSegmMax(tree[i].rc, md + 1, r,
ql, qr, y); tree[i].rc = nrc; }
    return i;
}
int _addSegmMin(int i, lng l, lng r, lng ql, lng qr, Line y) {
    if (ql <= l && r <= qr) { return _addLineMin(i, l, r, y); }
    if (i == -1) { i = createNode(); }
    lng md = std::midpoint(l, r);
    if (ql <= md) { int nlc = _addSegmMin(tree[i].lc, l,
ql, qr, y); tree[i].lc = nlc; }
    if (qr > md) { int nrc = _addSegmMin(tree[i].rc, md + 1, r,
ql, qr, y); tree[i].rc = nrc; }
    return i;
}
void addLine(lng m, lng b) {
    root = _addLineMax(root, mn_x, mx_x, {m, b});
    root = _addLineMin(root, mn_x, mx_x, {m, b});
}
void addSegm(lng l, lng r, lng m, lng b) {
    if (l >= mn_x && l <= r && r <= mx_x) {
        root = _addSegmMax(root, mn_x, mx_x, l, r, {m, b});
        root = _addSegmMin(root, mn_x, mx_x, l, r, {m, b});
    }
}
lng _maxQuery(int i, lng l, lng r, lng x) {
    if (i == -1) { return -INF64; }
    lng res = tree[i].mx_l.eval(x);
    if (l == r) { return res; }
    lng md = std::midpoint(l, r);
    if (x <= md) { res = max(res, _maxQuery(tree[i].lc, l,
md, x)); }
    else { res = max(res, _maxQuery(tree[i].rc, md + 1, r,
x)); }
    return res;
}
lng _minQuery(int i, lng l, lng r, lng x) {
    if (i == -1) { return INF64; }
    lng res = tree[i].mn_l.eval(x);
    if (l == r) { return res; }
    lng md = std::midpoint(l, r);
    if (x <= md) { res = min(res, _minQuery(tree[i].lc, l,
md, x)); }
    else { res = min(res, _minQuery(tree[i].rc, md + 1, r,
x)); }
    return res;
}
}

```

```

lng maxQuery(lng x) {
    if (x < mn_x || x > mx_x) { return -INF64; }
    return _maxQuery(root, mn_x, mx_x, x);
}
lng minQuery(lng x) {
    if (x < mn_x || x > mx_x) { return INF64; }
    return _minQuery(root, mn_x, mx_x, x);
}
}
}

```

4 Graphs

4.1 Topological Sort

```

void toposort(int u) {
    visited[u] = true;
    for (auto v : graph[u])
        if (!visited[v]) {
            toposort(v);
        }
    stack.push(u); // the key difference between dfs and topological
sort
}

```

4.2 Cycle Find DFS

```

bool dfs(int u) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 0) {
            if (dfs(v))
                return true;
        } else if (color[v] == 1) {
            cycles.push_back({v, u});
            return true;
        }
    }
    color[u] = 2;
    return false;
}
void find_cycles() {
    for (int u = 0; u < n; u++) {
        if (color[u] == 0)
            dfs(u);
    }
    for (int i = 0; i < cycles.size(); i++) {
        auto& [start, end] = cycles[i];
        for (int v = end; v != start; v = prev[v]) {
            cycle_grp[v] = i+1;
        }
        auto it = find(adj[end].begin(), adj[end].end(), start);
        adj[end].erase(it);
        cycle_grp[start] = i+1;
    }
}
// S: O(n), U: O(a(n)), Q: O(a(n)), M: O(n)
struct DSU {
    int n, ncon;
    vector<int> par, siz, esz;
    vector<lng> dis;
    vector<bool> is_bip;
    DSU(int N) : n(N), ncon(N), par(n, 0), siz(n, 1), esz(n, 0), dis(n,
0), is_bip(n, true) {
        iota(par.begin(), par.end(), 0);
    }
}

```

```

bool uniteSets(int u, int v, lng w = 1) {
    int ru = findSet(u); int rv = findSet(v);
    if (ru == rv) {
        esz[ru]++;
        if ((dis[u] - dis[v] - w) & 1) { is_bip[ru] = false; }
        return false;
    }
    if (siz[ru] < siz[rv]) { std::swap(ru, rv); std::swap(u, v); w
= -w; }
    ncon--;
    par[rv] = ru; siz[ru] += siz[rv]; esz[ru] += esz[rv] + 1;
    dis[rv] = dis[u] - dis[v] - w;
    is_bip[ru] = is_bip[ru] && is_bip[rv];
    return true;
}
int findSet(int u) {
    if (u == par[u]) { return u; }
    int p = par[u];
    par[u] = findSet(p);
    dis[u] += dis[p];
    return par[u];
}
bool isSameSet(int u, int v) { return findSet(u) == findSet(v); }
}
}

```

4.4 Shortest Paths

4.4.1 Dijkstra

```

struct Dijkstra {
    int n;
    vector<bool> is_proc;
    vector<lng> dis;
    priority_queue<pair<lng, int>, vector<pair<lng, int>>,
greater<pair<lng, int>>> unproc;
    Dijkstra(int n) : n(n) {
        is_proc.resize(n + 1);
        dis.resize(n + 1, INF64);
    }
    void runDijkstra(const vector<int> &src, const vector<vector<int>>
&adjl) {
        fill(is_proc.begin(), is_proc.end(), false);
        fill(dis.begin(), dis.end(), INF64);
        priority_queue<pair<lng, int>, vector<pair<lng, int>>,
greater<pair<lng, int>>> empty;
        unproc.swap(empty);
        for (int u : src) {
            dis[u] = 0;
            unproc.push({dis[u], u});
        }
        while (!unproc.empty()) {
            int cur = unproc.top().se;
            unproc.pop();
            if (is_proc[cur]) {
                continue;
            }
            is_proc[cur] = true;
            for (int nxt : adjl[cur]) {
                if (dis[cur] + 1 < dis[nxt]) {
                    dis[nxt] = dis[cur] + 1;
                    unproc.push({dis[nxt], nxt});
                }
            }
        }
    }
};

```

```

void runDijkstra(const vector<int> &src, const
vector<vector<pair<int, lng>>> &adjl) {
    fill(is_proc.begin(), is_proc.end(), false);
    fill(dis.begin(), dis.end(), INF64);
    priority_queue<pair<lng, int>, vector<pair<lng, int>>,
greater<pair<lng, int>>> empty;
    unproc.swap(empty);

    for (int u : src) {
        dis[u] = 0;
        unproc.push({dis[u], u});
    }
    while (!unproc.empty()) {
        int cur = unproc.top().se;
        unproc.pop();

        if (is_proc[cur]) {
            continue;
        }
        is_proc[cur] = true;

        for (auto [nxt, w] : adjl[cur]) {
            if (dis[cur] + w < dis[nxt]) {
                dis[nxt] = dis[cur] + w;
                unproc.push({dis[nxt], nxt});
            }
        }
    }
};

```

4.4.2 Bellman-Ford

```

void bellman_ford(int src) {
    dist[src] = 0;

    for (int k = 0; k < n-1; k++) {
        for (int u = 0; u < n; u++) {
            for (auto e : adj[u]) {
                int v = e.second;
                ll w = e.first;
                if (dist[u] < INF) { // if graph has negative weights
                    dist[v] = min(dist[v], dist[u] + w);
                }
            }
        }
    }

    bool has_negative_cycle() { // call after running bellman-ford
        for (int u = 0; u < n; u++) {
            for (auto e : adj[u]) {
                int v = e.second;
                ll w = e.first;
                if (dist[v] > dist[u] + w)
                    return true;
            }
        }
        return false;
    }
};

```

4.4.3 Floyd-Warshall

```

void floyd_warshall() {
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
};

```

4.5 Trees

4.5.1 Lowest Common Ancestor

```

// S: O(n * log(n)), U: NA, Q: O(log(n)), M: O(n * log(n))
struct LCA {
    int n, l, root, tm;
    vector<int> dep, tin, tout;
    vector<vector<int>> up;

    LCA(int n, int rt, const vector<vector<int>> &adj) :
        n(n), root(rt), tm(0), dep(n, 0), tin(n, 0), tout(n, 0) {
        l = std::ceil(std::log2(max(n, 1)));
        up.assign(n, vector<int>(l + 1));

        auto dfs = [&](auto &&dfs, int u, int p, int d) -> void {
            dep[u] = d; tin[u] = ++tm;
            up[u][0] = p;
            for (int i = 1; i <= l; i++) { up[u][i] = up[up[u][i - 1]][i - 1]; }
            for (int v : adj[u]) {
                if (v != p) { dfs(dfs, v, u, d + 1); }
            }
            tout[u] = ++tm;

            dfs(dfs, root, root, 0);
        };

        bool isAnc(int u, int v) const { return tin[u] <= tin[v] && tout[u]
>= tout[v]; }
        int getDis(int u, int v) const { return dep[u] + dep[v] - 2 *
dep[getLCA(u, v)]; }
        int getKthAncestor(int u, int k) const {
            if (k <= 0) { return u; }
            if (k > dep[u]) { return -1; }
            for (int i = l; i >= 0; i--) {
                if (k & (1 << i)) { u = up[u][i]; }
            }
            return u;
        }
        int getLCA(int u, int v) const {
            if (isAnc(u, v)) { return u; }
            if (isAnc(v, u)) { return v; }
            for (int i = l; i >= 0; i--) {
                if (!isAnc(up[u][i], v)) { u = up[u][i]; }
            }
            return up[u][0];
        }
};

```

4.5.2 Heavy Light Decomposition

```

// S: O(n), U: O(log(n)) or O(log(n)^2), Q: O(log(n)) or O(log(n)^2), M:
O(n)
struct HLD {
    int n, tm = 0;
    vector<int> dep, par, siz, head, in, out, rev;
    vector<vector<int>> adj;

    HLD(int N) : n(N), dep(N), par(N, -1), siz(N), head(N), in(N),
out(N), rev(N), adj(N) {}

    void addEdge(int u, int v) { adj[u].push_back(v);
adj[v].push_back(u); }
    void build(int root = 0) { tm = 0; dfsSiz(root, -1, 0);
dfsHld(root, -1, root); }
    void dfsSiz(int u, int p, int d) {
        dep[u] = d; par[u] = p; siz[u] = 1;
        if (adj[u].size() > 1 && adj[u][0] == p) { std::swap(adj[u][0],
adj[u][1]); }
        for (int &v : adj[u]) {
            if (v == p) { continue; }
            dfsSiz(v, u, d + 1);
            siz[u] += siz[v];
            if (siz[v] > siz[adj[u][0]]) { std::swap(v, adj[u][0]); }
        }
    }
};

```

```

}
void dfsHld(int u, int p, int h) {
    head[u] = h; in[u] = tm++; rev[in[u]] = u;
    for (int v : adj[u]) {
        if (v == p) { continue; }
        dfsHld(v, u, (v == adj[u][0] ? h : v));
    }
    out[u] = tm - 1;
}

int getDis(int u, int v) { return dep[u] + dep[v] - 2 *
dep[getLCA(u, v)]; }
int getLCA(int u, int v) {
    while (head[u] != head[v]) { std::swap(u, v); }
    if (dep[head[u]] < dep[head[v]]) { std::swap(u, v); }
    u = par[head[u]];
    return dep[u] < dep[v] ? u : v;
}
int kthAnc(int u, int k) {
    if (dep[u] < k) { return -1; }
    while (dep[u] - dep[head[u]] < k) {
        k -= dep[u] - dep[head[u]] + 1;
        u = par[head[u]];
    }
    return rev[in[u] - k];
}

template<typename F>
void processPath(int u, int v, bool is_edge, F f) {
    while (head[u] != head[v]) {
        if (dep[head[u]] < dep[head[v]]) { std::swap(u, v); }
        f(in[head[u]], in[u]);
        u = par[head[u]];
    }
    if (dep[u] > dep[v]) { std::swap(u, v); }
    if (u == v && is_edge) { return; }
    f(in[u] + is_edge, in[v]);
}

template<typename F>
void processSubt(int u, bool is_edge, F f) {
    if (in[u] == out[u] && is_edge) { return; }
    f(in[u] + is_edge, out[u]);
}

template<typename T>
vector<T> reorderArray(const vector<T> &a) {
    vector<T> res(n);
    for (int i = 0; i < n; i++) { res[in[i]] = a[i]; }
    return res;
}
};

```

4.5.3 Centroid Decomposition

```

struct tree {
    int n;
    vector<vector<int>> adj;
    vector<int> sz;
    vector<bool> done;

    tree(int n) : n(n) {
        adj.resize(n, {});
        sz.resize(n);
        done.resize(n);
    }

    int get_size(int u, int par = 0) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (!done[v] and v != par)
                sz[u] += get_size(v, u);
        }
        return sz[u];
    }
};

```

```

}
int get_centroid(int target, int u, int par = 0) {
    for (int v : adj[u])
        if (!done[v] and v != par and sz[v] >= target)
            return get_centroid(target, v, u);
    return u;
}
void centroid_decomp(int root = 0) {
    int u = get_centroid(get_size(root)/2, root);

    // do something

    done[u] = true;

    for (int v : adj[u]) {
        if (!done[v])
            centroid_decomp(v);
    }
}
};

```

4.6 Connectivity and Components

4.6.1 Bridge-Finding Algorithm

```

struct BridgeAlgo {
    int n, timer = 0;
    vector<bool> vst;
    vector<int> t_in, low;
    set<pair<int, int>> bridges;

    BridgeAlgo(int n, const vector<vector<int>> &adjl): n(n) {
        vst.resize(n);
        t_in.resize(n, -1);
        low.resize(n, -1);

        for (int i = 0; i < n; i++) {
            if (!vst[i]) {
                dfs(i, i, adjl);
            }
        }

        void dfs(int cur, int prv, const vector<vector<int>> &adjl) {
            vst[cur] = true;
            t_in[cur] = timer;
            low[cur] = timer;
            timer++;

            bool prv_skipped = false;
            for (int nxt : adjl[cur]) {
                if (nxt == prv && !prv_skipped) {
                    prv_skipped = true;
                    continue;
                }

                if (vst[nxt]) {
                    low[cur] = min(low[cur], t_in[nxt]);
                } else {
                    dfs(nxt, cur, adjl);

                    low[cur] = min(low[cur], low[nxt]);
                    if (low[nxt] > t_in[cur]) {
                        bridges.insert({min(cur, nxt), max(cur, nxt)});
                    }
                }
            }
        }
};

```

4.6.2 Block-Cut Tree

```

struct tree {
    int n;

```

```

    vector<int> is_cut, id;
    vector<vector<int>> adj;

    tree(int _n, vector<int> &cutpoints, vector<vector<int>> &comps) {
        n = cutpoints.size() + comps.size();
        l = ceil(log2(n));
        adj.resize(n);
        id.resize(_n);
        is_cut.resize(_n, false);

        for (int i = 0; i < cutpoints.size(); i++) {
            id[cutpoints[i]] = i;
            is_cut[cutpoints[i]] = true;
        }
        for (int i = 0; i < comps.size(); i++) {
            int _id = i + cutpoints.size();
            int (int u : comps[i]) {
                if (is_cut[u])
                    add_edge(id[u], _id);
                else
                    id[u] = _id;
            }
        }

        void add_edge(int u, int v) {
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
    }

    4.6.3 Cutpoints-Bridges

    struct graph {
        int n, timer;
        vector<vector<int>> adj, comps;
        vector<int> tin, low, vis, stk, cutpoints, bridges;

        graph(int n) : n(n) {
            adj.resize(n);

            void add_edge(int u, int v) {
                adj[u].push_back(v);
                adj[v].push_back(u);
            }

            void dfs(int u, int p = -1) {
                vis[u] = true;
                tin[u] = low[u] = timer++;
                stk.push_back(u);
                int children = 0;
                bool has_low_child = false;

                for (int v : adj[u]) {
                    if (v == p)
                        continue;
                    if (vis[v])
                        low[u] = min(low[u], tin[v]);
                    else {
                        dfs(v, u);
                        children++;
                        low[u] = min(low[u], low[v]);
                        if (low[v] > tin[u])
                            bridges.push_back(u);
                        if (low[v] >= tin[u]) {
                            has_low_child = true;
                            comps.push_back({u});
                            while (comps.back().back() != v and !stk.empty()) {
                                comps.back().push_back(stk.back());
                                stk.pop_back();
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        if ((p == -1 and children >= 2) or (p != -1 and has_low_child))
            cutpoints.push_back(u);
    }

    void find_cutpoints_bridges() {
        vis.assign(n, false);
        tin.assign(n, -1);
        low.assign(n, -1);
        cutpoints.clear();
        bridges.clear();
        comps.clear();
        stk.clear();
        timer = 0;
        for (int u = 0; u < n; u++)
            if (!vis[u])
                dfs(u);
    }
}

```

4.6.4 Kosaraju

```

struct graph {
    int n;
    vector<bool> vis;
    vector<vector<vector<int>>> adj;
    vector<vector<int>> comps;

    graph(int n) : n(n) {
        vis.resize(n);
        adj.resize(2);
        for (int dir = 0; dir < 2; dir++)
            adj[dir] = vector<vector<int>> (n);
    }

    void add_edge(int u, int v) {
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
    }

    void dfs(int u, int p, int dir, vector<int> &topo) {
        vis[u] = true;
        for (int v : adj[dir][u])
            if (!vis[v] and v != p)
                dfs(v, u, dir, topo);
        topo.push_back(u);
    }

    void kosaraju() {
        vector<int> topo;
        vis.assign(n, false);
        for (int u = 0; u < n; u++)
            if (!vis[u])
                dfs(u, -1, 0, topo);
        vis.assign(n, false);
        for (int i = n-1; i >= 0; i--) {
            if (!vis[topo[i]]) {
                comps.push_back({});
                dfs(topo[i], -1, 1, comps.back());
            }
        }
    }
}

```

4.6.5 2SAT

```

#include <bits/stdc++.h>

using namespace std;

struct two_sat {
    int n;
    vector<vector<int>> g, gr;
    vector<int> comp, topological_order, answer;

```



```

vector<bool> vis;
two_sat() {}
two_sat(int _n) { init(_n); }
void init(int _n) {
    n = _n;
    g.assign(2 * n, vector<int>());
    gr.assign(2 * n, vector<int>());
    comp.resize(2 * n);
    vis.resize(2 * n);
    answer.resize(2 * n);
}

void add_edge(int u, int v) {
    g[u].push_back(v);
    gr[v].push_back(u);
}

// At least one of them is true
void add_clause_or(int i, bool f, int j, bool g) {
    add_edge(i + (f ? n : 0), j + (g ? 0 : n));
    add_edge(j + (g ? n : 0), i + (f ? 0 : n));
}

// Only one of them is true
void add_clause_xor(int i, bool f, int j, bool g) {
    add_clause_or(i, f, j, g);
    add_clause_or(i, !f, j, !g);
}

// Both of them have the same value
void add_clause_and(int i, bool f, int j, bool g) {
    add_clause_xor(i, !f, j, g);
}

void dfs(int u) {
    vis[u] = true;
    for (const auto &v : g[u])
        if (!vis[v]) dfs(v);
    topological_order.push_back(u);
}

void scc(int u, int id) {
    vis[u] = true;
    comp[u] = id;
    for (const auto &v : gr[u])
        if (!vis[v]) scc(v, id);
}

bool satisfiable() {
    fill(vis.begin(), vis.end(), false);
    for (int i = 0; i < 2 * n; i++)
        if (!vis[i]) dfs(i);
    fill(vis.begin(), vis.end(), false);
    reverse(topological_order.begin(), topological_order.end());
    int id = 0;
    for (const auto &v : topological_order)
        if (!vis[v]) scc(v, id++);
    for (int i = 0; i < n; i++) {
        if (comp[i] == comp[i + n]) return false;
        answer[i] = (comp[i] > comp[i + n] ? 1 : 0);
    }
    return true;
}

int main() {
    int N, M;
    cin >> N >> M;
    two_sat solver(M); // 0 index
    for (int i = 0; i < N; i++){
        int t1, t2;
        char p1, p2;
        cin >> p1 >> t1 >> p2 >> t2; //2nd/4th are bool if cond. is
        true

```

```

        solver.add_clause_or(t1 - 1, p1 == '+', t2 - 1, p2 == '+');
    }
    if (!solver.satisfiable()) {
        cout << "IMPOSSIBLE";
        return 0;
    }
    for (int i = 0; i < M; i++)
        cout << (solver.answer[i] ? '+' : '-') << " ";
}

```

4.6.6 Flows and Bipartite Matching

4.6.7 Dinic

```

struct edge {
    int u, v;
    ll c, f;
    ll resid() { return c - f; }
};

struct flow_network {
    vector<edge> edges;
    vector<vector<int>> adj;
    vector<int> level, ptr;
    queue<int> q;
    int n, s, t;
    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }
    void add_edge(int u, int v, ll cap) {
        edges.push_back({u, v, cap, 0});
        adj[u].push_back(len(edges)-1);
        edges.push_back({v, u, 0, 0});
        adj[v].push_back(len(edges)-1);
    }
    bool bfs() {
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : adj[u]) {
                if (edges[id].c == edges[id].f)
                    continue;
                if (level[edges[id].v] != -1)
                    continue;
                level[edges[id].v] = level[u] + 1;
                q.push(edges[id].v);
            }
        }
        return level[t] != -1;
    }
    ll dfs(int u, ll pushed) {
        if (pushed == 0)
            return 0;
        if (u == t)
            return pushed;
        for (int& cid = ptr[u]; cid < len(adj[u]); cid++) {
            int id = adj[u][cid];
            int v = edges[id].v;
            if (level[u] + 1 != level[v])
                continue;
            ll tr = dfs(v, min(pushed, edges[id].resid()));
            if (tr == 0)
                continue;
            edges[id].f += tr;
            edges[id ^ 1].f -= tr;
            return tr;
        }
        return 0;
    }
    ll flow() {
        ll f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;

```

```

            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (ll pushed = dfs(s, INF)) {
                f += pushed;
            }
        }
        return f;
    }
};

4.6.8 Ford-Fulkerson Directed

struct flow_network {
    vector<vector<ll>> f, c;
    vector<vector<int>> adj;
    vector<int> par;
    int n, s, t;

    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        par.resize(n);
        f.resize(n, vector<ll>(n, 0));
        c.resize(n, vector<ll>(n, 0));
    }
    void add_edge(int u, int v, ll cap) {
        adj[u].push_back(v);
        c[u][v] = cap;
        adj[v].push_back(u);
        c[v][u] = 0;
    }
    ll bfs() {
        fill(par.begin(), par.end(), -1);
        par[s] = -2;
        queue<pair<int, ll>> q;
        q.push({s, INF});

        while (!q.empty()) {
            int u = q.front().first;
            ll flow = q.front().second; q.pop();

            for (int v : adj[u]) {
                if (c[u][v]-f[u][v] > 0 and par[v] == -1) {
                    par[v] = u;
                    ll new_flow = min(flow, c[u][v]-f[u][v]);
                    if (v == t)
                        return new_flow;
                    q.push({v, new_flow});
                }
            }
        }
        return 0;
    }
    ll max_flow() {
        ll flow = 0;
        while (ll new_flow = bfs()) {
            flow += new_flow;
            int v = t;
            while (v != s) {
                int u = par[v];
                f[u][v] += new_flow;
                f[v][u] -= new_flow;
                v = u;
            }
        }
        return flow;
    }
};

4.6.9 Ford-Fulkerson Undirected
struct edge {

```



```

    size_t i;
    int v;
    ll c, f;
    ll resid() { return c - f; }
};

struct flow_network {
    vector<edge> edges;
    vector<vector<int>> adj;
    vector<int> par;
    int n, s, t;

    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        par.resize(n);
    }

    void add_edge(int u, int v, ll cap) {
        edges.push_back({edges.size(), v, cap, 0});
        adj[u].push_back(edges.size()-1);
        edges.push_back({edges.size(), u, 0, 0});
        adj[v].push_back(edges.size()-1);
    }

    bool find_aug_path() {
        fill(par.begin(), par.end(), -1);
        par[s] = -2;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (u == t)
                break;
            for (int i : adj[u]) {
                edge& e = edges[i];
                if (e.resid() > 0 && par[e.v] == -1) {
                    par[e.v] = e.i;
                    q.push(e.v);
                }
            }
        }
        return par[t] != -1;
    }
    ll augment() {
        ll flow = INF;
        for (int v = t; v != s; v = edges[par[v]^1].v)
            flow = min(flow, edges[par[v]].resid());
        for (int v = t; v != s; v = edges[par[v]^1].v) {
            edges[par[v]].f += flow;
            edges[par[v]^1].f -= flow;
        }
        return flow;
    }
    ll max_flow() {
        ll flow = 0;
        while (find_aug_path()) {
            flow += augment();
        }
        return flow;
    }
};

```

4.6.10 Kuhn

```

struct flow_network {
    int n;
    vector<vector<int>> adj;
    vector<int> mt, used;

    flow_network(int n) : n(n) {
        adj.resize(n);
        mt.resize(n, -1);
        used.resize(n, false);
    }
}

```

```

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

bool kuhn(int u) {
    if (used[u])
        return false;
    used[u] = true;
    for (int v : adj[u]) {
        if (mt[v] == -1 or kuhn(mt[v])) {
            mt[v] = v;
            return true;
        }
        return false;
    }
}

};

4.6.11 Min Cost Max Flow

struct edge {
    int u, v; ld cap, cost, flow;
    edge(int u, int v, ld cap, ld cost) : u(u), v(v), cap(cap),
    cost(cost), flow(0) {}
    ld res() {
        return cap - flow;
    }
};

struct flow_network {
    int n, s, t, *par, *in_queue, *num_vis;
    ld *cost, *pot;
    vector<edge> edges;
    vector<int> *adj;
    map<pair<int,int>, vector<int>> edge_idx;

    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj = new vector<int>[n];
        par = new int[n];
        in_queue = new int[n];
        num_vis = new int[n];
        cost = new ld[n];
        pot = new ld[n];
        for (int i = 0; i < n; i++) {
            pot[i] = 0;
        }
    }

    void add_edge(int u, int v, ld cap, ld cost) {
        adj[u].push_back(edges.size());
        edge_idx[{u, v}].push_back(edges.size());
        edges.push_back({u, v, cap, cost});

        adj[v].push_back(edges.size());
        edge_idx[{v, u}].push_back(edges.size());
        edges.push_back({v, u, 0, -cost});
    }

    ld get_flow(int u, int v) {
        ld flow = 0;
        for (int i : edge_idx[{u, v}])
            flow += edges[i].flow;
        return flow;
    }

    void bellman_ford() {
        for (int i = 0; i < n; i++)
            pot[i] = INF;
        pot[s] = 0;
        for (int it = 0; it < n-1; it++) {
            for (auto e : edges) {
                if (e.res() > 0)
                    pot[e.v] = min(pot[e.v], pot[e.u] + e.cost);
            }
        }
    }
}

```

```

bool spfa() {
    queue<int> q; q.push(s);
    while (not q.empty()) {
        int u = q.front(); q.pop();
        in_queue[u] = 0;
        if (num_vis[u]++ >= n) {
            cost[u] = -INF;
            return false;
        }
        for (int v : adj[u]) {
            edge e = edges[v];
            if (e.res() <= 0)
                continue;
            ld nd = cost[u] + e.cost + pot[u] - pot[e.v];
            if (cost[e.v] > nd) {
                cost[e.v] = nd;
                par[e.v] = v;
                if (not in_queue[e.v]) {
                    q.push(e.v);
                    in_queue[e.v] = true;
                }
            }
        }
    }
    return cost[t] != INF;
}

bool aug_path() {
    for (int i = 0; i < n; i++) {
        par[i] = -1;
        in_queue[i] = 0;
        num_vis[i] = 0;
        cost[i] = INF;
    }
    cost[s] = 0;
    in_queue[s] = true;
    return spfa();
}

pdd max_flow(bool do_bellman_ford=false) {
    ld total_cost = 0, total_flow = 0;
    if (do_bellman_ford)
        bellman_ford();
    while (aug_path()) {
        ld flow = INF;
        for (int i = par[t]; i != -1; i = par[edges[i].u])
            flow = min(flow, edges[i].res());
        for (int i = par[t]; i != -1; i = par[edges[i].u]) {
            edges[i].flow += flow;
            edges[i^1].flow -= flow;
        }
        total_cost += flow * (cost[t] + pot[t] - pot[s]);
        total_flow += flow;
        for (int i = 0; i < n; i++) {
            if (par[i] != -1)
                pot[i] += cost[i];
        }
    }
    return {total_cost, total_flow};
}

};
// When updating cost, make sure to update the back edge to use -cost

```

5 Math

5.1 Binary and Ternary Search

```

#pragma once
#include "../1-Core/01-template.hpp"

// T: O(log(n)), M: O(1)
template<typename F>
lng binSearch(lng ok, lng ng, F f) {
    while (abs(ok - ng) > 1) {

```



```

return res;}
// T:  $O(n^3)$ 
friend Matrix inv(Matrix a)
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
    assert(a.n == a.m);
    return solveLin(std::move(a), eye(a.n, a.n));}
// T:  $O(n^3 * \log(n))$ 
friend Matrix pow(Matrix a, lng b) {
    assert(a.n == a.m && b >= 0);
    Matrix res = eye(a.n, a.n), tmp(a.n, a.n);
    while (b > 0) {
        if (b & 1) { res *= a; }
        b >>= 1;
        if (b > 0) { a *= a; }}
    return res;}
// T:  $O(n^3)$ 
friend Matrix solveLin(Matrix a, Matrix b)
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
    assert(a.n == b.n);
    int r = 0; vector<int> pivs(a.m, -1);
    for (int c = 0; r < a.n && c < a.m; c++) {
        int piv = r;
        if constexpr (std::is_floating_point_v<T>) {
            for (int p = r + 1; p < a.n; p++) {
                if (abs(a[p][c]) > abs(a[piv][c])) { piv = p; }}
            if (isNil(a[piv][c])) { piv = a.n; }}
        else { while (piv < a.n && isNil(a[piv][c])) { piv++; }}
        if (piv == a.n) { continue; }
        if (r != piv) {
            std::swap_ranges(a[r], a[r] + a.m, a[piv]);
            std::swap_ranges(b[r], b[r] + b.m, b[piv]);}
        T d = T(1) / a[r][c]; a[r][c] = T(1);
        for (int j = c + 1; j < a.m; j++) { a[r][j] *= d; }
        for (int j = 0; j < b.m; j++) { b[r][j] *= d; }
        for (int i = 0; i < a.n; i++) {
            if (i == r || isNil(a[i][c])) { continue; }
            T a_ic = a[i][c]; a[i][c] = T(0);
            #ifdef __AVX2__
            if constexpr ((requires { T::is_prime; } && sizeof(T)
== 4) || std::is_floating_point_v<T>) {
                FastGaussian::elimRow(a[i] + c + 1, a[r] + c + 1,
a_ic, a.m - c - 1);
                FastGaussian::elimRow(b[i], b[r], a_ic, b.m);}
            else
            #endif
            { for (int j = c + 1; j < a.m; j++) { a[i][j] -=
a[r][j] * a_ic; }
                for (int j = 0; j < b.m; j++) { b[i][j] -= b[r][j]
* a_ic; }}}
            pivs[c] = r++;}
        for (int i = r; i < a.n; i++) {
            for (int j = 0; j < b.m; j++) {
                if (!isNil(b[i][j])) { return Matrix(); }}}
    Matrix res(a.m, b.m);
    for (int c = 0; c < a.m; c++) {
        if (pivs[c] != -1) { std::copy_n(b[pivs[c]], b.m, res[c]); }
    }
    return res;}
// T:  $O(n)$ 
friend T trc(const Matrix &a) {
    assert(a.n == a.m);
    T res = 0;
    for (int i = 0; i < a.n; i++) { res += a[i][i]; }
    return res;}
friend Matrix trp(const Matrix &a) {
    Matrix res(a.m, a.n);
    for (int i = 0; i < a.m; i++) {
        for (int j = 0; j < a.n; j++) { res[i][j] = a[j][i]; }}
    return res;}
// T:  $O(n^3)$ 
friend Matrix adj(const Matrix a0)

```

```
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
    assert(a0.n == a0.m);
    int n = a0.n;
    if (n == 0) { return Matrix(); }
    if (n == 1) { return Matrix(vector<T>{T(1)}, 0); }
    Matrix a = a0, b = eye(n, n);
    int r = 0; T det_a = 1; vector<int> pivs(n, -1);
    for (int c = 0; c < n; c++) {
        int piv = r;
        if constexpr (std::is_floating_point_v<T>) {
            for (int p = r + 1; p < a.n; p++) {
                if (abs(a[p][c]) > abs(a[piv][c])) { piv = p; }
                if (isNil(a[piv][c])) { piv = a.n; }
            }
        } else { while (piv < a.n && isNil(a[piv][c])) { piv++; } }
        if (piv == n) { det_a = T(0); continue; }
        if (r != piv) {
            std::swap_ranges(a[r], a[r] + n, a[piv]);
            std::swap_ranges(b[r], b[r] + n, b[piv]);
            det_a = -det_a;
        }
        det_a *= a[r][c]; T d = T(1) / a[r][c]; a[r][c] = T(1);
        for (int j = c + 1; j < n; j++) { a[r][j] *= d; }
        for (int j = 0; j < n; j++) { b[r][j] *= d; }
        for (int i = 0; i < n; i++) {
            if (i == r || isNil(a[i][c])) { continue; }
            T a_ic = a[i][c]; a[i][c] = T(0);
            #ifdef __AVX2__
            if constexpr ((requires { T::is_prime; } && sizeof(T)
== 4) || std::is_floating_point_v<T>) {
                FastGaussian::elimRow(a[i] + c + 1, a[r] + c + 1,
a_ic, n - c - 1);
                FastGaussian::elimRow(b[i], b[r], a_ic, n);
            } else
            #endif
            { for (int j = c + 1; j < n; j++) { a[i][j] -=
a[r][j] * a_ic; }
                for (int j = 0; j < n; j++) { b[i][j] -= b[r][j] *
a_ic; } }
            pivs[c] = r++;
        }
        if (!isNil(det_a)) {
            Matrix res(n, n);
            for (int c = 0; c < n; c++) { std::copy_n(b[pivs[c]], n,
res[c]); }
            res *= det_a;
            return res;
        }
        if (r < n - 1) { return Matrix(n, n); }
        Matrix u = ker(a0), v = ker(trp(a0));
        int nz_i = -1, nz_j = -1;
        for (int i = 0; i < n; i++) { if (!isNil(u[0][i])) { nz_i = i;
break; } }
        for (int j = 0; j < n; j++) { if (!isNil(v[0][j])) { nz_j = j;
break; } }
        Matrix sub(n - 1, n - 1);
        for (int i = 0, sr = 0; i < n; i++) {
            if (i == nz_j) { continue; }
            for (int j = 0, sc = 0; j < n; j++) {
                if (j == nz_i) { continue; }
                sub[sr][sc++] = a0[i][j];
            }
            sr++;
        }
        T cof = det(sub);
        if ((nz_i + nz_j) & 1) { cof = -cof; }
        T q = cof / (u[0][nz_i] * v[0][nz_j]);
        Matrix res(n, n);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) { res[i][j] = u[0][i] * v[0][j]
* q; } }
        return res;
    }
    // T: O(n^3)
    friend vector<T> charPoly(Matrix a)
    requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
```



```

        r++;}
    return a;}

// T: O(1)
static bool isNil(const T &x) {
    if constexpr (std::is_floating_point_v<T>) { return abs(x) <
        1e-9; }
    else { return x == T(0); }}
friend bool operator==(const Matrix &a, const Matrix &b) = default;

friend ostream &operator<<(ostream &os, const Matrix &a) {
    if (a.n == 0 || a.m == 0) { return os << "[]"; }
    os << "[";
    for (int i = 0; i < a.n; i++) {
        os << "[";
        for (int j = 0; j < a.m; j++) { os << a[i][j] << (j < a.m -
            1 ? ", " : ""); }
        os << "]" << (i < a.n - 1 ? ", " : "");}
    return os << "];"}
};

```

5.4 Modulo Pow

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-exgcd.hpp"

```

```

// T: O(log(n)), M: O(1)
lng modPow(lng a, lng b, lng mod = INF64) {
    a = (a % mod + mod) % mod;
    if (b < 0) {
        lng x, y;
        if (exGcd(a, mod, x, y) != 1) { return -1; }
        a = (x % mod + mod) % mod; b = -b;}
    lng res = 1;
    while (b > 0) {
        if (b & 1) { res = lll(res) * a % mod; }
        a = lll(a) * a % mod; b >>= 1;}
    return res;}

```

5.5 Modulo Combinatorics

```

struct ModFac {
    int n;
    vector<mint> fac, inv_fac;

    ModFac(int n): n(n) {
        fac.resize(n + 1);
        fac[0] = 1;
        for (int i = 1; i <= n; i++) {
            fac[i] = fac[i - 1] * i;
        }

        inv_fac.resize(n + 1);
        inv_fac[n] = fac[n].inv();
        for (int i = n - 1; i >= 0; i--) {
            inv_fac[i] = inv_fac[i + 1] * (i + 1);
        }
    }

    mint combiNR(int x, int y) const {
        if (y > x || y < 0) {
            return mint(0);
        }

        assert(x <= n);

        return fac[x] * inv_fac[y] * inv_fac[x - y];
    }
}

```

```

mint combiWR(int x, int y) const {
    if (x < 1 || y < 0) {
        return mint(0);
    }
}

```

```

    }

    assert(x + y - 1 <= n);

    return fac[x + y - 1] * inv_fac[y] * inv_fac[x - 1];
}

mint permuNR(int x, int y) const {
    if (y > x || y < 0) {
        return mint(0);
    }

    assert(x <= n);

    return fac[x] * inv_fac[x - y];
}

mint permuWR(int x, int y) const {
    if (y < 0) {
        return mint(0);
    }

    return mint(x).pow(y);
}
};

```

5.6 Dynamic MEX

```

// S: O(n * log(n)), U: O(log(n)), Q: O(1), M: O(n)
struct DynMex {
    int n; vector<int> a;
    map<int, int> frq; set<int> mis;

    DynMex(const vector<int> &A) : n(A.size()), a(A) {
        for (int i = 0; i <= n; i++) { mis.insert(i); }
        for (int x : a) { frq[x]++; mis.erase(x); }
    }

    int query() const { return *mis.begin(); }

    void update(int i, int v) {
        if (--frq[a[i]] == 0) { mis.insert(a[i]); }
        a[i] = v;
        frq[v]++; mis.erase(v);
    }
};

```

5.7 Extended Euclidean GCD

```

#pragma once
#include "../1-Core/01-template.hpp"

// T: O(log(n)), M: O(1)
lng exGcd(lng a, lng b, lng &x, lng &y) {
    x = 1; y = 0; lng X = 0, Y = 1;
    while (b != 0) {
        lng q = a / b;
        x = std::exchange(X, x - q * X);
        y = std::exchange(Y, y - q * Y);
        a = std::exchange(b, a % b);}
    return a;}

```

5.8 Sieve of Eratosthenes

```

#pragma once
#include "../1-Core/01-template.hpp"

// S: O(n * loglog(n)), U: NA, Q: O(1), M: O(n)
struct SieveOfErath {
    int n; vector<char> is_prime;
    SieveOfErath(int N) : n(N) {
        is_prime.assign(n + 1, true);
        if (n >= 0) { is_prime[0] = false; }
    }
}

```

```

        if (n >= 1) { is_prime[i] = false; }
        for (int i = 4; i <= n; i += 2) { is_prime[i] = false; }
        for (int i = 3; 1LL * i * i <= n; i += 2) {
            if (!is_prime[i]) { continue; }
            for (lng j = 1LL * i * i; j <= n; j += 2 * i) { is_prime[j]
                = false; }}
    };
}

```

5.9 Linear Sieve

```

#pragma once
#include "../1-Core/01-template.hpp"

// S: O(n), U: NA, Q: O(1), M: O(n)
struct LinearSieve {
    int n;
    vector<int> prms, spf, num_div, phi;
    vector<lng> sum_div; vector<signed char> mu;
    LinearSieve(int N) : n(N) {
        spf.assign(n + 1, -1);
        for (int i = 2; i <= n; i++) {
            if (spf[i] == -1) { prms.push_back(i); spf[i] = i; }
            for (int p : prms) {
                if (1LL * i * p > n) { break; }
                spf[i * p] = p;
                if (i % p == 0) { break; }}}}

    void getNumDiv() {
        num_div.assign(n + 1, 0); vector<int> spf_exp(n + 1, 0);
        if (n >= 1) { num_div[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) {
                spf_exp[i] = spf_exp[j] + 1;
                num_div[i] = num_div[j] * (spf_exp[j] + 1) *
                    (spf_exp[i] + 1);}
            else {
                spf_exp[i] = 1;
                num_div[i] = num_div[j] * 2;}}}

    void getSumDiv() {
        sum_div.assign(n + 1, 0); vector<lng> spf_exp_sum(n + 1, 0);
        if (n >= 1) { sum_div[1] = 1; spf_exp_sum[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) {
                spf_exp_sum[i] = spf_exp_sum[j] * p + 1;
                sum_div[i] = sum_div[j] / spf_exp_sum[j] *
                    spf_exp_sum[i];}
            else {
                spf_exp_sum[i] = p + 1;
                sum_div[i] = sum_div[j] * (p + 1);}}}

    void getPhi() {
        phi.assign(n + 1, 0);
        if (n >= 1) { phi[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) { phi[i] = phi[j] * p; }
            else { phi[i] = phi[j] * (p - 1); }}}

    void getMu() {
        mu.assign(n + 1, 0);
        if (n >= 1) { mu[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) { mu[i] = 0; }
            else { mu[i] = -mu[j]; }}}

// Q: O(log(n))
vector<pair<int, int>> getPrimeFac(int a) {
    assert(a <= n);
    vector<pair<int, int>> res;
    while (a > 1) {
        int p = spf[a], cnt = 0;
        while (spf[a] == p) { cnt++; a /= p; }
        res.push_back({p, cnt});}
    return res;}
}

```


};

5.10 Segmented Sieve

```
#pragma once
#include "../1-Core/01-template.hpp"

// S:  $O(r^{1/2} * \log\log(r^{1/2})) + n * \log\log(r)$ , U: NA, Q:  $O(1)$ , M:  $O(r^{1/2} + n)$ 
struct SegmentedSieve {
    lng l, r; vector<char> is_prime;
    vector<lng> bprms, prms, num_div, sum_div, phi, rem;
    vector<signed char> mu;
    SegmentedSieve(lng L, lng R) : l(L), r(R) {
        assert(0 <= l && l <= r);
        lng lim = sqrt(r) + 1;
        while (lim * lim > r) { lim--; }
        vector<char> is_bprime(lim + 1, true);
        if (lim >= 0) { is_bprime[0] = false; }
        if (lim >= 1) { is_bprime[1] = false; }
        if (lim >= 2) { bprms.push_back(2); }
        for (int i = 4; i <= lim; i += 2) { is_bprime[i] = false; }
        for (lng i = 3; i <= lim; i += 2) {
            if (!is_bprime[i]) { continue; }
            bprms.push_back(i);
            for (lng j = i * i; j <= lim; j += 2 * i) { is_bprime[j] = false; }
        }
        is_prime.assign(r - l + 1, true);
        if (l <= 1) { is_prime[0] = false; if (l == 0 && r >= 1) { is_prime[1] = false; } }
        for (lng p : bprms) {
            lng j0 = max(p * p, (l + p - 1) / p * p);
            for (lng j = j0; j <= r; j += p) { is_prime[j - l] = false; }
        }
        for (lng i = 0; i <= r - l; i++) {
            if (is_prime[i]) { prms.push_back(i + l); }
        }
    }

    void getNumDiv() {
        num_div.assign(r - l + 1, 1); rem.resize(r - l + 1);
        for (lng i = 0; i <= r - l; i++) { rem[i] = i + l; }
        if (l == 0) { num_div[0] = 0; rem[0] = 1; }
        for (lng p : bprms) {
            lng j0 = (l + p - 1) / p * p;
            if (j0 == 0) { j0 = p; }
            for (lng j = j0; j <= r; j += p) {
                lng cnt = 0;
                while (rem[j - l] % p == 0) { rem[j - l] /= p; cnt++; }
                num_div[j - l] *= (cnt + 1); }
        }
        for (lng i = 0; i <= r - l; i++) {
            if (rem[i] > 1) { num_div[i] *= 2; }
        }
    }

    void getSumDiv() {
        sum_div.assign(r - l + 1, 1); rem.resize(r - l + 1);
        for (lng i = 0; i <= r - l; i++) { rem[i] = i + l; }
        if (l == 0) { sum_div[0] = 0; rem[0] = 1; }
        for (lng p : bprms) {
            lng j0 = (l + p - 1) / p * p;
            if (j0 == 0) { j0 = p; }
            for (lng j = j0; j <= r; j += p) {
                lng cur = 1, sum = 1;
                while (rem[j - l] % p == 0) { rem[j - l] /= p; cur *= p; sum += cur; }
                sum_div[j - l] *= sum; }
        }
        for (lng i = 0; i <= r - l; i++) {
            if (rem[i] > 1) { sum_div[i] *= (rem[i] + 1); }
        }
    }

    void getPhi() {
        phi.assign(r - l + 1, 0); rem.resize(r - l + 1);
        for (lng i = 0; i <= r - l; i++) { phi[i] = rem[i] = i + l; }
        if (l == 0) { phi[0] = 0; rem[0] = 1; }
        for (lng p : bprms) {
            lng j0 = (l + p - 1) / p * p;
            if (j0 == 0) { j0 = p; }
            for (lng j = j0; j <= r; j += p) {
                phi[j - l] -= phi[j - l] / p;
                while (rem[j - l] % p == 0) { rem[j - l] /= p; }
            }
        }
    }
};
```

```
for (lng i = 0; i <= r - l; i++) {
    if (rem[i] > 1) { phi[i] -= phi[i] / rem[i]; }
}
void getMu() {
    mu.assign(r - l + 1, 1); rem.resize(r - l + 1);
    for (lng i = 0; i <= r - l; i++) { rem[i] = i + l; }
    if (l == 0) { mu[0] = 0; rem[0] = 1; }
    for (lng p : bprms) {
        lng j0 = (l + p - 1) / p * p;
        if (j0 == 0) { j0 = p; }
        for (lng j = j0; j <= r; j += p) {
            lng cnt = 0;
            while (rem[j - l] % p == 0) { rem[j - l] /= p; cnt++; }
            if (cnt > 1) { mu[j - l] = 0; }
            else { mu[j - l] = -mu[j - l]; }
        }
    }
    for (lng i = 0; i <= r - l; i++) {
        if (rem[i] > 1) { mu[i] = -mu[i]; }
    }
};
```

5.11 Is Prime Miller-Rabin

```
#pragma once
#include "../1-Core/01-template.hpp"

// T:  $O(\log(n))$ , M:  $O(1)$ 
bool isPrimeMR(lng n) { // Miller-Rabin
    if (n < 2) { return false; }
    if (n == 2 || n == 3) { return true; }
    if (n % 2 == 0) { return false; }
    int s = __builtin_ctzll(n - 1); lng d = (n - 1) >> s;
    static constexpr lng bsm[] = {2LL, 7LL, 61LL};
    static constexpr lng blg[] = {2LL, 325LL, 9375LL, 28178LL, 450775LL, 9780504LL, 1795265022LL};
    const lng *B = n < 4759123141LL ? bsm : blg;
    for (int i = 0; i < (n < 4759123141LL ? 3 : 7); i++) {
        lng a = B[i];
        if (a % n == 0) { continue; }
        a %= n; lng b = d, x = 1;
        while (b > 0) {
            if (b & 1) { x = ulll(x) * a % n; }
            a = ulll(a) * a % n;
            b >>= 1; }
        if (x == 1 || x == n - 1) { continue; }
        bool is_comp = true;
        for (int j = 1; j < s; j++) {
            x = ulll(x) * x % n;
            if (x == n - 1) { is_comp = false; break; }
            if (x == 1) { return false; }
        }
        if (is_comp) { return false; }
    }
    return true;
}
```

5.12 Get One Factor Brent's Pollard-Rho

```
#pragma once
#include "../1-Core/01-template.hpp"

// T:  $O(n^{1/4})$ , M:  $O(1)$ 
lng getOneFacBPR(lng n, lng x0 = 2, lng c = 1) { // Brent's Pollard's Rho
    auto f = [](lng x, lng c, lng mod) -> lng {
        return (ulll(x) * x + c) % mod;
    };
    int m = 128, l = 1;
    lng x = x0, xs = -1, y = -1, g = 1, q = 1;
    while (g == 1) {
        y = x; int k = 0;
        while (g == 1 && k < l) {
            xs = x;
            for (int i = 0; i < min(m, l - k); i++) {
                x = f(x, c, n);
                q = ulll(q) * (x > y ? x - y : y - x) % n;
                g = gcd(q, n); k += m; }
            l *= 2; }
        if (g == n) {
            do {
```

```
xs = f(xs, c, n);
g = gcd(xs > y ? xs - y : y - xs, n); }
while (g == 1); }
return g; }
```

5.13 Get Prime Factors

```
#pragma once
#include "../1-Core/01-template.hpp"

// T:  $O(n^{1/2})$ , M:  $O(\log(n))$ 
vector<lng> getPrimeFacSlow(lng n) {
    vector<lng> res;
    for (int p : {2, 3, 5}) { while (n % p == 0) { res.push_back(p); n /= p; } }
    int j = 0; static constexpr int inc[] = {4, 2, 4, 2, 4, 6, 2, 6};
    for (lng i = 7; i * i <= n; i += inc[j], j = (j + 1) & 7) {
        while (n % i == 0) { res.push_back(i); n /= i; }
    }
    if (n > 1) { res.push_back(n); }
    sort(res.begin(), res.end());
    return res; }

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/05-isprimemr.hpp"
#include "../5-Mathematics/07-getonefacbpr.hpp"
#include "../6-Miscellaneous/random.hpp"

// T:  $O(n^{1/4})$ , M:  $O(\log(n))$ 
Random rnd;
void getPrimeFacFast(lng n, vector<lng> &res) {
    if (n <= 1) { return; }
    if (isPrimeMR(n)) { res.push_back(n); return; }
    static constexpr int prms[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
    for (int p : prms) {
        if (n % p == 0) {
            res.push_back(p);
            getPrimeFacFast(n / p, res);
            return; }
    }
    lng div = n, x0 = rnd.randLng(2, n - 1), c = rnd.randLng(1, n - 1);
    while (div == n) {
        div = getOneFacBPR(n, x0, c);
        if (div == n) {
            x0 = rnd.randLng(2, n - 1);
            c = rnd.randLng(1, n - 1); }
    }
    getPrimeFacFast(div, res);
    getPrimeFacFast(n / div, res); }
}
```

5.14 Get All Factors

```
#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/06-getprimefacslow.hpp"
#include "../5-Mathematics/08-getprimefacfast.hpp"

// T:  $O(n^{1/4})$  or  $O(n^{1/2})$ , M:  $O(d(n))$ 
vector<lng> getAllFac(lng n, bool opt = 0) {
    if (n <= 0) { return {}; }
    if (n == 1) { return {1}; }
    vector<lng> prms;
    /**
    if (!opt) { prms = getPrimeFacSlow(n); } /**/
    /**
    if (opt) { getPrimeFacFast(n, prms); } /**/
    sort(prms.begin(), prms.end());
    vector<lng> res = {1};
    for (int i = 0; i < prms.size(); i++) {
        int cnt = 0; lng p = prms[i];
        while (i < prms.size() && prms[i] == p) { cnt++; i++; }
        int len = res.size();
        for (int j = 0; j < len; j++) {
            lng cur = res[j];
            for (int k = 0; k < cnt; k++) { cur *= p; res.push_back(cur); }
        }
    }
}
```

```
// T:  $O(n^{1/4} * \log(n))$  or  $O(n^{1/2} * \log(n))$ , M:  $O(\log(n))$ 
lng getPrimRoot(lng mod, bool opt = 0) {
```

```

if (mod <= 0) { return -1; }
if (mod == 1 || mod == 2 || mod == 4) { return mod - 1; }
if (mod % 4 == 0) { return -1; }
vector<lng> prms;
/**
if (!opt) { prms = getPrimeFacSlow(mod >> (!mod & 1)); }/**/
if (opt) { getPrimeFacFast(mod >> (!mod & 1), prms); }/**/
sort(prms.begin(), prms.end());
if (!prms.empty()) { if (prms[0] != prms.back()) { return -1; } }
lng phi = getPhi(mod, opt); prms.clear();
/**
if (!opt) { prms = getPrimeFacSlow(phi); }/**/
if (opt) { getPrimeFacFast(phi, prms); }/**/
sort(prms.begin(), prms.end());
prms.erase(unique(prms.begin(), prms.end()), prms.end());
for (lng res = 2; res < mod; res++) {
    if (gcd(res, mod) != 1) { continue; }
    bool chk = true;
    for (int i = 0; i < prms.size() && chk; i++) {
        if (modPow(res, phi / prms[i], mod) == 1) { chk = false; }
    }
    if (chk) { return res; }
}
return -1;
}

```

5.23 Modulo Root

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-eagcd.hpp"
#include "../5-Mathematics/10-getphi.hpp"
#include "../5-Mathematics/16-modlog.hpp"
#include "../5-Mathematics/17-getprimroot.hpp"

```

```

// T: O(n^(1/2)), M: O(n^(1/2))
vector<lng> modRoot(lng a, lng b, lng mod, bool opt = 0) { // x^a = b
    mod m
    if (b == 0) { return {0}; }
    lng g = getPrimRoot(mod, opt);
    if (g == -1) { return {}; }
    lng k = modLog(g, b, mod);
    if (k == -1) { return {}; }
    lng phi = getPhi(mod, opt), x, y, d = exGcd(a, phi, y, x);
    if (k % d != 0) { return {}; }
    y = (lll(y) % phi + phi) % phi * (k / d) % phi;
    vector<lng> res;
    for (lng i = 0; i < d; i++) {
        res.push_back(modPow(g, y + i * (phi / d), mod));
    }
    sort(res.begin(), res.end());
    res.erase(unique(res.begin(), res.end()), res.end());
    return res;
}

```

5.24 Floor Sum

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T: O(log(n)), M: O(1)
lll getFloorSum(lng n, lng a, lng b, lng mod) { // \sum_{i=0}^{n-1}
    floor((a * x + b) / m)
    lll res = 0;
    if (a >= mod) { res += lll(n - 1) * n / 2 * (a / mod); a %= mod; }
    if (b >= mod) { res += lll(n) * (b / mod); b %= mod; }
    lng y = (lll(a) * n + b) / mod;
    if (y == 0) { return res; }
    lll x = lll(y) * mod - b;
    res += lll(n - (x + a - 1) / a) * y;
    res += getFloorSum(y, mod, (a - x % a) % a, a);
    return res;
}

```

5.25 Montgomery Multiplication

```

#pragma once

```

```

#include "../1-Core/01-template.hpp"
// S: O(1), U: NA, Q: O(1), M: O(1)
struct Montgomery {
    ulng mod, inv, rsq;
    Montgomery(ulng n = 1) : mod(n), inv(n) {
        assert(n & 1);
        for (int i = 0; i < 5; i++) { inv *= 2 - mod * inv; }
        inv = -inv; rsq = -ulll(mod) % mod; }

    ulng red(ulll x) const {
        ulng q = ulng(x) * inv, res = (x + ulll(q) * mod) >> 64;
        return res - (res >= mod) * mod; }
    ulng init(ulng x) const { return red(ulll(x) * rsq); }
    ulng mul(ulng a, ulng b) const { return red(ulll(a) * b); }
    // Q: O(log(n))
    ulng pow(ulng a, ulng b) const {
        a = init(a); ulng res = init(1);
        while (b > 0) {
            if (b & 1) { res = mul(res, a); }
            b >>= 1;
            if (b > 0) { a = mul(a, a); }
        }
        return red(res); }
};

```

5.26 Poly FFT

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T: O(n) or O(n * log(n)), M: O(n)
struct Poly {
    using cld = std::complex<long double>;
    static constexpr long double pi = 3.14159265358979323846;

    vector<lng> v; // ascending powers
    Poly(int n = 1) : v(max(n, 1), 0) {}
    Poly(const vector<lng> &a) : v(a) { trim(); }
    Poly(vector<lng> &a) noexcept : v(std::move(a)) { trim(); }

    lng &operator[](int i) { return v[i]; }
    lng operator[](int i) const { return v[i]; }
    int deg() const { return v.size() - 1; }
    lng lead() const { return v.empty() ? 0 : v.back(); }
    int size() const { return v.size(); }
    void trim() {
        while (v.size() > 1 && v.back() == 0) { v.pop_back(); }
        if (v.empty()) { v.push_back(0); }
    }

    Poly &operator+=(const Poly &o) {
        if (o.size() > size()) { v.resize(o.size(), 0); }
        for (int i = 0; i < o.size(); i++) { v[i] += o[i]; }
        trim();
        return *this; }
    Poly &operator-=(const Poly &o) {
        if (o.size() > size()) { v.resize(o.size(), 0); }
        for (int i = 0; i < o.size(); i++) { v[i] -= o[i]; }
        trim();
        return *this; }
    Poly &operator*=(lng o) {
        if (o == 0) { v = {0}; return *this; }
        for (lng &x : v) { x *= o; }
        trim();
        return *this; }
    static void fft(vector<cld> &a, bool is_inv) {
        int n = a.size();
        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) { j ^= bit; }
            if (i < j) { std::swap(a[i], a[j]); }
        }
        static vector<cld> w = {cld(0, 0), cld(1, 0)};
        if (w.size() < n) {
            int m = w.size(); w.resize(n);

```

```

        for (int len = m; len < n; len <= 1) {
            for (int i = 0; i < len; i++) {
                w[len + i] = cld(std::cos(pi * i / len),
                    std::sin(pi * i / len));
            }
        }
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    cld u = a[i + j];
                    cld v = a[i + j + len] * w[len + j];
                    a[i + j] = u + v;
                    a[i + j + len] = u - v;
                }
            }
        }
        if (is_inv) {
            reverse(a.begin() + 1, a.end());
            long double n_inv = 1.0L / n;
            for (cld &x : a) { x *= n_inv; }
        }
        Poly &operator+=(const Poly &o) {
            if (isNil() || o.isNil()) { v = {0}; return *this; }
            if (size() == 1) {
                lng v0 = v[0]; v = o.v;
                for (auto &x : v) { x *= v0; }
                trim();
                return *this; }
            if (o.size() == 1) {
                for (lng &x : v) { x += o[0]; }
                trim();
                return *this; }
            int n = 1;
            while (n < size() + o.size() - 1) { n <= 1; }
            #ifdef __AVX2__
            static vector<double> Ra, Ia, Rb, Ib;
            Ra.assign(v.begin(), v.end()); Ra.resize(n, 0);
            Ia.assign(n, 0);
            Rb.assign(o.v.begin(), o.v.end()); Rb.resize(n, 0);
            Ib.assign(n, 0);
            FastPoly::fft(Ra, Ia, false); FastPoly::fft(Rb, Ib, false);
            int i = 0;
            for (; i + 3 < n; i += 4) {
                __m256d ra = __m256d_loadu_pd(&Ra[i]), ia =
                    __m256d_loadu_pd(&Ia[i]);
                __m256d rb = __m256d_loadu_pd(&Rb[i]), ib =
                    __m256d_loadu_pd(&Ib[i]);
                __m256d tr = __m256d_sub_pd(__m256d_mul_pd(ra, rb),
                    __m256d_mul_pd(ia, ib));
                __m256d ti = __m256d_add_pd(__m256d_mul_pd(ra, ib),
                    __m256d_mul_pd(ia, rb));
                __m256d_storeu_pd(&Ra[i], tr); __m256d_storeu_pd(&Ia[i],
                    ti);
            }
            for (; i < n; i++) {
                double tr = Ra[i] * Rb[i] - Ia[i] * Ib[i];
                double ti = Ra[i] * Ib[i] + Ia[i] * Rb[i];
                Ra[i] = tr; Ia[i] = ti;
            }
            FastPoly::fft(Ra, Ia, true); v.resize(n);
            for (int i = 0; i < n; i++) { v[i] = std::llround(Ra[i]); }
            #else
            static vector<cld> fa, fb;
            fa.assign(v.begin(), v.end()); fa.resize(n);
            fb.assign(o.v.begin(), o.v.end()); fb.resize(n);
            fft(fa, false); fft(fb, false);
            for (int i = 0; i < n; i++) { fa[i] *= fb[i]; }
            fft(fa, true); v.resize(n);
            for (int i = 0; i < n; i++) { v[i] =
                std::llround(fa[i].real()); }
            #endif
            trim();
            return *this; }

```

```

Poly operator+(const Poly &a, const Poly &b) { return *this; }
Poly operator-(const Poly &a, const Poly &b) { return *this; }
Poly operator*(const Poly &a, const Poly &b) { return *this; }
Poly operator/=(const Poly &a, const Poly &b) { return *this; }
Poly operator+=(const Poly &a, const Poly &b) { return *this; }
Poly operator-=(const Poly &a, const Poly &b) { return *this; }
Poly operator*=(const Poly &a, const Poly &b) { return *this; }
Poly operator/=(const Poly &a, const Poly &b) { return *this; }

```

```

return *this;};

Moly &operator*=(mint o) {
    if (o.n == 0) { v = {mint::init(0)}; return *this; }
    for (mint &x : v) { x *= o; }
    trim();
    return *this;};

static void ntt(vector<mint> &a, bool is_inv) {
    #ifdef __AVX2__
    FastMoly::ntt(a, is_inv);
    #else
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j < bit; bit >= 1) { j ^= bit; }
        j ^= bit;
        if (i < j) { std::swap(a[i], a[j]); }
    }
    static vector<mint> w = {mint::init(0), mint::init(1)};
    if (w.size() < n) {
        int m = w.size(); w.resize(n);
        for (int len = m; len < n; len <= 1) {
            mint w0 = pow(mint(NTT_G), (NTT_P - 1) / (2 * len));
            for (int i = len / 2; i < len; i++) {
                w[2 * i] = w[i]; w[2 * i + 1] = w[i] * w0; }
        }
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    uint u = a[i + j].n;
                    uint v = 1ULL * a[i + j + len].n * w[j + len].n %
                        NTT_P;
                    a[i + j].n = u + v - (u + v >= NTT_P) *
                        NTT_P;
                    a[i + j + len].n = u - v + (u < v) * NTT_P; }
            }
        }
        if (is_inv) {
            reverse(a.begin() + 1, a.end());
            mint n_inv = inv(mint(n));
            for (mint &x : a) { x *= n_inv; }
        }
    }
    #endif
}

Moly &operator*=(const Moly &o) {
    if (isNil() || o.isNil()) { v = {mint::init(0)}; return *this; }
    if (size() == 1) {
        mint v0 = v[0]; v = o.v;
        for (auto &x : v) { x *= v0; }
        trim();
        return *this;};
    if (o.size() == 1) {
        for (mint &x : v) { x *= o[0]; }
        trim();
        return *this;};
    int n = 1;
    while (n < size() + o.size() - 1) { n <= 1; }
    static vector<mint> fa, fb;
    fa.assign(v.begin(), v.end()); fa.resize(n, mint::init(0));
    fb.assign(o.v.begin(), o.v.end()); fb.resize(n, mint::init(0));
    ntt(fa, false); ntt(fb, false);
    for (int i = 0; i < n; i++) { fa[i] *= fb[i]; }
    ntt(fa, true);
    v.assign(fa.begin(), fa.begin() + size() + o.size() - 1);
    trim();
    return *this;};

friend pair<Moly, Moly> divMod(Moly a, const Moly &b) {
    if (a.size() < b.size()) { return {Moly({mint::init(0)}),
        std::move(a)}; }
    int len = a.size() - b.size() + 1; Moly a_rev(len), b_rev(len);
    for (int i = 0; i < min(len, a.size()); i++) { a_rev[i] =
        a[a.size() - i - 1]; }
    for (int i = 0; i < min(len, b.size()); i++) { b_rev[i] =
        b[b.size() - i - 1]; }
    Moly q = std::move(a_rev) * invS(b_rev, len - 1);
    q.v.resize(len); reverse(q.v.begin(), q.v.end()); q.trim();
    Moly r = std::move(a); r -= q * b; r.trim();
    return {std::move(q), std::move(r)};};

Moly &operator/=(const Moly &o) {

```

```

    if (size() < o.size()) { v = {mint::init(0)}; return *this; }
    return *this = divMod(std::move(*this), o).first; }

Moly &operator%=(const Moly &o) {
    if (size() < o.size()) { return *this; }
    return *this = divMod(std::move(*this), o).second; }

friend Moly operator+(Moly a) { return a; }
friend Moly operator-(Moly a) { for (mint &x : a.v) { x = -x; }
return a; }
friend Moly operator+(Moly a, const Moly &b) { a += b; return a; }
friend Moly operator-(Moly a, const Moly &b) { a -= b; return a; }
friend Moly operator*(Moly a, mint b) { a *= b; return a; }
friend Moly operator*(mint b, Moly a) { a *= b; return a; }
friend Moly operator*(Moly a, const Moly &b) { a *= b; return a; }
friend Moly operator/(Moly a, const Moly &b) { a /= b; return a; }
friend Moly operator%(Moly a, const Moly &b) { a %= b; return a; }

// Polynomial Ring Operations
friend mint eval(const Moly &a, mint c) { // A(c)
    uint res = 0;
    for (int i = a.size() - 1; i >= 0; i--) { res = (1ULL * res *
c.n + a[i].n) % NTT_P; }
    return mint::init(res); }
friend vector<mint> chirpZ(const Moly &a, mint c, mint r, int m) {
// A(c * r^-i), i in [0, m)
    if (a.isNil() || m <= 0) { return vector<mint>(max(m, 0),
mint::init(0)); }
    if (r == mint::init(0)) { vector<mint> res(m, a[0]); res[0] =
eval(a, c); return res; }
    int n = a.size();
    mint c_pow = mint::init(1), r_cur = mint::init(1), r_pow =
mint::init(1);
    mint r_inv = inv(r), r_icur = mint::init(1), r_ipow =
mint::init(1);
    Moly A(n), B(n + m - 1);
    for (int i = 0; i < n; i++) {
        A.v[n - i - 1] = a[i] * c_pow * r_ipow; c_pow *= c;
        r_ipow *= r_icur; r_icur *= r_inv; }
    r_cur = mint::init(1), r_pow = mint::init(1);
    for (int i = 0; i < n + m - 1; i++) { B.v[i] = r_pow; r_pow *=
r_cur; r_cur *= r; }
    A *= B;
    if (A.size() < n + m - 1) { A.v.resize(n + m - 1,
mint::init(0)); }
    A.v.erase(A.v.begin(), A.v.begin() + n - 1); A.v.resize(m);
    r_icur = mint::init(1), r_ipow = mint::init(1);
    for (int i = 0; i < m; i++) {
        A.v[i] *= r_ipow;
        r_ipow *= r_icur; r_icur *= r_inv; }
    return std::move(A.v); }
// T: O(n * log(n)^2)
friend vector<mint> eval(const Moly &a, const vector<mint> &c) { //
A(c_n), i in [0, m)
    if (c.empty()) { return {}; }
    int m = c.size(); vector<Moly> segt(4 * m);
    auto build = [&](auto &&build, int i, int l, int r) -> void {
        if (l == r) { segt[i] = Moly{c[l], mint::init(1)};
return; }
        int md = std::midpoint(l, r);
        build(build, 2 * i, l, md);
        build(build, 2 * i + 1, md + 1, r);
        segt[i] = segt[2 * i] * segt[2 * i + 1]; }
    build(build, 1, 0, m - 1);
    vector<mint> res(m);
    auto solve = [&](auto &&solve, int i, int l, int r, Moly p) ->
void {
        p %= segt[i];
        if (p.isNil()) { for (int j = l; j <= r; j++) { res[j] =
mint::init(0); } return; }
        if (l == r) { res[l] = p[0]; return; }
        int md = std::midpoint(l, r);
        solve(solve, 2 * i, l, md, p);
        solve(solve, 2 * i + 1, md, p);
    }
    solve(solve, 1, 0, m - 1, A);
    return res; }

```



```

        fac.push_back(fac.back() * mint::init(i));
        ifc.push_back(ifc.back() * inv.back());
        mint b_pow = mint::init(1); Moly A(n), B(n);
        for (int i = 0; i < n; i++) {
            A[n - i - 1] = a[i] * fac[i];
            B[i] = b_pow * ifc[i]; b_pow *= b;
        }
        A *= B;
        A.v.resize(n, mint::init(0));
        std::reverse(A.v.begin(), A.v.end());
        for (int i = 0; i < n; i++) { A[i] *= ifc[i]; }
        A.trim();
        return A;
}

// T:  $O(n * \log(n)^2)$ 
static Moly inter(const vector<mint> &x, const vector<mint> &y) {
    // Lagrange Interpolation
    assert(x.size() == y.size());
    if (x.empty()) { return Moly({mint::init(0)}); }
    int n = x.size(); vector<Moly> segt(4 * n);
    auto build = [&](auto &&build, int i, int l, int r) -> void {
        if (l == r) { segt[i] = Moly({-x[l], mint::init(1)});
            return; }
        int md = std::midpoint(l, r);
        build(build, 2 * i, l, md);
        build(build, 2 * i + 1, md + 1, r);
        segt[i] = segt[2 * i] * segt[2 * i + 1];
    };
    build(build, 1, 0, n - 1);
    vector<mint> dM(n);
    auto calc = [&](auto &&calc, int i, int l, int r, Moly p) ->
    void {
        p %= segt[i];
        if (p.isNil()) { for (int j = 1; j <= r; j++) { dM[j] =
            mint::init(0); } return; }
        if (l == r) { dM[l] = p[0]; return; }
        int md = std::midpoint(l, r);
        calc(calc, 2 * i, l, md, p);
        calc(calc, 2 * i + 1, md + 1, r, std::move(p));
        calc(calc, 1, 0, n - 1, deriv(segt[1]));
        auto solve = [&](auto &&solve, int i, int l, int r) -> Moly {
            if (l == r) { return Moly({y[l] * inv(dM[l])}); }
            int md = std::midpoint(l, r);
            Moly l_res = solve(solve, 2 * i, l, md);
            Moly r_res = solve(solve, 2 * i + 1, md + 1, r);
            l_res *= segt[2 * i + 1]; r_res *= segt[2 * i]; l_res +=
            r_res;
            return l_res;
        };
        return solve(solve, 1, 0, n - 1);
    };

// Formal Power Series Operations
// T:  $O(n^2 + n^{\frac{3}{2}} * \log(n))$ 
friend Moly comp(const Moly &a, const Moly &b, int deg) { //
    Brent-Kung:  $A(B(x)) \bmod x^{\deg+1}$ 
    if (a.isNil()) { return Moly({mint::init(0)}); }
    int n = min(a.size(), deg + 1), m = std::sqrt(n + 1);
    vector<Moly> b_pow(m + 1); b_pow[0] = Moly({mint::init(1)});
    for (int i = 1; i <= m; i++) {
        b_pow[i] = b_pow[i - 1] * b;
        if (b_pow[i].size() > deg + 1) { b_pow[i].v.resize(deg +
            1); }
        b_pow[i].trim();
    }
    Moly res, cur;
    for (int i = (n - 1) / m; i >= 0; i--) {
        cur.v.clear();
        for (int j = 0; j <= min(n - i * m - 1, m - 1); j++) {
            if (a[i * m + j].n == 0) { continue; }
            if (cur.size() < b_pow[j].size()) {
                cur.v.resize(b_pow[j].size(), mint::init(0));
                for (int k = 0; k < b_pow[j].size(); k++) {
                    cur[k] += b_pow[j][k] * a[i * m + j];
                }
            }
            if (!res.isNil()) {
                res = std::move(res) * b_pow[m];
                if (res.size() > deg + 1) { res.v.resize(deg + 1); }
            }
            if (res.size() < cur.size()) { res.v.resize(cur.size(),
                mint::init(0)); }
        }
    }
}

```

```

for (int j = 0; j < cur.size(); j++) { res[j] += cur[j];
    res.trim();}
if (res.size() > deg + 1) { res.v.resize(deg + 1);
    res.trim();
    return res;}
friend Moly deriv(Moly a) {
    if (a.size() <= 1) { return Moly({mint::init(0)}); }
    for (int i = 1; i < a.size(); i++) { a[i - 1] = a[i] *
        mint::init(i); }
    a.v.pop_back(); a.trim();
    return a;}
friend Moly exp(const Moly &a, int deg) { //  $e^A(x) \bmod x^{\deg + 1}$ 
    assert(a.isNil() || a[0].n == 0);
    Moly res({mint::init(1)});
    for (int len = 2; len < 2 * deg + 2; len <= 1) {
        Moly cur = log(res, len - 1); cur.v.resize(len,
            mint::init(0));
        for (int i = 0; i < len; i++) {
            cur[i] = (i < a.size() ? a[i] : mint::init(0)) -
                cur[i];}
        cur[0] += mint::init(1);
        res *= cur; res.v.resize(len, mint::init(0)); res.trim();}
    res.v.resize(deg + 1, mint::init(0)); res.trim();
    return res;}
friend Moly integ(Moly a) {
    int n = a.size();
    static vector<mint> inv = {mint::init(0), mint::init(1)};
    for (int i = inv.size(); i < n; i++) {
        inv.push_back(mint::init(NTT_P - NTT_P / i) * inv[NTT_P %
            i]);}
    a.v.resize(n + 1);
    for (int i = n; i >= 1; i--) { a[i] = a[i - 1] * inv[i]; }
    a[0] = mint::init(0); a.trim();
    return a;}
friend Moly invS(const Moly &a, int deg) { //  $A^{-1}(x) \bmod x^{\deg + 1}$ 
    assert(a[0] != mint::init(0));
    Moly res({inv(a[0])});
    for (int len = 2; len < 2 * deg + 2; len <= 1) {
        static vector<mint> fa, fb;
        fa.assign(a.v.begin(), a.v.begin() + min(int(a.size()),
            len));
        fa.resize(2 * len, mint::init(0));
        fb.assign(res.v.begin(), res.v.end());
        fb.resize(2 * len, mint::init(0));
        ntt(fa, false); ntt(fb, false);
        for (int i = 0; i < 2 * len; i++) { fb[i] = fb[i] *
            (mint::init(2) - fa[i] * fb[i]); }
        ntt(fb, true);
        res.v.assign(fb.begin(), fb.begin() + len);}
    res.v.resize(deg + 1, mint::init(0)); res.trim();
    return res;}
friend Moly log(const Moly &a, int deg) { //  $\ln(A(x)) = \text{integ}$ 
 $A'(x)/A(x) \bmod x^{\deg + 1}$ 
    assert(a.size() > 0 && a[0] == mint::init(1));
    Moly da = deriv(a);
    if (da.size() > deg) { da.v.resize(deg); }
    Moly res = integ(std::move(da) * invS(a, deg - 1));
    res.v.resize(deg + 1, mint::init(0)); res.trim();
    return res;}
friend Moly powS(const Moly &a, lng b, int deg) { //  $A(x)^b \bmod$ 
 $x^{\deg + 1}$ 
    assert(b >= 0);
    if (b == 0) { return Moly({mint::init(1)}); }
    if (a.isNil()) { return Moly({mint::init(0)}); }
    int n = a.size(), i = 0;
    while (i < n && a[i].n == 0) { i++; }
    if (i == n || (i > 0 && b > deg / i)) { return
        Moly({mint::init(0)}); }
    mint a_inv = inv(a[i]); Moly res(n - i);
    for (int j = i; j < n; j++) { res[j - i] = a[j] * a_inv; }
    int shf = b * i; mint a_pow = pow(a[i], b);

```



```

res = log(res, deg - shf); res *= mint(b);
res = exp(res, deg - shf); res.v.resize(deg + 1);
for (int j = deg; j >= shf; j--) { res[j] = res[j - shf] *
a_pow; }
for (int j = 0; j < shf; j++) { res[j] = mint::init(0); }
res.trim();
return res; }
// T: O(n * log(n)^2)
friend mint solveKthTerm(Moly a, Moly b, lng k) { // Bostan-Mori
assert(!b.isNil() && b[0].n != 0);
Moly b_neg;
while (k > 0) {
b_neg.v.assign(b.v.begin(), b.v.end());
for (int i = 1; i < b_neg.size(); i += 2) { b_neg[i] =
-b_neg[i]; }
a *= b_neg; b *= b_neg;
int a_len = 0;
for (int i = k & 1; i < a.size(); i += 2) { a[a_len++] =
a[i]; }
a.v.resize(a_len); a.trim();
int b_len = 0;
for (int i = 0; i < b.size(); i += 2) { b[b_len++] = b[i]; }
b.v.resize(b_len); b.trim();
k >= 1; }
return a.isNil() ? mint::init(0) : a[0] * inv(b[0]); }
friend Moly sqrt(const Moly &a, int deg) { // A(x)^{1/2} mod x^{deg}
+ 1 }
if (a.isNil()) { return Moly({mint::init(0)}); }
int n = a.size(), i = 0;
while (i < n && a[i].n == 0) { i++; }
if (i == n || i / 2 > deg) { return Moly({mint::init(0)}); }
if (i & 1) { return Moly({mint::init(-1)}); }
mint a_rt2 = sqrt(a[i]);
if (a_rt2.n == -1) { return Moly({mint::init(-1)}); }
int shf = i / 2; mint inv2 = inv(mint::init(2)); Moly
res({a_rt2});
for (int len = 2; len / 2 <= deg - shf; len <= 1) {
Moly cur(min(n - i, len));
for (int j = 0; j < cur.size(); j++) { cur[j] = a[i + j]; }
cur = std::move(cur) * invS(res, len - 1);
res.v.resize(len, mint::init(0));
for (int j = 0; j < cur.size(); j++) { res[j] = (res[j] +
cur[j]) * inv2; }
for (int j = cur.size(); j < len; j++) { res[j] *= inv2; }
res.trim(); }
res.v.resize(deg + 1, mint::init(0));
for (int j = deg; j >= shf; j--) { res[j] = res[j - shf]; }
for (int j = 0; j < shf; j++) { res[j] = mint::init(0); }
res.trim();
return res; }
// T: O(n^2)
static Moly calcMinLinRec(const vector<mint> &seq) { //
Berlekamp-Massey
if (seq.empty()) { return Moly({mint::init(1)}); }
int len = 0, p = -1; mint d_prv = mint::init(1);
Moly res({mint::init(1)}), cur({mint::init(1)}), tmp;
for (int i = 0; i < seq.size(); i++) {
mint d = mint::init(0);
for (int j = 0; j < min(len + 1, int(res.size())); j++) { d
+= res[j] * seq[i - j]; }
if (d.n == 0) { continue; }
int shf = i - p; mint dd = d * inv(d_prv);
if (2 * len <= i) { tmp = res; }
if (res.size() < cur.size() + shf) {
res.v.resize(cur.size() + shf, mint::init(0)); }
for (int j = 0; j < cur.size(); j++) { res[j + shf] -= dd *
cur[j]; }
if (2 * len <= i) { len = i - len + 1; p = i; d_prv = d;
std::swap(cur, tmp); } }
res.trim();
return res; }
// T: O(n^2)

```

```

static mint guessKthTerm(const vector<mint> &seq, lng k) {
if (k < seq.size()) { return seq[k]; }
Moly b = calcMinLinRec(seq);
if (b.size() <= 1) { return mint::init(0); }
Moly a(vector<mint>(seq.begin(), seq.begin() + b.size() - 1));
a = std::move(a) * b; a.v.resize(b.size() - 1); a.trim();
return solveKthTerm(std::move(a), std::move(b), k); }
// T: O(1)
bool isNil() const { return v.empty() || (v.size() == 1 && v[0].n
== 0); }
bool operator==(const Moly &o) const { return v == o.v; }
bool operator!=(const Moly &o) const { return !(*this == o); }
Moly &operator<=(int shf) {
if (size() == 0 || (size() == 1 && v[0] == mint::init(0))) {
return *this; }
v.insert(v.begin(), shf, mint::init(0));
return *this; }
Moly &operator>=(int shf) {
if (shf >= size()) { v = {mint::init(0)}; return *this; }
v.erase(v.begin(), v.begin() + shf);
return *this; }
friend Moly operator<<(Moly a, int shf) { a <= shf; return a; }
friend Moly operator>>(Moly a, int shf) { a >= shf; return a; }
friend ostream &operator<<(ostream &os, const Moly &a) {
if (a.size() == 0) { return os << "[]"; }
os << "[";
for (int i = 0; i < a.size(); i++) { os << a[i].n << (i <
a.size() - 1 ? ", " : ""); }
return os << "];"; }
};

```

5.28 Fast Is Square

```

#pragma once
#include "../1-Core/01-template.hpp"
// T: O(1), M: O(1)
bool isSquare(lng x) {
static constexpr ulng msk = []() {
ulng res = 0;
for (int i = 0; i < 64; i++) { res |= (1ULL << (63 - (i * i) %
64)); }
return res; }();
if (x < 0) { return false; }
if (x == 0) { return true; }
if ((lng(msk << (x & 63))) >= 0) { return false; }
int cnt = __builtin_ctzll(x);
if (cnt & 1) { return false; }
x >= cnt;
if ((x & 7) != 1) { return false; }
lng xrt = std::round(std::sqrtl(x));
return xrt * xrt == x; }

```

5.29 Josephus Problem

```

#pragma once
#include "../1-Core/01-template.hpp"
// T: O(n * log(n)), M: O(log(n))
lng josephus(lng n, lng k) {
if (n == 1) { return 0; }
if (k == 1) { return n - 1; }
if (n < k) {
lng res = 0;
for (lng i = 2; i <= n; i++) { res = (res + k) % i; }
return res; }
lng nk = n - n / k;
return k * ((josephus(nk, k) + nk - n % k % nk) % nk) / (k - 1); }

```

5.30 Simpson Integration

```

#pragma once
#include "../1-Core/01-template.hpp"
// T: O(n), M: O(1)
template<typename F>
double fixedIntegS(double a, double b, F f, int n = 1000000) {
n += n & 1; double res = f(a) + f(b), d = (b - a) / n;
for (int i = 1; i < n; i++) { res += f(a + i * d) * (2.0 + (i & 1)
* 2.0); }
return res * d / 3.0; }
// T: O(n), M: O(log(n))
template<typename F>
double adaptiveIntegS(double a, double b, F f, double eps = 1e-9) {
auto calc = [&](auto &&calc, double l, double r,
double fl, double fm, double fr, double res, double
eps) -> double {
double m = l + (r - l) / 2.0;
double flm = f(l + (m - l) / 2.0), lres = (fl + 4.0 * flm + fm)
* (m - l) / 6.0;
double frm = f(m + (r - m) / 2.0), rres = (fm + 4.0 * frm + fr)
* (r - m) / 6.0;
if (abs(lres + rres - res) <= 15.0 * eps) {
return lres + rres + (lres + rres - res) / 15.0; }
return calc(calc, l, m, fl, flm, fm, lres, eps / 2.0) +
calc(calc, m, r, fm, frm, fr, rres, eps / 2.0); }
double m = a + (b - a) / 2.0, fa = f(a), fm = f(m), fb = f(b);
return calc(calc, a, b, fa, fm, fb, (fa + 4.0 * fm + fb) * (b - a)
/ 6.0, eps); }

```

5.31 Solve Diophantine Equation

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-exgcd.hpp"
// T: O(log(n)), M: O(1)
bool solveDioEq(lng a, lng b, lng c, lng &x, lng &y, lng &g) { // ax +
by = c
if (a == 0 && b == 0) {
if (c != 0) { return false; }
x = y = g = 0;
return true; }
if (a == 0) {
if (c % b != 0) { return false; }
x = 0; y = c / b; g = abs(b);
return true; }
if (b == 0) {
if (c % a != 0) { return false; }
x = c / a; y = 0; g = abs(a);
return true; }
g = exGcd(abs(a), abs(b), x, y);
if (c % g != 0) { return false; }
if (a < 0) { x = -x; }
if (b < 0) { y = -y; }
x = 1ll(x) * (c / g) % abs(b / g);
if (x < 0) { x += abs(b / g); }
y = (c - 1ll(a) * x) / b;
return true; }

```

5.32 Solve Modulo Equation

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-exgcd.hpp"
// T: O(log(n)), M: O(1)
pair<lng, lng> solveModEq(lng a, lng b, lng m0) { // ax = b (mod m0)
a = (a % m0 + m0) % m0; b = (b % m0 + m0) % m0;

```

```

        for (int i = 1; i <= n; i++) {
            for (int j = 2 * i; j <= n; j += i) { a[j] += a[i]; }
        } else {
            for (int i = n; i >= 1; i--) {
                for (int j = 2 * i; j <= n; j += i) { a[j] -= a[i]; }
            }
        }
    }

template<typename T> static void fctLcm(vector<T> &a, const
vector<int> &prms, bool inv) {
    int n = int(a.size()) - 1;
    if (!inv) {
        for (int p : prms) {
            if (p > n) break;
            for (int i = 1; i <= n / p; i++) { a[i * p] += a[i]; }
        }
    } else {
        for (int p : prms) {
            if (p > n) break;
            for (int i = n / p; i >= 1; i--) { a[i * p] -= a[i]; }
        }
    }
}

};

```

6.3 Kadane's Algorithm

```

template<typename T>
lng kadane(const vector<T> &arr) {
    if (arr.empty()) {
        return 0;
    }

    lng max_sum = -INF64, cur_sum = -INF64;
    for (T a : arr) {
        cur_sum = max(cur_sum + a, lng(a));
        max_sum = max(max_sum, cur_sum);
    }

    return max_sum;
}

```

6.4 Longest Common Subsequence

```

def lcs(X, Y, m, n):
    L = [[0 for i in range(n+1)] for j in range(m+1)]
    for i in range(m+1):
        for j in range(n+1):
            if i == 0 or j == 0:
                L[i][j] = 0
            elif X[i-1] == Y[j-1]:
                L[i][j] = L[i-1][j-1] + 1
            else:
                L[i][j] = max(L[i-1][j], L[i][j-1])

    lcs = ""
    i = m
    j = n
    while i > 0 and j > 0:
        if X[i-1] == Y[j-1]:
            lcs += X[i-1]
            i -= 1
            j -= 1
        elif L[i-1][j] > L[i][j-1]:
            i -= 1
        else:
            j -= 1
    lcs = lcs[::-1]
    print(lcs)

#test
X = "AGGTAB"
Y = "GXTXAYB"
m = len(X)
n = len(Y)
lcs(X, Y, m, n)

```

$$\left(x + k \frac{b}{\gcd(a, b)}, y - k \frac{a}{\gcd(a, b)}\right)$$

7.5 Misc

7.5.1 Determinants and PM

$$\begin{aligned}
 \det(A) &= \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^n a_{i, \sigma(i)} \\
 \operatorname{perm}(A) &= \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i, \sigma(i)} \\
 pf(A) &= \frac{1}{2^n n!} \sum_{\sigma \in S_{2n}} \operatorname{sgn}(\sigma) \prod_{i=1}^n a_{\sigma(2i-1), \sigma(2i)} \\
 &= \sum_{M \in \operatorname{PM}(n)} \operatorname{sgn}(M) \prod_{(i,j) \in M} a_{i,j}
 \end{aligned}$$

7.5.2 BEST Theorem

Count directed Eulerian cycles. Number of OST given by Kirchoff's Theorem (remove r/c with root) $\# \operatorname{OST}(G, r) \cdot \prod_v (d_v - 1)!$

7.5.3 Primitive Roots

Only exists when n is $2, 4, p^k, 2p^k$, where p odd prime. Assume n prime. Number of primitive roots $\phi(\phi(n))$. Let g be primitive root. All primitive roots are of the form g^k where $k, \phi(p)$ are coprime.
 k -roots: $g^{i \cdot \phi(n)/k}$ for $0 \leq i < k$

7.5.4 Sum of primes

For any multiplicative f :

$$S(n, p) = S(n, p-1) - f(p) \cdot (S(n/p, p-1) - S(p-1, p-1))$$

7.5.5 Floor

$$\begin{aligned}
 \lfloor \lfloor x/y \rfloor / z \rfloor &= \lfloor x / (yz) \rfloor \\
 x \% y &= x - y \lfloor x/y \rfloor
 \end{aligned}$$

7.5.6 Large Primes

100894373, 103893941, 999999937, 1000000007, 16208191877

7.6 Other Combinatorics Stuff

Catalan	$C_0 = 1, C_n = \frac{1}{n+1} \binom{2n}{n} = \sum_{i=0}^{n-1} C_i C_{n-i-1} = \frac{4n-2}{n+1} C_{n-1}$	
Stirling 1st kind	$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1, \begin{bmatrix} n \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ n \end{bmatrix} = 0, \begin{bmatrix} n \\ k \end{bmatrix} = (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}$	#perms of n objs with exactly k cycles
Stirling 2nd kind	$\left\{ \begin{smallmatrix} n \\ 1 \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n \\ n \end{smallmatrix} \right\} = 1, \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\} + \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\}$	#ways to partition n objs into k nonempty sets
Euler	$\langle \begin{smallmatrix} n \\ 0 \end{smallmatrix} \rangle = \langle \begin{smallmatrix} n \\ n-1 \end{smallmatrix} \rangle = 1, \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle = (k+1) \langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \rangle + (n-k) \langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \rangle$	#perms of n objs with exactly k ascents
Euler 2nd Order	$\ll \begin{smallmatrix} n \\ k \end{smallmatrix} \gg = (k+1) \ll \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \gg + (2n-k-1) \ll \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \gg$	#perms of $1, 1, 2, 2, \dots, n, n$ with exactly k ascents
Bell	$B_1 = 1, B_n = \sum_{k=0}^{n-1} B_k \binom{n-1}{k} = \sum_{k=0}^n \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	#partitions of $1..n$ (Stirling 2nd, no limit on k)

#labeled rooted trees

$$n^{n-1}$$

#labeled unrooted trees

$$n^{n-2}$$

#forests of k rooted trees

$$\frac{k}{n} \binom{n}{k} n^{n-k}$$

$$\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$$

$$\sum_{i=1}^n i^3 = n^2(n+1)^2/4$$

$$!n = n \times! (n-1) + (-1)^n$$

$$!n = (n-1)!(n-1) +!(n-2))$$

$$\sum_{i=1}^n \binom{n}{i} F_i = F_{2n}$$

$$\sum_i \binom{n-i}{i} = F_{n+1}$$

$$\sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}$$

$$x^k = \sum_{i=0}^k i! \left\{ \begin{smallmatrix} k \\ i \end{smallmatrix} \right\} \binom{x}{i} = \sum_{i=0}^k \left\langle \begin{smallmatrix} k \\ i \end{smallmatrix} \right\rangle \binom{x+i}{k}$$

$$a \equiv b \pmod{x, y} \Rightarrow a \equiv b \pmod{\text{lcm}(x, y)}$$

$$\sum_{d|n} \phi(d) = n$$

$$ac \equiv bc \pmod{m} \Rightarrow a \equiv b \pmod{\frac{m}{\text{gcd}(c, m)}}$$

$$(\sum_{d|n} \sigma_0(d))^2 = \sum_{d|n} \sigma_0(d)^3$$

$$p \text{ prime} \Leftrightarrow (p-1)! \equiv -1 \pmod{p}$$

$$\text{gcd}(n^a - 1, n^b - 1) = n^{\text{gcd}(a, b)} - 1$$

$$\sigma_x(n) = \prod_{i=0}^r \frac{p_i^{(a_i+1)x-1}}{p_i^x - 1}$$

$$\sigma_0(n) = \prod_{i=0}^r (a_i + 1)$$

$$\sum_{k=0}^m (-1)^k \binom{n}{k} = (-1)^m \binom{n-1}{m}$$

$$2^{\omega(n)} = O(\sqrt{n})$$

$$\sum_{i=1}^n 2^{\omega(i)} = O(n \log n)$$

$$d = v_i t + \frac{1}{2} a t^2$$

$$v_f^2 = v_i^2 + 2ad$$

$$v_f = v_i + at$$

$$d = \frac{v_i + v_f}{2} t$$

7.7 The Twelffold Way

Putting n balls into k boxes.

	Balls Boxes	same same	distinct same	same distinct	distinct distinct	Remarks
-	-	$p_k(n)$	$\sum_{i=0}^k \left\{ \begin{smallmatrix} n \\ i \end{smallmatrix} \right\}$	$\binom{n+k-1}{k-1}$	k^n	$p_k(n)$: #partitions of n into $\leq k$ positive parts
size ≥ 1	-	$p(n, k)$	$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	$\binom{n-1}{k-1}$	$k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	$p(n, k)$: #partitions of n into k positive parts
size ≤ 1	-	$[n \leq k]$	$[n \leq k]$	$\binom{k}{n}$	$n! \binom{k}{n}$	$[cond]$: 1 if $cond = true$, else 0

AdMU ProgVar

12/11/2023

CONTENTS

1. Data Structures	1	3.9. Polygon Centroid	6	4.15. Maximum Weighted Ind. Set in a Bipartite Graph	13
1.1. Dynamic Data Structures	1	3.10. Convex Hull	6	4.16. Synchronizing word problem	13
1.2. Fenwick Tree	1	3.10.1. 2D Convex Hull	6	4.17. Max flow with lower bounds on edges	13
1.3. Leq Counter	1	3.10.2. 3D Convex Hull	6	4.18. Tutte matrix for general matching	13
1.3.1. Leq Counter Array	1	3.10.3. Line upper/lower envelope	6	4.19. Heavy Light Decomposition	13
1.3.2. Leq Counter Map	1	3.11. Delaunay Triangulation	6	4.20. Centroid Decomposition	13
1.4. Monotonic Stack & Queue	1	3.12. Point in Polygon	6	4.21. Least Common Ancestor	13
1.5. Misof Tree	1	3.13. Cut Polygon by a Line	7	4.21.1. Binary Lifting	13
1.6. Mo's Algorithm	1	3.14. Triangle Centers	7	4.21.2. Euler Tour Sparse Table	13
1.7. Ordered Statistics Tree	2	3.15. Convex Polygon Intersection	7	4.21.3. Tarjan Off-line LCA	14
1.8. Segment Tree	2	3.16. Pick's Theorem for Lattice Points	7	4.22. Counting Spanning Trees	14
1.8.1. Recursive, Point-update Segment Tree	2	3.17. Minimum Enclosing Circle	7	4.23. Erdős-Gallai Theorem	14
1.8.2. Iterative, Point-update Segment Tree	2	3.18. Shamos Algorithm	7	4.24. Tree Isomorphism	14
1.8.3. Pointer-based, Range-update Segment Tree	2	3.19. kD Tree	7	5. Math I - Algebra	14
1.8.4. Array-based, Range-update Segment Tree -	2	3.20. Line Sweep (Closest Pair)	7	5.1. Generating Function Manager	14
1.8.5. 2D Segment Tree	2	3.21. Formulas	8	5.2. Fast Fourier Transform	15
1.8.6. Persistent Segment Tree	3	4. Graphs	8	5.3. FFT Polynomial Multiplication	15
1.8.7. Persistent Lazy Segment Tree	3	4.1. Single-Source Shortest Paths	8	5.4. Number Theoretic Transform	15
1.9. Sparse Table	3	4.1.1. Dijkstra	8	5.5. Polynomial Long Division	16
1.9.1. 1D Sparse table	3	4.1.2. Bellman-Ford	8	5.6. Matrix Multiplication	16
1.9.2. 2D Sparse Table	3	4.1.3. Shortest Path Faster Algorithm	8	5.7. Matrix Power	16
1.10. Splay Tree	3	4.2. All-Pairs Shortest Paths	8	5.8. Fibonacci Matrix	16
1.11. Treap	3	4.2.1. Floyd-Washall	8	5.9. Gauss-Jordan/Matrix Determinant	16
1.11.1. Implicit Treap	3	4.3. Strongly Connected Components	8	6. Math II - Combinatorics	16
1.11.2. Persistent Treap	4	4.3.1. Kosaraju	8	6.1. Lucas Theorem	16
1.12. Union Find	4	4.3.2. Tarjan's Offline Algorithm	8	6.2. Granville's Theorem	16
1.13. Unique Counter	4	4.4. Minimum Mean Weight Cycle	9	6.3. Derangements	17
2. Dynamic Programming	4	4.5. Biconnected Components	9	6.4. Factoradics	17
2.1. Convex Hull Trick	4	4.5.1. Bridges and Articulation Points	9	6.5. kth Permutation	17
2.1.1. Dynamic Convex Hull	4	4.5.2. Block Cut Tree	9	6.6. Catalan Numbers	17
2.1.2. Persistent LiChao Tree	4	4.5.3. Bridge Tree	9	6.7. Stirling Numbers	17
2.2. Divide and Conquer Optimization	5	4.6. Minimum Spanning Tree	9	6.8. Partition Function	17
2.3. Knuth Optimization	5	4.6.1. Kruskal	9	7. Math III - Number Theory	17
3. Geometry	5	4.6.2. Prim	9	7.1. Linear Prime Sieve	17
3.1. Dots and Cross Products	5	4.7. Euler Path/Cycle	9	7.2. Miller Rabin	17
3.2. Angles and Rotations	5	4.7.1. Euler Path/Cycle in a Directed Graph	9	7.3. Number/Sum of Divisors	17
3.3. Spherical Coordinates	5	4.7.2. Euler Path/Cycle in an Undirected Graph	10	7.4. Möbius Sieve	17
3.4. Point Projection	5	4.8. Bipartite Matching	10	7.5. Möbius Inversion	17
3.5. Great Circle Distance	5	4.8.1. Alternating Paths Algorithm	10	7.6. GCD Subset Counting	18
3.6. Point/Line/Plane Distances	5	4.8.2. Hopcroft-Karp Algorithm	10	7.7. Euler Totient	18
3.7. Intersections	5	4.8.3. Minimum Vertex Cover in Bipartite Graphs	10	7.8. Extended Euclidean	18
3.7.1. Line-Segment Intersection	5	4.9. Maximum Flow	10	7.9. Modular Exponentiation	18
3.7.2. Circle-Line Intersection	6	4.9.1. Edmonds-Karp	10	7.10. Modular Inverse	18
3.7.3. Circle-Circle Intersection	6	4.9.2. Dinic	10	7.11. Modulo Solver	18
3.8. Areas	6	4.9.3. Push-relabel	11	7.12. Linear Diophantine	18
3.8.1. Polygon Area	6	4.9.4. Gomory-Hu (All-pairs Maximum Flow)	11	7.13. Chinese Remainder Theorem	18
3.8.2. Triangle Area	6	4.10. Minimum Cost Maximum Flow	11	7.13.1. Super Chinese Remainder	18
3.8.3. Cyclic Quadrilateral Area	6	4.10.1. Hungarian Algorithm	11	7.14. Primitive Root	18
		4.11. Minimum Arborescence	12	7.15. Josephus	18
		4.12. Blossom algorithm	12	7.16. Number of Integer Points under a Lines	18
		4.13. Maximum Density Subgraph	12	8. Math IV - Numerical Methods	18
		4.14. Maximum-Weight Closure	12	8.1. Fast Square Testing	18
				8.2. Simpson Integration	18
				9. Strings	19
				9.1. Knuth-Morris-Pratt	19
				9.2. Trie	19

9.2.1.	Persistent Trie
9.3.	Suffix Array
9.4.	Longest Common Prefix
9.5.	Aho-Corasick Trie
9.6.	Palindromes
9.6.1.	Palindromic Tree
9.6.2.	Eertree
9.7.	Z Algorithm
9.8.	Booth's Minimum String Rotation
9.9.	Hashing
9.9.1.	Rolling Hash
10.	Other Algorithms
10.1.	2SAT
10.2.	DPLL Algorithm
10.3.	Stable Marriage
10.4.	Cycle-Finding
10.5.	Longest Increasing Subsequence
10.6.	Dates
10.7.	Simulated Annealing
10.8.	Simplex
10.9.	Fast Input Reading
10.10.	128-bit Integer
10.11.	Bit Hacks
11.	Misc
11.1.	Debugging Tips
11.2.	Solution Ideas
12.	Formulas
12.1.	Physics
12.2.	Markov Chains
12.3.	Burnside's Lemma
12.4.	Bézout's identity
12.5.	Misc
12.5.1.	Determinants and PM
12.5.2.	BEST Theorem
12.5.3.	Primitive Roots
12.5.4.	Sum of primes
12.5.5.	Floor
12.5.6.	Large Primes
13.	Other Combinatorics Stuff
13.1.	The Twelfefold Way

1. DATA STRUCTURES

1.1. Dynamic Data Structures.

```
struct state { /*...*/ };
struct operation { /*...*/ };
struct data_struct {
- state s;
- void apply_operation(operation &op) {
- - /*...*/
- - s = /*...*/; }
- void rollback(state &old_s) {
- - /*...*/
```

```
19 - - s = old_s; } } ;
19 // this is a segtree over time, not over the data structure
19 struct segtree {
19 - int l, r;
19 - data_struct &ds; // any data structure with the above interface
20 - std::vector<operation> operations;
20 - segtree *left, *right;
20 - segtree(int l, int r) : l(l), r(r) {
20 - - if (l == r) {
20 - - - left = right = NULL;
20 - - } else {
21 - - - int m = (l + r) / 2;
21 - - - left = new segtree(l, m);
21 - - - right = new segtree(m + 1, r); } }
21 - void add_operation(int _l, int _r, operation &op) {
21 - - if (_l <= l && r <= _r) {
21 - - - operations.push_back(op);
21 - - } else if (_r < l || r < _l) {
21 - - - return;
21 - - } else {
22 - - - left->add_operation(_l, _r, op);
22 - - - right->add_operation(_l, _r, op); } }
22 - void solve(std::vector<int> &ans) {
22 - - state old_s = ds.s;
22 - - for (operation &op : operations)
22 - - - ds.apply_operation(op);
22 - - if (l == r) {
23 - - - ans[l] = /*...*/; // process answers for time l
23 - - } else {
23 - - - left->solve(ds, ans);
24 - - - right->solve(ds, ans); }
24 - - ds.rollback(old_s); } } ;
```

1.2. Fenwick Tree.

```
24 struct fenwick {
24 - vi ar;
24 - fenwick(vi &ar) : ar(_ar.size(), 0) {
24 - - for (int i = 0; i < ar.size(); ++i) {
24 - - - ar[i] += _ar[i];
24 - - - int j = i | (i+1);
24 - - - if (j < ar.size())
24 - - - - ar[j] += ar[i]; } }
24 - int sum(int i) {
24 - - int res = 0;
24 - - for (; i >= 0; i = (i & (i+1)) - 1)
24 - - - res += ar[i];
24 - - return res; }
24 - int sum(int i, int j) { return sum(j) - sum(i-1); }
24 - void add(int i, int val) {
24 - - for (; i < ar.size(); i |= i+1)
24 - - - ar[i] += val; }
24 - int get(int i) {
24 - - int res = ar[i];
24 - - if (i) {
24 - - - int lca = (i & (i+1)) - 1;
24 - - - for (--i; i != lca; i = (i&(i+1))-1)
```

```
----- res -= ar[i]; } -----
----- return res; } -----
- void set(int i, int val) { add(i, -get(i) + val); } -----
- // range update, point query // -----
void add(int i, int j, int val) {
- - add(i, val); add(j+1, -val); }
- int get1(int i) { return sum(i); } } ;
```

1.3. Leq Counter.

1.3.1. Leq Counter Array.

```
#include "segtree.cpp"
struct LeqCounter {
- segtree **roots;
- LeqCounter(int *ar, int n) {
- - std::vector<ii> nums;
- - for (int i = 0; i < n; ++i)
- - - nums.push_back({ar[i], i});
- - std::sort(nums.begin(), nums.end());
- - roots = new segtree*[n];
- - roots[0] = new segtree(0, n);
- - int prev = 0;
- - for (ii &e : nums) {
- - - for (int i = prev+1; i < e.first; ++i)
- - - - roots[i] = roots[prev];
- - - roots[e.first] = roots[prev]->update(e.second, 1);
- - - prev = e.first; }
- - for (int i = prev+1; i < n; ++i)
- - - roots[i] = roots[prev]; }
- int count(int i, int j, int x) {
- - return roots[x]->query(i, j); } } ;
```

1.3.2. Leq Counter Map.

```
struct LeqCounter {
- std::map<int, segtree*> roots;
- std::set<int> neg_nums;
- LeqCounter(int *ar, int n) {
- - std::vector<ii> nums;
- - for (int i = 0; i < n; ++i) {
- - - nums.push_back({ar[i], i});
- - - neg_nums.insert(-ar[i]);
- - }
- - std::sort(nums.begin(), nums.end());
- - roots[0] = new segtree(0, n);
- - int prev = 0;
- - for (ii &e : nums) {
- - - roots[e.first] = roots[prev]->update(e.second, 1);
- - - prev = e.first; } }
- int count(int i, int j, int x) {
- - auto it = neg_nums.lower_bound(-x);
- - if (it == neg_nums.end()) return 0;
- - return roots[*it]->query(i, j); } } ;
```

1.4. Monotonic Stack & Queue.

```
struct min_stack {
- std::stack<ii> st;
- bool empty() { return st.empty(); }
- void add(int x) {
```

```
--- int mn = st.empty() ? x : std::min(x, st.top().second); --
--- st.push({x, mn}); }
--- void pop() { st.pop(); }
--- int top() { return st.top().first; }
--- int min() { return st.top().second; } };
struct min_queue {
- min_stack st1, st2;
- void add(int x) { st1.add(x); }
- void flip() {
--- while (!st1.empty()) st2.add(st1.top()), st1.pop(); }
- void pop() { if (st2.empty()) flip(); st2.pop(); }
- int front() { if (st2.empty()) flip(); return st2.top(); }
- int min() { return std::min(st1.min(), st2.min()); } };
```

1.5. **Misof Tree.** A simple tree data structure for inserting, erasing, and querying the nth largest element.

```
#define BITS 15
struct misof_tree {
- int cnt[BITS][1<<BITS];
- misof_tree() { memset(cnt, 0, sizeof(cnt)); }
- void insert(int x) {
--- for (int i = 0; i < BITS; cnt[i++][x]++, x >= 1); }
- void erase(int x) {
--- for (int i = 0; i < BITS; cnt[i++][x]--, x >= 1); }
- int nth(int n) {
--- int res = 0;
--- for (int i = BITS-1; i >= 0; i--)
----- if (cnt[i][res <= 1] <= n) n -= cnt[i][res], res |= 1;
--- return res; } };
```

1.6. **Mo's Algorithm.**

```
struct query {
- int id, l, r; ll hilbert_index;
- query(int id, int l, int r) : id(id), l(l), r(r) {
--- hilbert_index = hilbert_order(l, r, LOGN, 0); }
- ll hilbert_order(int x, int y, int pow, int rotate) {
--- if (pow == 0) return 0;
--- int hpow = 1 << (pow-1);
--- int seg = ((x<hpow) ? ((y<hpow)?0:3) : ((y<hpow)?1:2));
--- seg = (seg + rotate) & 3;
--- const int rotate_delta[4] = {3, 0, 0, 1};
--- int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
--- int nrot = (rotate + rotate_delta[seg]) & 3;
--- ll sub_sq_size = ll(1) << (2*pow - 2);
--- ll ans = seg * sub_sq_size;
--- ll add = hilbert_order(nx, ny, pow-1, nrot);
--- ans += (seg==1 || seg==2) ? add : (sub_sq_size-add-1);
--- return ans; }
- bool operator<(const query& other) const {
--- return this->hilbert_index < other.hilbert_index; } };
std::vector<query> queries;
for(const query &q : queries) { // [l,r] inclusive
- for(; r > q.r; r--) update(r, -1);
- for(r = r+1; r <= q.r; r++) update(r);
- r--;
- for( ; l < q.l; l++) update(l, -1);
```

```
- for(l = l-1; l >= q.l; l--) update(l);
- l++; }
```

1.7. **Ordered Statistics Tree.**

```
#include <ext/pb_ds/assoc_container.hpp> // for rope ds:
#include <ext/pb_ds/tree_policy.hpp> // <ext/rope>
using namespace __gnu_pbds; // __gnu_cxx;
template <typename T>
using index_set = tree<T, null_type, less<T>,
rb_tree_tag, tree_order_statistics_node_update>;
// indexed_set<int> t; t.insert(...); // rope<int> v;
// t.find_by_order(index); // 0-based // v.substr(l,r-l+1);
// t.order_of_key(key); // v.mutable_begin();
```

1.8. **Segment Tree.**

1.8.1. *Recursive, Point-update Segment Tree*.

1.8.2. *Iterative, Point-update Segment Tree.*

```
struct segtree {
- int n;
- int *vals;
- segtree(vi &ar, int n) {
--- this->n = n;
--- vals = new int[2*n];
--- for (int i = 0; i < n; ++i)
----- vals[i+n] = ar[i];
--- for (int i = n-1; i > 0; --i)
----- vals[i] = vals[i<<1] + vals[i<<1|1]; }
- void update(int i, int v) {
--- for (vals[i += n] += v; i > 1; i >= 1)
----- vals[i>>1] = vals[i] + vals[i^1]; }
- int query(int l, int r) {
--- int res = 0;
--- for (l += n, r += n+1; l < r; l >= 1, r >= 1) {
----- if (l&1) res += vals[l++];
----- if (r&1) res += vals[--r]; }
--- return res; } };
```

1.8.3. *Pointer-based, Range-update Segment Tree.*

```
struct segtree {
- int i, j, val, temp_val = 0;
- segtree *l, *r;
- segtree(vi &ar, int _i, int _j) : i(_i), j(_j) {
--- if (i == j) {
--- val = ar[i];
--- l = r = NULL;
--- } else {
--- int k = (i + j) >> 1;
--- l = new segtree(ar, i, k);
--- r = new segtree(ar, k+1, j);
--- val = l->val + r->val; } }
- void visit() {
--- if (temp_val) {
--- val += (j-i+1) * temp_val;
--- if (l) {
----- l->temp_val += temp_val;
----- r->temp_val += temp_val; }
```

```
--- temp_val = 0; } }
- void increase(int _i, int _j, int _inc) {
--- visit();
--- if (_i <= i && j <= _j) {
--- temp_val += _inc;
--- visit();
--- } else if (_j < i or j < _i) {
--- // do nothing
--- } else {
--- l->increase(_i, _j, _inc);
--- r->increase(_i, _j, _inc);
--- val = l->val + r->val; } }
- int query(int _i, int _j) {
--- visit();
--- if (_i <= i and j <= _j)
--- return val;
--- else if (_j < i || j < _i)
--- return 0;
--- else
--- return l->query(_i, _j) + r->query(_i, _j);
} };
```

1.8.4. *Array-based, Range-update Segment Tree*.

```
struct segtree {
- int n, *vals, *deltas;
- segtree(vi &ar) {
--- n = ar.size();
--- vals = new int[4*n];
--- deltas = new int[4*n];
--- build(ar, 1, 0, n-1); }
- void build(vi &ar, int p, int i, int j) {
--- deltas[p] = 0;
--- if (i == j)
--- vals[p] = ar[i];
--- else {
--- int k = (i + j) / 2;
--- build(ar, p<<1, i, k);
--- build(ar, p<<1|1, k+1, j);
--- pull(p); } }
- void pull(int p) { vals[p] = vals[p<<1] + vals[p<<1|1]; }
- void push(int p, int i, int j) {
--- if (deltas[p]) {
--- vals[p] += (j - i + 1) * deltas[p];
--- if (i != j) {
----- deltas[p<<1] += deltas[p];
----- deltas[p<<1|1] += deltas[p]; }
--- deltas[p] = 0; } }
- void update(int _i, int _j, int v, int p, int i, int j) {
--- push(p, i, j);
--- if (_i <= i && j <= _j) {
--- deltas[p] += v;
--- push(p, i, j);
--- } else if (_j < i || j < _i) {
--- // do nothing
--- } else {
--- int k = (i + j) / 2;
```

```
----- update(_i, _j, v, p<<1, i, k); -----
----- update(_i, _j, v, p<<1|1, k+1, j); -----
----- pull(p); } } -----
- int query(int _i, int _j, int p, int i, int j) {
--- push(p, i, j);
--- if (_i <= i and j <= _j)
----- return vals[p];
--- else if (_j < i || j < _i)
----- return 0;
--- else {
----- int k = (i + j) / 2;
----- return query(_i, _j, p<<1, i, k) +
----- query(_i, _j, p<<1|1, k+1, j); } } };
```

1.8.5. 2D Segment Tree.

```
struct segtree_2d {
- int n, m, **ar;
- segtree_2d(int n, int m) {
--- this->n = n; this->m = m;
--- ar = new int[n];
--- for (int i = 0; i < n; ++i) {
----- ar[i] = new int[m];
----- for (int j = 0; j < m; ++j)
----- ar[i][j] = 0; } }
- void update(int x, int y, int v) {
--- ar[x + n][y + m] = v;
--- for (int i = x + n; i > 0; i >>= 1) {
--- for (int j = y + m; j > 0; j >>= 1) {
----- ar[i>>1][j] = min(ar[i][j], ar[i^1][j]);
----- ar[i][j>>1] = min(ar[i][j], ar[i][j^1]);
- } } } // just call update one by one to build
- int query(int x1, int x2, int y1, int y2) {
--- int s = INF;
--- if (~x2) for(int a=x1+n, b=x2+n+1; a<b; a>>=1, b>>=1) {
----- if (a & 1) s = min(s, query(a++, -1, y1, y2));
----- if (b & 1) s = min(s, query(--b, -1, y1, y2));
--- } else for (int a=y1+m, b=y2+m+1; a<b; a>>=1, b>>=1) {
----- if (a & 1) s = min(s, ar[x1][a++]);
----- if (b & 1) s = min(s, ar[x1][--b]);
--- } return s; } };
```

1.8.6. Persistent Segment Tree.

```
struct segtree {
- int i, j, val;
- segtree *l, *r;
- segtree(vi &ar, int _i, int _j) : i(_i), j(_j) {
--- if (i == j) {
----- val = ar[i];
----- l = r = NULL;
--- } else {
----- int k = (i+j) >> 1;
----- l = new segtree(ar, i, k);
----- r = new segtree(ar, k+1, j);
----- val = l->val + r->val;
- } }
- segtree(int i, int j, segtree *l, segtree *r, int val) :
--- i(i), j(j), l(l), r(r), val(val) {
```

```
- segtree* update(int _i, int _val) {
--- if (_i <= i and j <= _i)
----- return new segtree(i, j, l, r, val + _val);
--- else if (_i < i or j < _i)
----- return this;
--- else {
----- segtree *nl = l->update(_i, _val);
----- segtree *nr = r->update(_i, _val);
----- return new segtree(i, j, nl, nr, nl->val + nr->val); } }
- int query(int _i, int _j) {
--- if (_i <= i and j <= _j)
----- return val;
--- else if (_j < i or j < _i)
----- return 0;
--- else
----- return l->query(_i, _j) + r->query(_i, _j); } };
```

1.8.7. Persistent Lazy Segment Tree.

```
struct segtree {
- int i, j, v, delta;
- segtree *l, *r;
- segtree(int i, int j) : segtree(i, j, 0, 0, NULL, NULL) {}
- segtree(int i, int j, int v, int d, segtree *l, segtree *r)
: i(i), j(j), v(v), delta(d), l(l), r(r) {}
- segtree *apply_delta(int d) {
--- return new segtree(i, j, v, delta+d, l, r); }
- segtree *push() {
--- int k = (i+j)/2;
--- if (!l) l = new segtree(i, k);
--- if (!r) r = new segtree(k+1, j);
--- if (delta != 0) {
--- v += (j-i+1) * delta;
--- if (i != j) {
----- l = l->apply_delta(delta);
----- r = r->apply_delta(delta); }
--- delta = 0; }
--- return this; }
- segtree *update(int _i, int _j, int _v) {
--- push();
--- if (_i <= i && j <= _j) return apply_delta(_v)->push();
--- else if (_j < i || j < _i) return this;
--- else {
----- segtree *l = l->update(_i, _j, _v);
----- segtree *r = r->update(_i, _j, _v);
----- return new segtree(i, j, l->v + r->v, 0, l, r); } }
- int query(int _i, int _j) {
--- push();
--- if (_i <= i && j <= _j) return v;
--- else if (_j < i || j < _i) return 0;
--- else return l->query(_i, _j) + r->query(_i, _j); } };
```

1.9. Sparse Table.

1.9.1. 1D Sparse table.

```
int lg[MAXN+1], spt[20][MAXN];
void build(vi &arr, int n) {
- lg[0] = lg[1] = 0;
```

```
- for (int i = 2; i <= n; ++i) lg[i] = lg[i>>1] + 1;
- for (int i = 0; i < n; ++i) spt[0][i] = arr[i];
- for (int j = 0; (2 << j) <= n; ++j)
--- for (int i = 0; i + (2 << j) <= n; ++i)
----- spt[j+1][i] = std::min(spt[j][i], spt[j][i+(1<<j)]); }
int query(int a, int b) {
- int k = lg[b-a+1], ab = b - (1<<k) + 1;
- return std::min(spt[k][a], spt[k][ab]); }
```

1.9.2. 2D Sparse Table .

```
const int N = 100, LGN = 20;
int lg[N], A[N][N], st[LGN][LGN][N][N];
void build(int n, int m) {
- for(int k=2; k<=std::max(n,m); ++k) lg[k] = lg[k>>1]+1;
- for(int i = 0; i < n; ++i)
--- for(int j = 0; j < m; ++j)
----- st[0][0][i][j] = A[i][j];
- for(int bj = 0; (2 << bj) <= m; ++bj)
--- for(int j = 0; j + (2 << bj) <= m; ++j)
----- for(int i = 0; i < n; ++i)
----- st[0][bj+1][i][j] =
----- std::max(st[0][bj][i][j],
----- st[0][bj][i][j + (1 << bj)]);
- for(int bi = 0; (2 << bi) <= n; ++bi)
--- for(int i = 0; i + (2 << bi) <= n; ++i)
----- for(int j = 0; j < m; ++j)
----- st[bi+1][0][i][j] =
----- std::max(st[bi][0][i][j],
----- st[bi][0][i + (1 << bi)][j]);
- for(int bi = 0; (2 << bi) <= n; ++bi)
--- for(int i = 0; i + (2 << bi) <= n; ++i)
----- for(int bj = 0; (2 << bj) <= m; ++bj)
----- for(int j = 0; j + (2 << bj) <= m; ++j) {
----- int ik = i + (1 << bi);
----- int jk = j + (1 << bj);
----- st[bi+1][bj+1][i][j] =
----- std::max(std::max(st[bi][bj][i][j],
----- st[bi][bj][ik][j]),
----- std::max(st[bi][bj][i][jk],
----- st[bi][bj][ik][jk])); } }
int query(int x1, int x2, int y1, int y2) {
- int kx = lg[x2 - x1 + 1], ky = lg[y2 - y1 + 1];
- int x12 = x2 - (1<<kx) + 1, y12 = y2 - (1<<ky) + 1;
- return std::max(std::max(st[kx][ky][x1][y1],
----- st[kx][ky][x1][y12]),
----- std::max(st[kx][ky][x12][y1],
----- st[kx][ky][x12][y12])); }
```

1.10. Splay Tree .

1.11. Treap.

1.11.1. Implicit Treap.

```
struct cartree {
- typedef struct _Node {
--- int node_val, subtree_val, delta, prio, size;
--- _Node *l, *r;
```

```

--- _Node(int val) : node_val(val), subtree_val(val), -----
----- delta(0), prio((rand()<<16)^rand()), size(1),
----- l(NULL), r(NULL) {}
--- ~Node() { delete l; delete r; }
- } *Node;
- int get_subtree_val(Node v) {
-     return v ? v->subtree_val : 0; }
- int get_size(Node v) { return v ? v->size : 0; }
- void apply_delta(Node v, int delta) {
-     if (!v) return;
-     v->delta += delta;
-     v->node_val += delta;
-     v->subtree_val += delta * get_size(v); }
- void push_delta(Node v) {
-     if (!v) return;
-     apply_delta(v->l, v->delta);
-     apply_delta(v->r, v->delta);
-     v->delta = 0; }
- void update(Node v) {
-     if (!v) return;
-     v->subtree_val = get_subtree_val(v->l) + v->node_val
-         + get_subtree_val(v->r);
-     v->size = get_size(v->l) + 1 + get_size(v->r); }
- Node merge(Node l, Node r) {
-     push_delta(l); push_delta(r);
-     if (!l || !r) return l ? l : r;
-     if (l->size <= r->size) {
-         l->r = merge(l->r, r);
-         update(l);
-         return l;
-     } else {
-         r->l = merge(l, r->l);
-         update(r);
-         return r; } }
- void split(Node v, int key, Node &l, Node &r) {
-     push_delta(v);
-     l = r = NULL;
-     if (!v) return;
-     if (key <= get_size(v->l)) {
-         split(v->l, key, l, v->l);
-         r = v;
-     } else {
-         split(v->r, key - get_size(v->l) - 1, v->r, r);
-         l = v; }
-     update(v); }
- Node root;
public:
- cartree() : root(NULL) {}
- ~cartree() { delete root; }
- int get(Node v, int key) {
-     push_delta(v);
-     if (key < get_size(v->l))
-         return get(v->l, key);
-     else if (key > get_size(v->l))
-         return get(v->r, key - get_size(v->l) - 1);
-     return v->node_val; }

```

```

- int get(int key) { return get(root, key); }
- void insert(Node item, int key) {
-     Node l, r;
-     split(root, key, l, r);
-     root = merge(merge(l, item), r); }
- void insert(int key, int val) {
-     insert(new _Node(val), key); }
- void erase(int key) {
-     Node l, m, r;
-     split(root, key + 1, m, r);
-     split(m, key, l, m);
-     delete m;
-     root = merge(l, r); }
- int query(int a, int b) {
-     Node l1, r1;
-     split(root, b+1, l1, r1);
-     Node l2, r2;
-     split(l1, a, l2, r2);
-     int res = get_subtree_val(r2);
-     l1 = merge(l2, r2);
-     root = merge(l1, r1);
-     return res; }
- void update(int a, int b, int delta) {
-     Node l1, r1;
-     split(root, b+1, l1, r1);
-     Node l2, r2;
-     split(l1, a, l2, r2);
-     apply_delta(r2, delta);
-     l1 = merge(l2, r2);
-     root = merge(l1, r1); }
- int size() { return get_size(root); } };

```

1.11.2. Persistent Treap

1.12. Union Find.

```

struct union_find {
- vi p; union_find(int n) : p(n, -1) {}
- int find(int x) { return p[x] < 0 ? x : p[x] = find(p[x]); }
- bool unite(int x, int y) {
-     int xp = find(x), yp = find(y);
-     if (xp == yp) return false;
-     if (p[xp] > p[yp]) std::swap(xp, yp);
-     p[xp] += p[yp], p[yp] = xp; return true; }
- int size(int x) { return -p[find(x)]; } };

```

1.13. Unique Counter.

```

struct UniqueCounter {
- int *B; std::map<int, int> last; LeqCounter *leq_cnt;
- UniqueCounter(int *ar, int n) { // 0-index A[i]
-     B = new int[n+1];
-     B[0] = 0;
-     for (int i = 1; i <= n; ++i) {
-         B[i] = last[ar[i-1]];
-         last[ar[i-1]] = i; }
-     leq_cnt = new LeqCounter(B, n+1); }
- int count(int l, int r) {
-     return leq_cnt->count(l+1, r+1, l); } };

```

2. DYNAMIC PROGRAMMING

2.1. Convex Hull Trick.

- $dp[i] = \min_{j < i} \{dp[j] + b[j] \times a[i]\}$
- $b[j] \geq b[j+1]$
- optionally $a[i] \leq a[i+1]$
- $O(n^2)$ to $O(n)$

2.1.1. Dynamic Convex Hull.

```

// USAGE: hull.insert_line(m, b); hull.gety(x); -----
bool UPPER_HULL = true; // you can edit this -----
bool IS_QUERY = false, SPECIAL = false;
struct line {
- ll m, b; line(ll m=0, ll b=0): m(m), b(b) {}
- mutable std::multiset<line>::iterator it;
- const line *see(std::multiset<line>::iterator it) const;
- bool operator < (const line& k) const {
-     if (!IS_QUERY) return m < k.m;
-     if (!SPECIAL) {
-         ll x = k.m; const line *s = see(it);
-         if (!s) return 0;
-         return (b - s->b) < (x) * (s->m - m);
-     } else {
-         ll y = k.m; const line *s = see(it);
-         if (!s) return 0;
-         ll n1 = y - b, d1 = m;
-         ll n2 = b - s->b, d2 = s->m - m;
-         if (d1 < 0) n1 *= -1, d1 *= -1;
-         if (d2 < 0) n2 *= -1, d2 *= -1;
-         return (n1) * d2 > (n2) * d1; } } };
struct dynamic_hull : std::multiset<line> {
- bool bad(iterator y) {
-     iterator z = next(y);
-     if (y == begin()) {
-         if (z == end()) return 0;
-         return y->m == z->m && y->b <= z->b; }
-     iterator x = prev(y);
-     if (z == end()) return y->m == x->m && y->b <= x->b;
-     return (x->b - y->b)*(z->m - y->m) >=
-         (y->b - z->b)*(y->m - x->m); }
- iterator next(iterator y) {return ++y;}
- iterator prev(iterator y) {return --y;}
- void insert_line(ll m, ll b) {
-     IS_QUERY = false;
-     if (!UPPER_HULL) m *= -1;
-     iterator y = insert(line(m, b));
-     y->it = y; if (bad(y)) {erase(y); return;}
-     while (next(y) != end() && bad(next(y)))
-         erase(next(y));
-     while (y != begin() && bad(prev(y)))
-         erase(prev(y)); }
- ll gety(ll x) {
-     IS_QUERY = true; SPECIAL = false;
-     const line& L = *lower_bound(line(x, 0));
-     ll y = (L.m) * x + L.b;
-     return UPPER_HULL ? y : -y; }
- ll getx(ll y) {

```



```
--- IS_QUERY = true; SPECIAL = true; -----
--- const line& l = *lower_bound(line(y, 0)); ---
--- return /*floor*/ ((y - l.b + l.m - 1) / l.m); } -----
} hull;
-----
const line* line::see(std::multiset<line>::iterator it)
const {return ++it == hull.end() ? NULL : &*it;}
```

2.1.2. Persistent LiChao Tree.

```
struct Line {
- ll k, d;
- ll eval(ll x) { return k * x + d; } };
struct LiChaoNode {
- Line line;
- int l, r;
- LiChaoNode() : l(0), r(0) {
-- line = Line({0, numeric_limits<long long>::max() / 2}); }
- LiChaoNode(Line line) : line(line), l(0), r(0) {}
} t[50 * N];
int T;
int upd(int pre, Line nw, int l, int r) {
- int m = (l + r) / 2;
- int id = ++T;
- t[id].line = t[pre].line;
- bool lef = nw.eval(l) < t[id].line.eval(l);
- bool mid = nw.eval(m) < t[id].line.eval(m);
- if(mid) swap(t[id].line, nw);
- if(l == r) return id;
- if(lef != mid) {
-- if(!t[pre].l) t[id].l = ++T, t[T] = LiChaoNode(nw);
-- else t[id].l = upd(t[pre].l, nw, l, m);
-- t[id].r = t[pre].r;
- } else {
-- if(!t[pre].r) t[id].r = ++T, t[T] = LiChaoNode(nw);
-- else t[id].r = upd(t[pre].r, nw, m + 1, r);
-- t[id].l = t[pre].l; }
- return id; }
ll Query(int cur, int x, int l, int r) {
- ll val = t[cur].line.eval(x);
- int m = (l + r) / 2;
- if(l < r) {
-- if(x<=m && t[cur].l) val=min(val, Query(t[cur].l,x,l,m));
-- if(x>m && t[cur].r) val=min(val, Query(t[cur].r,x,m+1,r)); }
- return val; }
struct PersistentLiChaoTree {
- int L, R;
- vector<int> roots;
- PersistentLiChaoTree() : L(-1e9), R(1e9) {
-- T = 1; roots = {1}; }
- PersistentLiChaoTree(int L, int R) : L(L), R(R) {
-- T = 1; roots.push_back(1); }
- void add_line(Line line) {
-- int root = upd(roots.back(), line, L, R);
-- roots.push_back(root); }
- ll query(int i, int x) { return Query(roots[i], x, L, R); } };
```

2.2. Divide and Conquer Optimization.

- $dp[i][j] = \min_{k < j} \{ dp[i-1][k] + C[k][j] \}$

```
•  $A[i][j] \leq A[i][j+1]$ 
•  $O(kn^2)$  to  $O(kn \log n)$ 
• sufficient:  $C[a][c] + C[b][d] \leq C[a][d] + C[b][c], a \leq b \leq c \leq d$  (QI)
ll dp[G+1][N+1];
void solve_dp(int g, int k_L, int k_R, int n_L, int n_R) {
- int n_M = (n_L+n_R)/2;
- dp[g][n_M] = INF;
- int best_k = -1;
- for (int k = k_L; k <= n_M && k <= k_R; k++)
-- if (dp[g-1][k]+cost(k+1,n_M) < dp[g][n_M]) {
--- dp[g][n_M] = dp[g-1][k]+cost(k+1,n_M);
--- best_k = k; }
- if (n_L <= n_M-1)
-- solve_dp(g, k_L, best_k, n_L, n_M-1);
- if (n_M+1 <= n_R)
-- solve_dp(g, best_k, k_R, n_M+1, n_R); }
```

2.3. Knuth Optimization.

```
•  $dp[i][j] = \min_{i < k < j} \{ dp[i][k] + dp[k][j] + C[i][j] \}$ 
•  $A[i][j-1] \leq A[i][j] \leq A[i+1][j]$ 
•  $O(n^3)$  to  $O(n^2)$ 
• sufficient: QI and  $C[b][c] \leq C[a][d], a \leq b \leq c \leq d$ 
// opt[i][j] is the optimal split point between i and j
int dp[N][N], opt[N][N];
int cost(int i, int j) { /*...*/ }
int knuth_dp_solve() {
- for (int i = 0; i < N; i++)
-- dp[i][i] = /*...*/, opt[i][i] = i;
- for (int i = N-2; i >= 0; --i)
-- for (int j = i+1; j < N; ++j) {
--- int mn = INF, c = cost(i, j);
--- for (int k=opt[i][j-1]; k<=min(j-1,opt[i+1][j]); ++k) {
---- int cur = dp[i][k] + dp[k+1][j] + c;
---- if (cur < mn) mn = cur, opt[i][j] = k; }
---- dp[i][j] = mn; } }
```

3. GEOMETRY

```
#include <complex>
#define x real()
#define y imag()
typedef std::complex<double> point; // 2D point only
const double PI = acos(-1.0), EPS = 1e-7;
```

3.1. Dots and Cross Products.

```
double dot(point a, point b) {
- return a.x * b.x + a.y * b.y; } // + a.z * b.z;
double cross(point a, point b) {
- return a.x * b.y - a.y * b.x; }
double cross(point a, point b, point c) {
- return cross(a, b) + cross(b, c) + cross(c, a); }
double cross3D(point a, point b) {
- return point(a.x*b.y - a.y*b.x, a.y*b.z -
- a.z*b.y, a.z*b.x - a.x*b.z); }
```

3.2. Angles and Rotations.

```
double angle(point a, point b, point c) {
- // angle formed by abc in radians:  $PI < x \leq PI$ 
- return abs(remainder(arg(a-b) - arg(c-b), 2*PI)); }
point rotate(point p, point a, double d) {
- //rotate point a about pivot p CCW at d radians
- return p + (a - p) * point(cos(d), sin(d)); }
```

3.3. Spherical Coordinates.

$$\begin{aligned} x &= r \cos \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \cos \theta \sin \phi & \theta &= \cos^{-1} x/r \\ z &= r \sin \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

3.4. Point Projection.

```
point proj(point p, point v) {
- // project point p onto a vector v (2D & 3D)
- return dot(p, v) / norm(v) * v; }
point projLine(point p, point a, point b) {
- // project point p onto line ab (2D & 3D)
- return a + dot(p-a, b-a) / norm(b-a) * (b-a); }
point projSeg(point p, point a, point b) {
- // project point p onto segment ab (2D & 3D)
- double s = dot(p-a, b-a) / norm(b-a);
- return a + min(1.0, max(0.0, s)) * (b-a); }
point projPlane(point p, double a, double b,
- double c, double d) {
- // project p onto plane ax+by+cz+d=0 (3D)
- // same as: o + p - project(p - o, n);
- double k = -d / (a*a + b*b + c*c);
- point o(a*k, b*k, c*k), n(a, b, c);
- point v(p.x-o.x, p.y-o.y, p.z-o.z);
- double s = dot(v, n) / dot(n, n);
- return point(o.x + p.x + s * n.x, o.y +
- p.y + s * n.y, o.z + p.z + s * n.z); }
```

3.5. Great Circle Distance.

```
double greatCircleDist(double lat1, double long1,
- double lat2, double long2, double R) {
- long1 *= PI / 180; lat1 *= PI / 180; // to radians
- long2 *= PI / 180; lat2 *= PI / 180;
- return R*acos(sin(lat1)*sin(lat2) +
- cos(lat1)*cos(lat2)*cos(abs(long1 - long2))); }
// another version, using actual (x, y, z)
double greatCircleDist(point a, point b) {
- return atan2(abs(cross3D(a, b)), dot3D(a, b)); }
```

3.6. Point/Line/Plane Distances.

```
double distPtLine(point p, double a, double b, double c) {
- // dist from point p to line ax+by+c=0
- return abs(a*p.x + b*p.y + c) / sqrt(a*a + b*b);}
double distPtLine(point p, point a, point b) {
- // dist from point p to line ab
- return abs((a.y - b.y) * (p.x - a.x) +
- (b.x - a.x) * (p.y - a.y)) /
- hypot(a.x - b.x, a.y - b.y);}
double distPtPlane(point p, double a, double b,
- double c, double d) {
```

```
- // distance to 3D plane ax + by + cz + d = 0 -----
- return (a*p.x+b*p.y+c*p.z+d)/sqrt(a*a+b*b+c*c); } -----
/*! // distance between 3D lines AB & CD (untested) -----
double distLine3D(point A,point B,point C,point D){ -----
- point u = B - A, v = D - C, w = A - C; -----
- double a = dot(u, u), b = dot(u, v); -----
- double c = dot(v, v), d = dot(u, w); -----
- double e = dot(v, w), det = a*c - b*b; -----
- double s = det < EPS ? 0.0 : (b*e - c*d) / det; -----
- double t = det < EPS -----
-- ? (b > c ? d/b : e/c) // parallel -----
-- : (a*e - b*d) / det; -----
- point top = A + u * s, bot = w - A - v * t; -----
- return dist(top, bot); -----
} // dist<EPS: intersection */ -----
```

3.7. Intersections.

3.7.1. *Line-Segment Intersection.* Get intersection points of 2D lines/segments $\overline{ab}$ and $\overline{cd}$.

```
point null(HUGE_VAL, HUGE_VAL); -----
point line_inter(point a, point b, point c, -----
----- point d, bool seg = false) { -----
- point ab(b.x - a.x, b.y - a.y); -----
- point cd(d.x - c.x, d.y - c.y); -----
- point ac(c.x - a.x, c.y - a.y); -----
- double D = -cross(ab, cd); // determinant -----
- double Ds = cross(cd, ac); -----
- double Dt = cross(ab, ac); -----
- if (abs(D) < EPS) { // parallel -----
-- if (seg && abs(Ds) < EPS) { // collinear -----
----- point p[] = {a, b, c, d}; -----
----- sort(p, p + 4, [](point a, point b) { -----
----- return a.x < b.x-EPS || -----
----- (dist(a,b) < EPS && a.y < b.y-EPS); }); -----
----- return dist(p[1], p[2]) < EPS ? p[1] : null; } -----
-- return null; } -----
- double s = Ds / D, t = Dt / D; -----
- if (seg && (min(s,t)<-EPS||max(s,t)>1+EPS)) return null; -----
- return point(a.x + s * ab.x, a.y + s * ab.y); } -----
/* double A = cross(d-a, b-a), B = cross(c-a, b-a); -----
return (B*d - A*c)/(B - A); */ -----
```

3.7.2. *Circle-Line Intersection.* Get intersection points of circle at center c , radius r , and line $\overline{ab}$.

```
std::vector<point> CL_inter(point c, double r, -----
-- point a, point b) { -----
- point p = projLine(c, a, b); -----
- double d = abs(c - p); vector<point> ans; -----
- if (d > r + EPS); // none -----
- else if (d > r - EPS) ans.push_back(p); // tangent -----
- else if (d < EPS) { // diameter -----
-- point v = r * (b - a) / abs(b - a); -----
-- ans.push_back(c + v); -----
-- ans.push_back(c - v); -----
- } else { -----
-- double t = acos(d / r); -----
```

```
-- p = c + (p - c) * r / d; -----
-- ans.push_back(rotate(c, p, t)); -----
-- ans.push_back(rotate(c, p, -t)); -----
- } return ans; } -----
```

3.7.3. *Circle-Circle Intersection.*

```
std::vector<point> CC_intersection(point c1, -----
-- double r1, point c2, double r2) { -----
- double d = dist(c1, c2); -----
- vector<point> ans; -----
- if (d < EPS) { -----
-- if (abs(r1-r2) < EPS); // inf intersections -----
- } else if (r1 < EPS) { -----
-- if (abs(d - r2) < EPS) ans.push_back(c1); -----
- } else { -----
-- double s = (r1*r1 + d*d - r2*r2) / (2*r1*d); -----
-- double t = acos(max(-1.0, min(1.0, s))); -----
-- point mid = c1 + (c2 - c1) * r1 / d; -----
-- ans.push_back(rotate(c1, mid, t)); -----
-- if (abs(sin(t)) >= EPS) -----
-- ans.push_back(rotate(c2, mid, -t)); -----
- } return ans; } -----
```

3.8. Areas.

3.8.1. *Polygon Area.* Find the area of any 2D polygon given as points in $O(n)$.

```
double area(point p[], int n) { -----
- double a = 0; -----
- for (int i = 0, j = n - 1; i < n; j = i++) -----
-- a += cross(p[i], p[j]); -----
- return abs(a) / 2; } -----
```

3.8.2. *Triangle Area.* Find the area of a triangle using only their lengths. Lengths must be valid.

```
double area(double a, double b, double c) { -----
- double s = (a + b + c) / 2; -----
- return sqrt(s*(s-a)*(s-b)*(s-c)); } -----
```

3.8.3. *Cyclic Quadrilateral Area.* Find the area of a cyclic quadrilateral using only their lengths. A quadrilateral is cyclic if its inner angles sum up to 360° .

```
double area(double a, double b, double c, double d) { -----
- double s = (a + b + c + d) / 2; -----
- return sqrt((s-a)*(s-b)*(s-c)*(s-d)); } -----
```

3.9. *Polygon Centroid.* Get the centroid/center of mass of a polygon in $O(m)$.

```
point centroid(point p[], int n) { -----
- point ans(0, 0); -----
- double z = 0; -----
- for (int i = 0, j = n - 1; i < n; j = i++) { -----
-- double cp = cross(p[j], p[i]); -----
-- ans += (p[j] + p[i]) * cp; -----
-- z += cp; -----
- } return ans / (3 * z); } -----
```

3.10. Convex Hull.

3.10.1. *2D Convex Hull.* Get the convex hull of a set of points using Graham-Andrew's scan. This sorts the points at $O(n \log n)$, then performs the Monotonic Chain Algorithm at $O(n)$.

```
// counterclockwise hull in p[], returns size of hull -----
bool xcmp(const point& a, const point& b) { -----
- return a.x < b.x || (a.x == b.x && a.y < b.y); } -----
int convex_hull(point p[], int n) { -----
- std::sort(p, p + n, xcmp); if (n <= 1) return n; -----
- int k = 0; point *h = new point[2 * n]; -----
- double zer = EPS; // -EPS to include collinears -----
- for (int i = 0; i < n; h[k++] = p[i++]) -----
-- while (k >= 2 && cross(h[k-2],h[k-1],p[i]) < zer) -----
-- --k; -----
- for(int i = n-2, t = k; i >= 0; h[k++] = p[i--]) -----
-- while (k > t && cross(h[k-2],h[k-1],p[i]) < zer) -----
-- --k; -----
- k -= 1 + (h[0].x==h[1].x&&h[0].y==h[1].y ? 1 : 0); -----
- copy(h, h + k, p); delete[] h; return k; } -----
```

3.10.2. *3D Convex Hull.* Currently $O(N^2)$, but can be optimized to a randomized $O(N \log N)$ using the Clarkson-Shor algorithm. Sauce: Efficient 3D Convex Hull Tutorial on CF.

```
typedef std::vector<bool> vb; -----
struct point3D { -----
- ll x, y, z; -----
- point3D(ll x = 0, ll y = 0, ll z = 0) : x(x), y(y), z(z) {} -----
- point3D operator-(const point3D &o) const { -----
-- return point3D(x - o.x, y - o.y, z - o.z); } -----
- point3D cross(const point3D &o) const { -----
-- return point3D(y*o.z-z*o.y, z*o.x-x*o.z, x*o.y-y*o.x); } -----
- ll dot(const point3D &o) const { -----
-- return x*o.x + y*o.y + z*o.z; } -----
- bool operator==(const point3D &o) const { -----
-- return std::tie(x, y, z) == std::tie(o.x, o.y, o.z); } -----
- bool operator<(const point3D &o) const { -----
-- return std::tie(x, y, z) < std::tie(o.x, o.y, o.z); } }; -----
struct face { -----
- std::vector<int> p_idx; -----
- point3D q; }; -----
std::vector<face> convex_hull_3D(std::vector<point3D> &points) { -----
- int n = points.size(); -----
- std::vector<face> faces; -----
- std::vector<vb> dead(points.size(), vb(points.size(), true)); -----
- auto add_face = [&](int a, int b, int c) { -----
-- faces.push_back({{a, b, c}, -----
-- (points[b] - points[a]).cross(points[c] - points[a])}); -----
-- dead[a][b] = dead[b][c] = dead[c][a] = false; }; -----
- add_face(0, 1, 2); -----
- add_face(0, 2, 1); -----
- for (int i = 3; i < n; ++i) { -----
-- std::vector<face> faces_inv; -----
-- for(face &f : faces) { -----
--- if ((points[i] - points[f.p_idx[0]]).dot(f.q) > 0) -----
--- for (int j = 0; j < 3; ++j) -----
--- dead[f.p_idx[j]][f.p_idx[(j+1)%3]] = true; -----
-- else -----
```

```
----- faces_inv.push_back(f); } -----
-- faces.clear();
--- for(face &f : faces_inv) { -----
---- for (int j = 0; j < 3; ++j) { -----
----- int a = f.p_idx[j], b = f.p_idx[(j + 1) % 3];
----- if(dead[b][a]) -----
----- add_face(b, a, i); } } -----
-- faces.insert( -----
----- faces.end(), faces_inv.begin(), faces_inv.end()); } -----
- return faces; } -----
```

3.10.3. *Line upper/lower envelope.* To find the upper/lower envelope of a collection of lines $a_i + b_ix$, plot the points (b_i, a_i) , add the point $(0, \pm\infty)$ (depending on if upper/lower envelope is desired), and then find the convex hull.

3.11. **Delaunay Triangulation.** Simply map each point (x,y) to (x,y,x^2+y^2) , find the 3d convex hull, and drop the 3rd dimension.

3.12. **Point in Polygon.** Check if a point is strictly inside (or on the border) of a polygon in $O(n)$.

```
bool inPolygon(point q, point p[], int n) { -----
- bool in = false; -----
- for (int i = 0, j = n - 1; i < n; j = i++) -----
-- in ^= (((p[i].y > q.y) != (p[j].y > q.y)) &&
----- q.x < (p[j].x - p[i].x) * (q.y - p[i].y) /
----- (p[j].y - p[i].y) + p[i].x);
- return in; }
bool onPolygon(point q, point p[], int n) { -----
- for (int i = 0, j = n - 1; i < n; j = i++) -----
- if (abs(dist(p[i], q) + dist(p[j], q)
----- dist(p[i], p[j])) < EPS) -----
-- return true;
- return false; } -----
```

3.13. **Cut Polygon by a Line.** Cut polygon by line $\overline{ab}$ to its left in $O(n)$, such that $\angle abp$ is counter-clockwise.

```
vector<point> cut(point p[],int n,point a,point b) { -----
- vector<point> poly;
- for (int i = 0, j = n - 1; i < n; j = i++) { -----
-- double c1 = cross(a, b, p[j]); -----
-- double c2 = cross(a, b, p[i]); -----
-- if (c1 > -EPS) poly.push_back(p[j]); -----
-- if (c1 * c2 < -EPS) -----
---- poly.push_back(line_inter(p[j], p[i], a, b)); -----
- } return poly; } -----
```

3.14. **Triangle Centers.**

```
point bary(point A, point B, point C, -----
----- double a, double b, double c) { -----
- return (A*a + B*b + C*c) / (a + b + c); }
point trilinear(point A, point B, point C, -----
----- double a, double b, double c) { -----
- return bary(A,B,C,abs(B-C)*a, -----
----- abs(C-A)*b,abs(A-B)*c); }
point centroid(point A, point B, point C) { -----
- return bary(A, B, C, 1, 1, 1); }
point circumcenter(point A, point B, point C) { -----
```

```
- double a=norm(B-C), b=norm(C-A), c=norm(A-B); -----
- return bary(A,B,C,a*(b+c-a),b*(c+a-b),c*(a+b-c)); } -----
point orthocenter(point A, point B, point C) { -----
- return bary(A,B,C, tan(angle(B,A,C)), -----
----- tan(angle(A,B,C)), tan(angle(A,C,B))); }
point incenter(point A, point B, point C) { -----
- return bary(A,B,C,abs(B-C),abs(A-C),abs(A-B)); } -----
// incircle radius given the side lengths a, b, c
double inradius(double a, double b, double c) { -----
- double s = (a + b + c) / 2; -----
- return sqrt(s * (s-a) * (s-b) * (s-c)) / s; }
point excenter(point A, point B, point C) { -----
- double a = abs(B-C), b = abs(C-A), c = abs(A-B); -----
- return bary(A, B, C, -a, b, c); } -----
- // return bary(A, B, C, a, -b, c); -----
- // return bary(A, B, C, a, b, -c); -----
point brocard(point A, point B, point C) { -----
- double a = abs(B-C), b = abs(C-A), c = abs(A-B); -----
- return bary(A,B,C,c/b*a,a/c*b,b/a*c); // CCW -----
- // return bary(A,B,C,b/c*a,c/a*b,a/b*c); // CW } -----
point symmedian(point A, point B, point C) { -----
- return bary(A,B,C,norm(B-C),norm(C-A),norm(A-B)); } -----
```

3.15. **Convex Polygon Intersection.** Get the intersection of two convex polygons in $O(n^2)$.

```
std::vector<point> convex_polygon_inter( -----
-- point a[], int an, point b[], int bn) { -----
- point ans[an + bn + an*bn]; -----
- int size = 0;
- for (int i = 0; i < an; ++i) -----
-- if (inPolygon(a[i],b,bn) || onPolygon(a[i],b,bn)) -----
-- ans[size++] = a[i]; -----
- for (int i = 0; i < bn; ++i) -----
-- if (inPolygon(b[i],a,an) || onPolygon(b[i],a,an)) -----
-- ans[size++] = b[i]; -----
- for (int i = 0, I = an - 1; i < an; I = i++) -----
-- for (int j = 0, J = bn - 1; j < bn; J = j++) { -----
-- try { -----
-- point p=line_inter(a[i],a[I],b[j],b[J],true); -----
-- ans[size++] = p; -----
-- } catch (exception ex) {} } -----
- size = convex_hull(ans, size);
- return vector<point>(ans, ans + size); } -----
```

3.16. **Pick's Theorem for Lattice Points.** Count points with integer coordinates inside and on the boundary of a polygon in $O(n)$ using Pick's theorem: $\text{Area} = I + B/2 - 1$.

```
int interior(point p[], int n) { -----
- return area(p,n) - boundary(p,n) / 2 + 1; }
int boundary(point p[], int n) { -----
- int ans = 0;
- for (int i = 0, j = n - 1; i < n; j = i++) -----
-- ans += gcd(p[i].x - p[j].x, p[i].y - p[j].y); -----
- return ans; } -----
```

3.17. **Minimum Enclosing Circle.** Get the minimum bounding ball that encloses a set of points (2D or 3D) in Θn .

```
std::pair<point, double> bounding_ball(point p[], int n){ -----
- std::random_shuffle(p, p + n); -----
- point center(0, 0); double radius = 0; -----
- for (int i = 0; i < n; ++i) { -----
-- if (dist(center, p[i]) > radius + EPS) { -----
-- center = p[i]; radius = 0; -----
-- for (int j = 0; j < i; ++j) -----
-- if (dist(center, p[j]) > radius + EPS) { -----
-- center.x = (p[i].x + p[j].x) / 2; -----
-- center.y = (p[i].y + p[j].y) / 2; -----
-- // center.z = (p[i].z + p[j].z) / 2; -----
-- radius = dist(center, p[i]); // midpoint -----
-- for (int k = 0; k < j; ++k) -----
-- if (dist(center, p[k]) > radius + EPS) { -----
-- center = circumcenter(p[i], p[j], p[k]); -----
-- radius = dist(center, p[i]); } } } } -----
- return {center, radius}; } -----
```

3.18. **Shamos Algorithm.** Solve for the polygon diameter in $O(n \log n)$.

```
double shamos(point p[], int n) { -----
- point *h = new point[n+1]; copy(p, p + n, h); -----
- int k = convex_hull(h, n); if (k <= 2) return 0; -----
- h[k] = h[0]; double d = HUGE_VAL; -----
- for (int i = 0, j = 1; i < k; ++i) { -----
-- while (distPtLine(h[j+1], h[i], h[i+1]) >= -----
-- distPtLine(h[j], h[i], h[i+1])) { -----
-- j = (j + 1) % k; } -----
-- d = min(d, distPtLine(h[j], h[i], h[i+1])); -----
- } return d; } -----
```

3.19. **kD Tree.** Get the k -nearest neighbors of a point within pruned radius in $O(k \log k \log n)$.

```
#define cpoint const point& -----
bool cmpx(cpoint a, cpoint b) {return a.x < b.x;} -----
bool cmpy(cpoint a, cpoint b) {return a.y < b.y;} -----
struct KDTree { -----
- KDTree(point p[],int n): p(p), n(n) {build(0,n);} -----
- priority_queue< pair<double, point*> > pq; -----
- point *p; int n, k; double qx, qy, prune; -----
- void build(int L, int R, bool dvx=false) { -----
-- if (L >= R) return; -----
-- int M = (L + R) / 2; -----
-- nth_element(p + L, p + M, p + R, dvx?cmpx:cmpy); -----
-- build(L, M, !dvx); build(M + 1, R, !dvx); } -----
- void dfs(int L, int R, bool dvx) { -----
-- if (L >= R) return; -----
-- int M = (L + R) / 2; -----
-- double dx = qx - p[M].x, dy = qy - p[M].y; -----
-- double delta = dvx ? dx : dy; -----
-- double D = dx * dx + dy * dy; -----
-- if (D<=prune && (pq.size()<k||D<pq.top().first)) { -----
-- pq.push(make_pair(D, &p[M])); -----
-- if (pq.size() > k) pq.pop(); } -----
-- int nL = L, nR = M, fL = M + 1, fR = R; -----
-- if (delta > 0) {swap(nL, fL); swap(nR, fR);} -----
-- dfs(nL, nR, !dvx); -----
-- D = delta * delta; -----
```

```
--- if (D<=prune && (pq.size())<k||D<pq.top().first)) -----
--- dfs(fL, fR, !dvx); } -----
- // returns k nearest neighbors of (x, y) in tree -----
- // usage: vector<point> ans = tree.knn(x, y, 2); -----
- vector<point> knn(double x, double y,
-----
-     int k=1, double r=-1) {
-----
--- qx=x; qy=y; this->k=k; prune=r<0?HUGE_VAL:r*r;
--- dfs(0, n, false); vector<point> v;
--- while (!pq.empty()) {
-----
---     v.push_back(*pq.top().second);
-----
---     pq.pop();
--- } reverse(v.begin(), v.end());
--- return v; } };
```

3.20. **Line Sweep (Closest Pair).** Get the closest pair distance of a set of points in $O(n\log n)$ by sweeping a line and keeping a bounded rectangle. Modifiable for other metrics such as Minkowski and Manhattan distance. For external point queries, see kD Tree.

```
bool cmpy(const point& a, const point& b) { return a.y < b.y; }
double closest_pair_sweep(point p[], int n) {
- if (n <= 1) return HUGE_VAL;
- std::sort(p, p + n, cmpy);
- std::set<point> box; box.insert(p[0]);
- double best = 1e13; // infinity, but not HUGE_VAL
- for (int L = 0, i = 1; i < n; ++i) {
-----
-     while(L < i && p[i].y - p[L].y > best)
-----
-         box.erase(p[L++]);
-----
-     point bound(p[i].x - best, p[i].y - best);
-----
-     std::set<point>::iterator it = box.lower_bound(bound);
-----
-     while (it != box.end() && p[i].x+best >= it->x){
-----
-         double dx = p[i].x - it->x;
-----
-         double dy = p[i].y - it->y;
-----
-         best = std::min(best, std::sqrt(dx*dx + dy*dy));
-----
-         ++it; }
-----
-     box.insert(p[i]);
- } return best; }
```

3.21. **Formulas.** Let $a = (a_x, a_y)$ and $b = (b_x, b_y)$ be two-dimensional vectors.

- $a \cdot b = |a||b| \cos \theta$, where θ is the angle between a and b .
- $a \times b = |a||b| \sin \theta$, where θ is the signed angle between a and b .
- $a \times b$ is equal to the area of the parallelogram with two of its sides formed by a and b . Half of that is the area of the triangle formed by a and b .
- The line going through a and b is $Ax + By = C$ where $A = b_y - a_y$, $B = a_x - b_x$, $C = Aa_x + Ba_y$.
- Two lines $A_1x + B_1y = C_1$, $A_2x + B_2y = C_2$ are parallel iff. $D = A_1B_2 - A_2B_1$ is zero. Otherwise their unique intersection is $(B_2C_1 - B_1C_2, A_1C_2 - A_2C_1)/D$.
- **Euler's formula:** $V - E + F = 2$
- Side lengths a, b, c can form a triangle iff. $a + b > c$, $b + c > a$ and $a + c > b$.
- Sum of internal angles of a regular convex n -gon is $(n - 2)\pi$.
- **Law of sines:** $\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$
- **Law of cosines:** $b^2 = a^2 + c^2 - 2accos B$
- Internal tangents of circles $(c_1, r_1), (c_2, r_2)$ intersect at $(c_1r_2 + c_2r_1)/(r_1 + r_2)$, external intersect at $(c_1r_2 - c_2r_1)/(r_1 + r_2)$.

4. GRAPHS

4.1. Single-Source Shortest Paths.

4.1.1. Dijkstra.

```
#include "graph_template_adjlist.cpp"
// insert inside graph; needs n, dist[], and adj[]
void dijkstra(int s) {
- for (int u = 0; u < n; ++u)
-----
-     dist[u] = INF;
-----
-     dist[s] = 0;
-----
-     std::priority_queue<ii, vii, std::greater<ii> > pq;
-     pq.push({0, s});
-     while (!pq.empty()) {
-----
-         int u = pq.top().second;
-----
-         int d = pq.top().first;
-----
-         pq.pop();
-----
-         if (dist[u] < d)
-----
-             continue;
-----
-         dist[u] = d;
-----
-         for (auto &e : adj[u]) {
-----
-             int v = e.first;
-----
-             int w = e.second;
-----
-             if (dist[v] > dist[u] + w) {
-----
-                 dist[v] = dist[u] + w;
-----
-                 pq.push({dist[v], v}); } } } }
```

4.1.2. Bellman-Ford.

```
#include "graph_template_adjlist.cpp"
// insert inside graph; needs n, dist[], and adj[]
void bellman_ford(int s) {
- for (int u = 0; u < n; ++u)
-----
-     dist[u] = INF;
-----
-     dist[s] = 0;
-----
-     for (int i = 0; i < n-1; ++i)
-----
-         for (int u = 0; u < n; ++u)
-----
-             for (auto &e : adj[u])
-----
-                 if (dist[u] + e.second < dist[e.first])
-----
-                     dist[e.first] = dist[u] + e.second; }
// you can call this after running bellman_ford()
bool has_neg_cycle() {
- for (int u = 0; u < n; ++u)
-----
-     for (auto &e : adj[u])
-----
-         if (dist[e.first] > dist[u] + e.second)
-----
-             return true;
-----
-     return false; }
```

4.1.3. Shortest Path Faster Algorithm.

```
#include "graph_template_adjlist.cpp"
// insert inside graph;
// needs n, dist[], in_queue[], num_vis[], and adj[]
bool spfa(int s) {
- for (int u = 0; u < n; ++u) {
-----
-     dist[u] = INF;
-----
-     in_queue[u] = 0;
-----
-     num_vis[u] = 0; }
-----
-     dist[s] = 0;
-----
-     in_queue[s] = 1;
```

```
- bool has_negative_cycle = false;
-----
- std::queue<int> q; q.push(s);
-----
- while (not q.empty()) {
-----
-     int u = q.front(); q.pop(); in_queue[u] = 0;
-----
-     if (++num_vis[u] >= n)
-----
-         dist[u] = -INF, has_negative_cycle = true;
-----
-     for (auto &[v, c] : adj[u])
-----
-         if (dist[v] > dist[u] + c) {
-----
-             dist[v] = dist[u] + c;
-----
-             if (!in_queue[v]) {
-----
-                 q.push(v);
-----
-                 in_queue[v] = 1; } } }
-     return has_negative_cycle; }
```

4.2. All-Pairs Shortest Paths.

4.2.1. Floyd-Washall.

```
#include "graph_template_adjmat.cpp"
// insert inside graph; needs n and mat[][]
void floyd_warshall() {
- for (int k = 0; k < n; ++k)
-----
-     for (int i = 0; i < n; ++i)
-----
-         for (int j = 0; j < n; ++j)
-----
-             if (mat[i][k] + mat[k][j] < mat[i][j])
-----
-                 mat[i][j] = mat[i][k] + mat[k][j]; }
```

4.3. Strongly Connected Components.

4.3.1. Kosaraju.

```
struct kosaraju_graph {
- int n, *vis;
- vi **adj;
- std::vector<vi> sccs;
- kosaraju_graph(int n) {
-----
-     this->n = n;
-----
-     vis = new int[n];
-----
-     adj = new vi*[2];
-----
-     for (int dir = 0; dir < 2; ++dir)
-----
-         adj[dir] = new vi[n]; }
-----
- void add_edge(int u, int v) {
-----
-     adj[0][u].push_back(v);
-----
-     adj[1][v].push_back(u); }
-----
- void dfs(int u, int p, int dir, vi &topo) {
-----
-     vis[u] = 1;
-----
-     for (int v : adj[dir][u])
-----
-         if (!vis[v] && v != p) dfs(v, u, dir, topo);
-----
-     topo.push_back(u); }
-----
- void kosaraju() {
-----
-     vi topo;
-----
-     for (int u = 0; u < n; ++u) vis[u] = 0;
-----
-     for (int u = 0; u < n; ++u) if(!vis[u]) dfs(u, -1, 0, topo);
-----
-     for (int u = 0; u < n; ++u) vis[u] = 0;
-----
-     for (int i = n-1; i >= 0; --i) {
-----
-         if (!vis[topo[i]]) {
-----
-             sccs.push_back({});
-----
-             dfs(topo[i], -1, 1, sccs.back()); } } } }
```


4.3.2. *Tarjan's Offline Algorithm* .

```
int n, id[N], low[N], st[N], in[N], TOP, ID;
int scc[N], SCC_SIZE; // 0 <= scc[u] < SCC_SIZE
vector<int> adj[N]; // 0-based adjlist
void dfs(int u) {
- id[u] = low[u] = ID++;
- st[TOP++] = u; in[u] = 1;
- for (int v : adj[u]) {
-- if (id[v] == -1) {
---- dfs(v);
---- low[u] = min(low[u], low[v]);
-- } else if (in[v] == 1)
---- low[u] = min(low[u], id[v]); }
- if (id[u] == low[u]) {
-- int sid = SCC_SIZE++;
-- do {
---- int v = st[--TOP];
---- in[v] = 0; scc[v] = sid;
-- } while (st[TOP] != u); } }
void tarjan() { // call tarjan() to load SCC
- memset(id, -1, sizeof(int) * n);
- SCC_SIZE = ID = TOP = 0;
- for (int i = 0; i < n; ++i)
-- if (id[i] == -1) dfs(i); }
```

4.4. **Minimum Mean Weight Cycle** . Run this for each strongly connected component

```
typedef std::vector<double> vd;
double min_mean_cycle(graph &g) {
- double mn = INF;
- std::vector<vd> dp(g.n+1, vd(g.n, mn));
- dp[0][0] = 0;
- for (int k = 1; k <= g.n; ++k)
-- for (int u = 0; u < g.n; ++u)
---- for (auto &[v, w]: g.adj[u])
----- dp[k][v] = std::min(ar[k][v], dp[k-1][u] + w);
- for (int k = 0; k < g.n; ++k) {
-- double mx = -INF;
-- for (int u = 0; u < g.n; ++u)
---- mx = std::max(mx, (dp[g.n][u] - dp[k][u]) / (g.n - k));
-- mn = std::min(mn, mx); }
- return mn; }
```

4.5. **Biconnected Components**.

4.5.1. *Bridges and Articulation Points*.

```
struct graph {
- int n, *disc, *low, TIME;
- vi *adj, stk, articulation_points;
- std::set<ii> bridges;
- vvi comps;
- graph (int n) : n(n) {
-- adj = new vi[n];
-- disc = new int[n];
-- low = new int[n]; }
- void add_edge(int u, int v) {
-- adj[u].push_back(v);
```

```
-- adj[v].push_back(u); }
- void _bridges_artics(int u, int p) {
-- disc[u] = low[u] = TIME++;
-- stk.push_back(u);
-- int children = 0;
-- bool has_low_child = false;
-- for (int v : adj[u]) {
---- if (disc[v] == -1) {
----- _bridges_artics(v, u);
----- children++;
---- if (disc[u] < low[v])
----- bridges.insert({std::min(u, v), std::max(u, v)});
---- if (disc[u] <= low[v]) {
----- has_low_child = true;
----- comps.push_back({u});
----- while (comps.back().back() != v and !stk.empty()) {
----- comps.back().push_back(stk.back());
----- stk.pop_back(); } }
---- low[u] = std::min(low[u], low[v]);
---- } else if (v != p)
---- low[u] = std::min(low[u], disc[v]); }
-- if ((p == -1 && children >= 2) ||
-- (p != -1 && has_low_child))
-- articulation_points.push_back(u); }
- void bridges_artics() {
-- for (int u = 0; u < n; ++u) disc[u] = -1;
-- stk.clear();
-- articulation_points.clear();
-- bridges.clear();
-- comps.clear();
-- TIME = 0;
-- for (int u = 0; u < n; ++u) if (disc[u] == -1)
---- _bridges_artics(u, -1); } }
```

4.5.2. *Block Cut Tree*.

```
// insert inside code for finding articulation points
graph build_block_cut_tree() {
- int bct_n = articulation_points.size() + comps.size();
- vi block_id(n), is_art(n, 0);
- graph tree(bct_n);
- for (int i = 0; i < articulation_points.size(); ++i) {
-- block_id[articulation_points[i]] = i;
-- is_art[articulation_points[i]] = 1; }
- for (int i = 0; i < comps.size(); ++i) {
-- int id = i + articulation_points.size();
-- for (int u : comps[i])
---- if (is_art[u]) tree.add_edge(block_id[u], id);
-- else
---- block_id[u] = id; }
- return tree; }
```

4.5.3. *Bridge Tree*.

```
// insert inside code for finding bridges
// requires union_find and hasher
graph build_bridge_tree() {
- union_find uf(n);
- for (int u = 0; u < n; ++u) {
-- for (int v : adj[u]) {
```

```
-- ii uv = { std::min(u, v), std::max(u, v) };
-- if (bridges.find(uv) == bridges.end())
-- uf.unite(u, v); } }
- hasher h;
- for (int u = 0; u < n; ++u)
-- if (u == uf.find(u)) h.get_hash(u);
- int tn = h.h.size();
- graph tree(tn);
- for (int i = 0; i < M; ++i) {
-- int ui = h.get_hash(uf.find(u));
-- int vi = h.get_hash(uf.find(v));
-- if (ui != vi) tree.add_edge(ui, vi); }
- return tree; }
```

4.6. **Minimum Spanning Tree**.

4.6.1. *Kruskal*.

```
#include "graph_template_edgelist.cpp"
#include "union_find.cpp"
// insert inside graph; needs n, and edges
void kruskal(viii &res) {
- viii().swap(res); // or use res.clear();
- std::priority_queue<iii, viii, std::greater<iii> > pq;
- for (auto &edge : edges)
-- pq.push(edge);
- union_find uf(n);
- while (!pq.empty()) {
-- auto node = pq.top(); pq.pop();
-- int u = node.second.first;
-- int v = node.second.second;
-- if (uf.unite(u, v))
---- res.push_back(node); } }
```

4.6.2. *Prim*.

```
#include "graph_template_adjlist.cpp"
// insert inside graph; needs n, vis[], and adj[]
void prim(viii &res, int s=0) {
- res.clear();
- std::priority_queue<iii, viii, std::greater<iii> > pq;
- vis[s] = true;
- for (auto &[v, w] : adj[s])
-- if (!vis[v]) pq.push({w, {s, v}});
- while (!pq.empty()) {
-- auto edge = pq.top(); pq.pop();
-- int u = edge.second.second;
-- if (vis[u]) continue;
-- vis[u] = true;
-- res.push_back(edge);
-- for (auto &[v, w] : adj[u])
---- if (!vis[v]) pq.push({w, {u, v}});}
```

4.7. **Euler Path/Cycle** .

4.7.1. Euler Path/Cycle in a Directed Graph .

```
#define MAXV 1000
#define MAXE 5000
int indeg[MAXV], outdeg[MAXV], res[MAXE + 1];
ii start_end(graph &g) {
- int start = -1, end = -1, any = 0, c = 0;
- for (int u = 0; u < n; ++u) {
--- if (outdeg[u] > 0) any = u;
--- if (indeg[u] + 1 == outdeg[u]) start = u, c++;
--- else if (indeg[u] == outdeg[u] + 1) end = u, c++;
--- else if (indeg[u] != outdeg[u]) return {-1, -1}; }
- if ((start == -1) != (end == -1) || (c != 2 && c != 0))
- return {-1, -1};
- if (start == -1) start = end = any;
- return {start, end}; }
bool euler_path(graph &g) {
- ii se = start_end(g);
- int cur = se.first, at = g.edges.size() + 1;
- if (cur == -1) return false;
- std::stack<int> s;
- while (true) {
--- if (outdeg[cur] == 0) {
--- res[at] = cur;
--- if (s.empty()) break;
--- cur = s.top(); s.pop();
--- } else s.push(cur), cur = g.adj[cur][--outdeg[cur]]; }
- return at == 0; }
```

4.7.2. Euler Path/Cycle in an Undirected Graph .

```
std::multiset<int> adj[1010];
std::list<int> L;
std::list<int>::iterator euler(
- int at, int to, std::list<int>::iterator it
) {
- if (at == to) return it;
- L.insert(it, at), --it;
- while (!adj[at].empty()) {
--- int nxt = *adj[at].begin();
--- adj[at].erase(adj[at].find(nxt));
--- adj[nxt].erase(adj[nxt].find(at));
--- if (to == -1) {
----- it = euler(nxt, at, it);
----- L.insert(it, at);
----- --it;
--- } else {
----- it = euler(nxt, to, it);
----- to = -1; } }
- return it; }
// euler(0,-1,L.begin())
```

4.8. Bipartite Matching .

4.8.1. Alternating Paths Algorithm .

```
vi* adj;
bool* done; // initially all false
int* owner; // initially all -1
```

```
int alternating_path(int left) {
- if (done[left]) return 0;
- done[left] = true;
- for (int right : adj[left]) {
--- if (owner[right] == -1 || alternating_path(owner[right])) {
---- owner[right] = left; return 1; } }
- return 0; }
```

4.8.2. Hopcroft-Karp Algorithm .

```
#define MAXN 5000
int dist[MAXN+1], q[MAXN+1];
#define dist(v) dist[v == -1 ? MAXN : v]
struct bipartite_graph {
- int n, m, *L, *R; vi *adj;
- bipartite_graph(int n, int m) : n(n), m(m),
- L(new int[n]), R(new int[m]), adj(new vi[n]) {
~bipartite_graph() { delete[] adj; delete[] L; delete[] R; }
- void add_edge(int u, int v) { adj[u].push_back(v); }
- bool bfs() {
- int l = 0, r = 0;
- for (int v = 0; v < n; ++v)
--- if (L[v] == -1) dist[v] = 0, q[r++] = v;
--- else dist[v] = INF;
- dist(-1) = INF;
- while (l < r) {
- int v = q[l++];
- if (dist[v] < dist(-1))
- for (int u : adj[v])
--- if (dist(R[u]) == INF) {
---- dist(R[u]) = dist[v] + 1;
---- q[r++] = R[u]; } }
- return dist(-1) != INF; }
- bool dfs(int v) {
- if (v != -1) {
- for (int u : adj[v])
--- if (dist(R[u]) == dist[v] + 1)
--- if (dfs(R[u])) { R[u] = v; L[v] = u; return true; }
- dist(v) = INF;
- return false; }
- return true; }
- int maximum_matching() {
- int matching = 0;
- for (int u = 0; u < n; ++u)
- L[u] = R[u] = -1;
- while (bfs())
- for (int u = 0; u < n; ++u)
--- matching += L[u] == -1 && dfs(u);
- return matching; } };
```

4.8.3. Minimum Vertex Cover in Bipartite Graphs .

```
#include "hopcroft_karp.cpp"
std::vector<bool> alt;
void dfs(bipartite_graph &g, int u) {
- alt[u] = true;
- for (int v : g.adj[u]) {
--- alt[v + g.n] = true;
```

```
--- if (g.R[v] != -1 && !alt[g.R[v]])
--- dfs(g, g.R[v]); } }
vi mvc_bipartite(bipartite_graph &g) {
- vi res; g.maximum_matching();
- alt.assign(g.n + g.m, false);
- for (int i = 0; i < g.n; ++i) if (g.L[i] == -1) dfs(g, i);
- for (int i = 0; i < g.n; ++i) if (!alt[i]) res.push_back(i);
- for (int i = 0; i < g.m; ++i)
--- if (alt[g.n + i]) res.push_back(g.n + i);
- return res; }
```

4.9. Maximum Flow.

4.9.1. Edmonds-Karp . $O(VE^2)$

4.9.2. Dinic. $O(V^2E)$

```
struct flow_network_dinic {
- struct edge {
--- int u, v; ll c, f;
--- edge(int u, int v, ll c) : u(u), v(v), c(c), f(0) {} };
- int n;
- std::vector<int> adj_ptr, par, dist;
- std::vector<std::vector<int>> adj;
- std::vector<edge> edges;
- flow_network_dinic(int n) : n(n) {
--- std::vector<std::vector<int>>(n).swap(adj);
--- reset(); }
- void reset() {
--- std::vector<int>(n).swap(adj_ptr);
--- std::vector<int>(n).swap(par);
--- std::vector<int>(n).swap(dist);
--- for (edge &e : edges) e.f = 0; }
- void add_edge(int u, int v, ll c, bool bi = false) {
--- adj[u].push_back(edges.size());
--- edges.push_back(edge(u, v, c));
--- adj[v].push_back(edges.size());
--- edges.push_back(edge(v, u, (bi ? c : 0LL))); }
- ll res(const edge &e) { return e.c - e.f; }
- bool make_level_graph(int s, int t) {
--- for (int u = 0; u < n; ++u) dist[u] = -1;
--- dist[s] = 0;
--- std::queue<int> q; q.push(s);
--- while (!q.empty()) {
--- int u = q.front(); q.pop();
--- for (int i : adj[u]) {
----- edge &e = edges[i];
----- if (dist[e.v] < 0 and res(e)) {
----- dist[e.v] = dist[u] + 1;
----- q.push(e.v); } } }
--- return dist[t] != -1; }
- bool is_next(int u, int v) {
--- return dist[v] == dist[u] + 1; }
- bool dfs(int u, int t) {
--- if (u == t) return true;
--- for (int &ii = adj_ptr[u]; ii < adj[u].size(); ++ii) {
--- int i = adj[u][ii];
--- edge &e = edges[i];
```

```

----- if (is_next(u, e.v) and res(e) > 0 and dfs(e.v, t)) {
-----   par[e.v] = i;
-----   return true; } }
----- return false; }
- bool aug_path(int s, int t) {
-   for (int u = 0; u < n; ++u) par[u] = -1;
-   return dfs(s, t); }
- ll calc_max_flow(int s, int t) {
-   ll total_flow = 0;
-   while (make_level_graph(s, t)) {
-       for (int u = 0; u < n; ++u) adj_ptr[u] = 0;
-       while (aug_path(s, t)) {
-           ll flow = pvl::LL_INF;
-           for (int i = par[t]; i != -1; i = par[edges[i].u])
-               flow = std::min(flow, res(edges[i]));
-           for (int i = par[t]; i != -1; i = par[edges[i].u]) {
-               edges[i].f += flow;
-               edges[i^1].f -= flow; }
-           total_flow += flow; } }
-   return total_flow; }
- std::vector<bool> min_cut(int s, int t) {
-   calc_max_flow(s, t);
-   assert(!make_level_graph(s, t));
-   std::vector<bool> cut_mem(n);
-   for (int u = 0; u < n; ++u)
-       cut_mem[u] = (dist[u] != -1);
-   return cut_mem; } };

```

4.9.3. Push-relabel. $\omega(VE + V^2\sqrt{E})$, $O(V^3)$

```

int n;
std::vector<vi> capacity, flow;
vi height, excess;
void push(int u, int v) {
-   int d = min(excess[u], capacity[u][v] - flow[u][v]);
-   flow[u][v] += d;   flow[v][u] -= d;
-   excess[u] -= d;   excess[v] += d; }
void relabel(int u) {
-   int d = INF;
-   for (int i = 0; i < n; i++)
-       if (capacity[u][i] - flow[u][i] > 0)
-           d = min(d, height[i]);
-   if (d < INF) height[u] = d + 1; }
vi find_max_height_vertices(int s, int t) {
-   vi max_height;
-   for (int i = 0; i < n; i++) {
-       if (i != s && i != t && excess[i] > 0) {
-           if (!max_height.empty() && height[i] > height[max_height[0]])
-               max_height.clear();
-           if (max_height.empty() || height[i] == height[max_height[0]])
-               max_height.push_back(i); } }
-   return max_height; }
int max_flow(int s, int t) {
-   flow.assign(n, vi(n, 0));
-   height.assign(n, 0);   height[s] = n;
-   excess.assign(n, 0);   excess[s] = INF;
-   for (int i = 0; i < n; i++) if (i != s) push(s, i);

```

```

-   vi current;
-   while (!current.empty()) {
-       for (int i : current) {
-           bool pushed = false;
-           for (int j = 0; j < n && excess[i]; j++) {
-               if (capacity[i][j] - flow[i][j] > 0 &&
-                   height[i] == height[j] + 1) {
-                   push(i, j);
-                   pushed = true; } }
-           if (!pushed) relabel(i), break; } }
-   int max_flow = 0;
-   for (int i = 0; i < n; i++) max_flow += flow[i][t];
-   return max_flow; }

```

4.9.4. Gomory-Hu (All-pairs Maximum Flow). $O(V^3E)$, possibly amortized $O(V^2E)$ with a big constant factor.

```

#include "dinic.cpp"
struct gomory_hu_tree {
-   int n;
-   std::vector<int> dep;
-   std::vector<std::pair<int, ll>> par;
-   explicit gomory_hu_tree(flow_network_dinic &g) : n(g.n) {
-       std::vector<std::pair<int, ll>>(n, {0, 0LL}).swap(par);
-       std::vector<int>(n, 0).swap(dep);
-       std::vector<int> temp_par(n, 0);
-       for (int u = 1; u < n; ++u) {
-           g.reset();
-           ll flow = g.calc_max_flow(u, temp_par[u]);
-           std::vector<bool> cut_mem = g.min_cut(u, temp_par[u]);
-           for (int v = u+1; v < n; ++v)
-               if (cut_mem[u] == cut_mem[v]
-                   and temp_par[u] == temp_par[v])
-                   temp_par[v] = u;
-           add_edge(temp_par[u], u, flow); } }
-   void add_edge(int u, int v, ll w) {
-       par[v] = {u, w}; dep[v] = dep[u] + 1; }
-   ll calc_max_flow(int s, int t) {
-       ll ans = pvl::LL_INF;
-       while (dep[s] > dep[t]) {
-           ans = std::min(ans, par[s].second); s = par[s].first; }
-       while (dep[s] < dep[t]) {
-           ans = std::min(ans, par[t].second); t = par[t].first; }
-       while (s != t) {
-           ans = std::min(ans, par[s].second); s = par[s].first;
-           ans = std::min(ans, par[t].second); t = par[t].first; }
-       return ans; } };

```

4.10. Minimum Cost Maximum Flow.

```

struct edge {
-   int u, v; ll cost, cap, flow;
-   edge(int u, int v, ll cap, ll cost) :
-       u(u), v(v), cap(cap), cost(cost), flow(0) {} };
struct flow_network {
-   int n, s, t, *par, *in_queue, *num_vis; ll *dist, *pot;
-   std::vector<edge> edges;
-   std::vector<int> *adj;
-   std::map<std::pair<int, int>, std::vector<int>> edge_idx;

```

```

-   flow_network(int n, int s, int t) : n(n), s(s), t(t) {
-       adj = new std::vector<int>(n);
-       par = new int[n];
-       in_queue = new int[n];
-       num_vis = new int[n];
-       dist = new ll[n];
-       pot = new ll[n];
-       for (int u = 0; u < n; ++u) pot[u] = 0; }
-   void add_edge(int u, int v, ll cap, ll cost) {
-       adj[u].push_back(edges.size());
-       edge_idx[{u, v}].push_back(edges.size());
-       edges.push_back(edge(u, v, cap, cost));
-       adj[v].push_back(edges.size());
-       edge_idx[{v, u}].push_back(edges.size());
-       edges.push_back(edge(v, u, 0LL, -cost)); }
-   ll get_flow(int u, int v) {
-       ll f = 0;
-       for (int i : edge_idx[{u, v}]) f += edges[i].flow;
-       return f; }
-   ll res(edge &e) { return e.cap - e.flow; }
-   void bellman_ford() {
-       for (int u = 0; u < n; ++u) pot[u] = INF;
-       pot[s] = 0;
-       for (int it = 0; it < n-1; ++it)
-           for (auto e : edges)
-               if (res(e) > 0)
-                   pot[e.v] = std::min(pot[e.v], pot[e.u] + e.cost); }
-   bool spfa() {
-       std::queue<int> q; q.push(s);
-       while (not q.empty()) {
-           int u = q.front(); q.pop(); in_queue[u] = 0;
-           if (++num_vis[u] >= n) {
-               dist[u] = -INF;
-               return false; }
-           for (int i : adj[u]) {
-               edge e = edges[i];
-               if (res(e) <= 0) continue;
-               ll nd = dist[u] + e.cost + pot[u] - pot[e.v];
-               if (dist[e.v] > nd) {
-                   dist[e.v] = nd;
-                   par[e.v] = i;
-                   if (not in_queue[e.v]) {
-                       q.push(e.v);
-                       in_queue[e.v] = 1; } } } }
-       return dist[t] != INF; }
-   bool aug_path() {
-       for (int u = 0; u < n; ++u) {
-           par[u] = -1;
-           in_queue[u] = 0;
-           num_vis[u] = 0;
-           dist[u] = INF; }
-       dist[s] = 0;
-       in_queue[s] = 1;
-       return spfa();
-   }
-   pll calc_max_flow(bool do_bellman_ford=false) {

```

```

--- ll total_cost = 0, total_flow = 0;
--- if (do_bellman_ford)
---     bellman_ford();
--- while (aug_path()) {
---     ll f = INF;
---     for (int i = par[t]; i != -1; i = par[edges[i].u])
---         f = std::min(f, res(edges[i]));
---     for (int i = par[t]; i != -1; i = par[edges[i].u]) {
---         edges[i].flow += f;
---         edges[i^1].flow -= f;
---     }
---     total_cost += f * (dist[t] + pot[t] - pot[s]);
---     total_flow += f;
---     for (int u = 0; u < n; ++u)
---         if (par[u] != -1) pot[u] += dist[u];
---     return {total_cost, total_flow};
}

```

4.10.1. Hungarian Algorithm.

```

int n, m; // size of A, size of B
int cost[N+1][N+1]; // input cost matrix, 1-indexed
int way[N+1], p[N+1]; // p[i]=j: Ai is matched to Bj
int minv[N+1], A[N+1], B[N+1]; bool used[N+1];
int hungarian() {
    for (int i = 0; i <= N; ++i)
        A[i] = B[i] = p[i] = way[i] = 0; // init
    for (int i = 1; i <= n; ++i) {
        p[0] = i; int R = 0;
        for (int j = 0; j <= m; ++j)
            minv[j] = INF, used[j] = false;
        do {
            int L = p[R], dR = 0;
            int delta = INF;
            used[R] = true;
            for (int j = 1; j <= m; ++j)
                if (!used[j]) {
                    int c = cost[L][j] - A[L] - B[j];
                    if (c < minv[j]) minv[j] = c, way[j] = R;
                    if (minv[j] < delta) delta = minv[j], dR = j;
                }
            for (int j = 0; j <= m; ++j)
                if (used[j]) A[p[j]] += delta, B[j] -= delta;
                else minv[j] -= delta;
            R = dR;
        } while (p[R] != 0);
        for (; R != 0; R = way[R])
            p[R] = p[way[R]];
        return -B[0];
    }
}

```

4.11. **Minimum Arborescence**. Given a weighted directed graph, finds a subset of edges of minimum total weight so that there is a unique path from the root r to each vertex. Returns a vector of size n , where the i th element is the edge for the i th vertex. The answer for the root is undefined!

```

#include "../data-structures/union_find.cpp"
struct arborescence {
    int n; union_find uf;
    vector<vector<pair<int,int>>> adj;
    arborescence(int _n) : n(_n), uf(n), adj(n) {}
}

```

```

void add_edge(int a, int b, int c) {
    adj[b].push_back(make_pair(ii(a,b),c));
}
vii find_min(int r) {
    vi vis(n,-1), mn(n,INF); vii par(n);
    rep(i,0,n) {
        if (uf.find(i) != i) continue;
        int at = i;
        while (at != r && vis[at] == -1) {
            vis[at] = i;
            iter(it,adj[at]) if (it->second < mn[at] &&
                uf.find(it->first.first) != at)
                mn[at] = it->second, par[at] = it->first;
            if (par[at] == ii(0,0)) return vii();
            at = uf.find(par[at].first);
        }
        if (at == r || vis[at] != i) continue;
        union_find tmp = uf; vi seq;
        do { seq.push_back(at); at = uf.find(par[at].first);
        } while (at != seq.front());
        iter(it,seq) uf.unite(*it,seq[0]);
        int c = uf.find(seq[0]);
        vector<pair<int,int>> > nw;
        iter(it,seq) iter(jt,adj[*it])
            nw.push_back(make_pair(jt->first,
                jt->second - mn[*it]));
        adj[c] = nw;
        vii rest = find_min(r);
        if (size(rest) == 0) return rest;
        ii use = rest[c];
        rest[at = tmp.find(use.second)] = use;
        iter(it,seq) if (*it != at)
            rest[*it] = par[*it];
        return rest;
    }
    return par;
}

```

4.12. **Blossom algorithm**. Finds a maximum matching in an arbitrary graph in $O(|V|^4)$ time. Be wary of loop edges.

```

#define MAXV 300
bool marked[MAXV], emarked[MAXV][MAXV];
int S[MAXV];
vi find_augmenting_path(const vector<vi> &adj, const vi &m){
    int n = size(adj), s = 0;
    vi par(n,-1), height(n), root(n,-1), q, a, b;
    memset(marked,0,sizeof(marked));
    memset(emarked,0,sizeof(emarked));
    rep(i,0,n) if (m[i] >= 0) emarked[i][m[i]] = true;
        else root[i] = i, S[s++] = i;
    while (s) {
        int v = S[--s];
        iter(wt,adj[v]) {
            int w = *wt;
            if (emarked[v][w]) continue;
            if (root[w] == -1) {
                int x = S[s++] = m[w];
                par[w]=v, root[w]=root[v], height[w]=height[v]+1;
                par[x]=w, root[x]=root[w], height[x]=height[w]+1;
            } else if (height[w] % 2 == 0) {

```

```

                if (root[v] != root[w]) {
                    while (v != -1) q.push_back(v), v = par[v];
                    reverse(q.begin(), q.end());
                    while (w != -1) q.push_back(w), w = par[w];
                    return q;
                } else {
                    int c = v;
                    while (c != -1) a.push_back(c), c = par[c];
                    c = w;
                    while (c != -1) b.push_back(c), c = par[c];
                    while (!a.empty() && !b.empty() && a.back() == b.back())
                        c = a.back(), a.pop_back(), b.pop_back();
                    memset(marked,0,sizeof(marked));
                    fill(par.begin(), par.end(), 0);
                    iter(it,a) par[*it] = 1; iter(it,b) par[*it] = 1;
                    par[c] = s = 1;
                    rep(i,0,n) root[par[i]] = par[i] ? 0 : s++; i;
                    vector<vi> adj2(s);
                    rep(i,0,n) iter(it,adj[i]) {
                        if (par[*it] == 0) continue;
                        if (par[i] == 0) {
                            if (!marked[par[*it]]) {
                                adj2[par[i]].push_back(par[*it]);
                                adj2[par[*it]].push_back(par[i]);
                                marked[par[*it]] = true;
                            } else adj2[par[i]].push_back(par[*it]);
                        }
                        vi m2(s, -1);
                        if (m[c] != -1) m2[m2[par[m[c]]] = 0] = par[m[c]];
                        rep(i,0,n) if (par[i] != 0 && m[i] != -1 && par[m[i]] != 0)
                            m2[par[i]] = par[m[i]];
                        vi p = find_augmenting_path(adj2, m2);
                        int t = 0;
                        while (t < size(p) && p[t] t++;
                        if (t == size(p)) {
                            rep(i,0,size(p)) p[i] = root[p[i]];
                            return p;
                        }
                        if (!p[0] || (m[c] != -1 && p[t+1] != par[m[c]]))
                            reverse(p.begin(), p.end()), t=(int)size(p)-t-1;
                        rep(i,0,t) q.push_back(root[p[i]]);
                        iter(it,adj[root[p[t-1]]) {
                            if (par[*it] != (s = 0)) continue;
                            a.push_back(c), reverse(a.begin(), a.end());
                            iter(jt,b) a.push_back(*jt);
                            while (a[s] != *it) s++;
                            if ((height[*it]&1)^(s<(int)size(a)-(int)size(b)))
                                reverse(a.begin(),a.end()), s=(int)size(a)-s-1;
                            while (a[s] != c) q.push_back(a[s]), s=(s+1)%size(a);
                            q.push_back(c);
                            rep(i,t+1,size(p)) q.push_back(root[p[i]]);
                            return q;
                        } }
                    emarked[v][w] = emarked[w][v] = true;
                }
                marked[v] = true;
            } return q;
        }
    } vii max_matching(const vector<vi> &adj) {
        vi m(size(adj), -1), ap; vii res, es;
        rep(i,0,size(adj)) iter(it,adj[i]) es.emplace_back(i,*it);
        random_shuffle(es.begin(), es.end());
    }
}

```

```
- iter(it,es) if (m[it->first] == -1 && m[it->second] == -1) -
--- m[it->first] = it->second, m[it->second] = it->first; ---
- do { ap = find_augmenting_path(adj, m); ---
----- rep(i,0,size(ap)) m[m[ap[i^1]] = ap[i]] = ap[i^1]; ---
- } while (!ap.empty()); ---
- rep(i,0,size(m)) if (i < m[i]) res.emplace_back(i, m[i]); --
- return res; } -----
```

4.13. **Maximum Density Subgraph.** Given (weighted) undirected graph G . Binary search density. If g is current density, construct flow network: (S, u, m) , $(u, T, m + 2g - d_u)$, $(u, v, 1)$, where m is a large constant (larger than sum of edge weights). Run floating-point max-flow. If minimum cut has empty S -component, then maximum density is smaller than g , otherwise it's larger. Distance between valid densities is at least $1/(n(n - 1))$. Edge case when density is 0. This also works for weighted graphs by replacing d_u by the weighted degree, and doing more iterations (if weights are not integers).

4.14. **Maximum-Weight Closure.** Given a vertex-weighted directed graph G . Turn the graph into a flow network, adding weight ∞ to each edge. Add vertices S, T . For each vertex v of weight w , add edge (S, v, w) if $w \geq 0$, or edge $(v, T, -w)$ if $w < 0$. Sum of positive weights minus minimum $S - T$ cut is the answer. Vertices reachable from S are in the closure. The maximum-weight closure is the same as the complement of the minimum-weight closure on the graph with edges reversed.

4.15. **Maximum Weighted Ind. Set in a Bipartite Graph.** This is the same as the minimum weighted vertex cover. Solve this by constructing a flow network with edges $(S, u, w(u))$ for $u \in L$, $(v, T, w(v))$ for $v \in R$ and (u, v, ∞) for $(u, v) \in E$. The minimum S, T -cut is the answer. Vertices adjacent to a cut edge are in the vertex cover.

4.16. **Synchronizing word problem.** A DFA has a synchronizing word (an input sequence that moves all states to the same state) iff. each pair of states has a synchronizing word. That can be checked using reverse DFS over pairs of states. Finding the shortest synchronizing word is NP-complete.

4.17. **Max flow with lower bounds on edges.** Change edge $(u, v, l \leq f \leq c)$ to $(u, v, f \leq c - l)$. Add edge (t, s, ∞) . Create super-nodes S, T . Let $M(u) = \sum_v l(v, u) - \sum_v l(u, v)$. If $M(u) < 0$, add edge $(u, T, -M(u))$, else add edge $(S, u, M(u))$. Max flow from S to T . If all edges from S are saturated, then we have a feasible flow. Continue running max flow from s to t in original graph.

4.18. **Tutte matrix for general matching.** Create an $n \times n$ matrix A . For each edge (i, j) , $i < j$, let $A_{ij} = x_{ij}$ and $A_{ji} = -x_{ij}$. All other entries are 0. The determinant of A is zero iff. the graph has a perfect matching. A randomized algorithm uses the Schwartz-Zippel lemma to check if it is zero.

4.19. Heavy Light Decomposition.

```
#include "segment_tree.cpp" -----
struct heavy_light_tree { -----
- int n, *par, *heavy, *dep, *path_root, *pos; -----
- std::vector<int> *adj; -----
- segtree *segment_tree; -----
- heavy_light_tree(int n) : n(n) { -----
--- this->adj = new std::vector<int>[n]; -----
```

```
--- segment_tree = new segtree(0, n-1); -----
--- par = new int[n]; -----
--- heavy = new int[n]; -----
--- dep = new int[n]; -----
--- path_root = new int[n]; -----
--- pos = new int[n]; } -----
- void add_edge(int u, int v) { -----
--- adj[u].push_back(v); -----
--- adj[v].push_back(u); } -----
- void decompose(int u, int &p) { -----
--- for (int v = u; v != -1; v = heavy[v]) -----
--- path_root[v] = u, pos[v] = p++; } -----
- void build(int root) { -----
--- for (int u = 0; u < n; ++u) heavy[u] = -1; -----
--- par[root] = root; dep[root] = 0; -----
--- dfs(root); -----
--- int p = 0; decompose(root, p); -----
--- for (int u = 0; u < n; ++u) -----
--- if (par[u] == -1 or heavy[par[u]] != u) -----
--- decompose(u, p); } -----
- int dfs(int u) { -----
--- sz = 1; -----
--- int max_subtree_sz = 0; -----
--- for (int v : adj[u]) { -----
--- if (v != par[u]) { -----
----- par[v] = u; -----
----- dep[v] = dep[u] + 1; -----
----- int subtree_sz = dfs(v); -----
----- if (max_subtree_sz < subtree_sz) { -----
----- max_subtree_sz = subtree_sz; -----
----- heavy[u] = v; } -----
----- sz += subtree_sz; } } -----
--- return sz; } -----
- int query(int u, int v) { -----
--- int res = 0; -----
--- while (path_root[u] != path_root[v]) { -----
----- if (dep[path_root[u]] > dep[path_root[v]]) swap(u, v); -----
----- res += segment_tree->sum(pos[path_root[v]], pos[v]); -----
----- v = par[path_root[v]]; } -----
--- res += segment_tree->sum(pos[u], pos[v]); -----
--- return res; } -----
- void update(int u, int v, int c) { -----
--- while (path_root[u] != path_root[v]) { -----
----- if (dep[path_root[u]] > dep[path_root[v]]) swap(u, v); -----
----- segment_tree->increase(pos[path_root[v]], pos[v], c); -----
----- v = par[path_root[v]]; } -----
--- segment_tree->increase(pos[u], pos[v], c); } };
```

4.20. Centroid Decomposition.

```
#define MAXV 100100 -----
#define LGMAXV 20 -----
int jmp[MAXV][LGMAXV], path[MAXV][LGMAXV], sz[MAXV],
- seph[MAXV], shortest[MAXV]; -----
struct centroid_decomposition { -----
- int n; vvi adj; -----
- centroid_decomposition(int _n) : n(_n), adj(n) { } -----
```

```
- void add_edge(int a, int b) { -----
--- adj[a].push_back(b); adj[b].push_back(a); } -----
- int dfs(int u, int p) { -----
--- sz[u] = 1; -----
--- for (int v : adj[u]) if (v != p) sz[u] += dfs(v, u); -----
--- return sz[u]; } -----
- void makepaths(int sep, int u, int p, int len) { -----
--- jmp[u][seph[sep]] = sep, path[u][seph[sep]] = len; -----
--- int bad = -1; -----
--- for (int i = 0; i < adj[u].size(); ++i) { -----
--- if (adj[u][i] == p) bad = i; -----
--- else makepaths(sep, adj[u][i], u, len + 1); } -----
--- if (p == sep) // remove bad edge -----
--- swap(adj[u][bad], adj[u].back()), adj[u].pop_back(); } -----
- void separate(int h=0, int u=0) { -----
--- dfs(u, -1); int sep = u; -----
--- down: -----
--- for (int nxt : adj[sep]) -----
--- if (sz[nxt] < sz[sep] && sz[nxt] > sz[u]/2) -----
--- sep = nxt, goto down; -----
--- seph[sep] = h, makepaths(sep, sep, -1, 0); -----
--- for (int i = 0; i < adj[sep].size(); ++i) -----
--- separate(h+1, adj[sep][i]); } -----
- void paint(int u) { -----
--- for (int h = 0; h < seph[u] + 1; ++h) -----
--- shortest[jmp[u][h]] = -----
--- std::min(shortest[jmp[u][h]], path[u][h]); } -----
- int closest(int u) { -----
--- int mn = INF/2; -----
--- for (int h = 0; h < seph[u] + 1; ++h) -----
--- mn = std::min(mn, path[u][h] + shortest[jmp[u][h]]); -----
--- return mn; } };
```

4.21. Least Common Ancestor.

4.21.1. Binary Lifting.

```
struct graph { -----
- int n, logn, *dep, **par; -----
- std::vector<int> *adj; -----
- graph(int n, int logn=20) : n(n), logn(logn) { -----
--- adj = new std::vector<int>[n]; -----
--- dep = new int[n]; -----
--- par = new int*[n]; -----
--- for (int i = 0; i < n; ++i) par[i] = new int[logn]; } -----
- void dfs(int u, int p, int d) { -----
--- dep[u] = d; -----
--- par[u][0] = p; -----
--- for (int v : adj[u]) -----
--- if (v != p) dfs(v, u, d+1); } -----
- int ascend(int u, int k) { -----
--- for (int i = 0; i < logn; ++i) -----
--- if (k & (1 << i)) u = par[u][i]; -----
--- return u; } -----
- int lca(int u, int v) { -----
--- if (dep[u] > dep[v]) u = ascend(u, dep[u] - dep[v]); -----
--- if (dep[v] > dep[u]) v = ascend(v, dep[v] - dep[u]); -----
--- if (u == v) return u; -----
```

```
--- for (int k = logn-1; k >= 0; --k) {
---- if (par[u][k] != par[v][k]) {
----- u = par[u][k]; v = par[v][k]; }
--- return par[u][0]; }
- bool is_anc(int u, int v) {
--- if (dep[u] < dep[v]) std::swap(u, v);
--- return ascend(u, dep[u] - dep[v]) == v; }
- void prep_lca(int root=0) {
--- dfs(root, root, 0);
--- for (int k = 1; k < logn; ++k)
---- for (int u = 0; u < n; ++u)
----- par[u][k] = par[par[u][k-1]][k-1]; } }
```

4.21.2. Euler Tour Sparse Table.

```
struct graph {
- int n, logn, *par, *dep, *first, *lg, **spt;
- vi *adj, euler; // spt size should be ~ 2n
- graph(int n, int logn=20) : n(n), logn(logn) {
--- adj = new vi[n];
--- par = new int[n];
--- dep = new int[n];
--- first = new int[n]; }
- void add_edge(int u, int v) {
--- adj[u].push_back(v); adj[v].push_back(u); }
- void dfs(int u, int p, int d) {
--- dep[u] = d; par[u] = p;
--- first[u] = euler.size();
--- euler.push_back(u);
--- for (int v : adj[u])
---- if (v != p) {
----- dfs(v, u, d+1);
----- euler.push_back(u); } }
- void prep_lca(int root=0) {
--- dfs(root, root, 0);
--- int en = euler.size();
--- lg = new int[en+1];
--- lg[0] = lg[1] = 0;
--- for (int i = 2; i <= en; ++i)
---- lg[i] = lg[i >> 1] + 1;
--- spt = new int*[en];
--- for (int i = 0; i < en; ++i) {
---- spt[i] = new int[lg[en]];
---- spt[i][0] = euler[i]; }
--- for (int k = 0; (2 << k) <= en; ++k)
---- for (int i = 0; i + (2 << k) <= en; ++i)
----- if (dep[spt[i][k]] < dep[spt[i+(1<<k)][k]])
----- spt[i][k+1] = spt[i][k];
---- else
----- spt[i][k+1] = spt[i+(1<<k)][k]; }
- int lca(int u, int v) {
--- int a = first[u], b = first[v];
--- if (a > b) std::swap(a, b);
--- int k = lg[b-a+1], ba = b - (1 << k) + 1;
--- if (dep[spt[a][k]] < dep[spt[ba][k]]) return spt[a][k];
--- return spt[ba][k]; } }
```

4.21.3. Tarjan Off-line LCA.

```
#include "data-structures/union_find.cpp"
struct tarjan_olca {
- vi ancestor, answers;
- vvi adj;
- vvi queries;
- std::vector<bool> colored;
- union_find uf;
- tarjan_olca(int n, vvi &adj) : adj(adj), uf(n) {
--- vi(n).swap(ancestor);
--- vvi(n).swap(queries);
--- std::vector<bool>(n, false).swap(colored); }
- void query(int x, int y) {
--- queries[x].push_back(ii(y, size(answers)));
--- queries[y].push_back(ii(x, size(answers)));
--- answers.push_back(-1); }
- void process(int u) {
--- ancestor[u] = u;
--- for (int v : adj[u]) {
---- process(v);
---- uf.unite(u, v);
---- ancestor[uf.find(u)] = u; }
--- colored[u] = true;
--- for (auto &[a, b]: queries[u])
---- if (colored[a]) answers[b] = ancestor[uf.find(a)];
--- } };
```

4.22. Counting Spanning Trees. Kirchoff's Theorem: The number of spanning trees of any graph is the determinant of any cofactor of the Laplacian matrix in $O(n^3)$.

- (1) Let A be the adjacency matrix.
- (2) Let D be the degree matrix (matrix with vertex degrees on the diagonal).
- (3) Get $D - A$ and delete exactly one row and column. Any row and column will do. This will be the cofactor matrix.
- (4) Get the determinant of this cofactor matrix using Gauss-Jordan.
- (5) Spanning Trees = $|\text{cofactor}(D - A)|$

4.23. Erdős-Gallai Theorem. A sequence of non-negative integers $d_1 \geq \dots \geq d_n$ can be represented as the degree sequence of finite simple graph on n vertices if and only if $d_1 + \dots + d_n$ is even and the following holds for $1 \leq k \leq n$:

$$\sum_{i=1}^n d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

4.24. Tree Isomorphism.

```
// REQUIREMENT: list of primes pr[], see prime sieve
typedef long long LL;
int pre[N], q[N], path[N]; bool vis[N];
// perform BFS and return the last node visited
int bfs(int u, vector<int> adj[]) {
--- memset(vis, 0, sizeof(vis));
--- int head = 0, tail = 0;
--- q[tail++] = u; vis[u] = true; pre[u] = -1;
--- while (head != tail) {
----- u = q[head]; if (++head == N) head = 0;
----- for (int i = 0; i < adj[u].size(); ++i) {
```

```
--- int v = adj[u][i];
--- if (!vis[v]) {
----- vis[v] = true; pre[v] = u;
----- q[tail++] = v; if (tail == N) tail = 0;
--- } }
--- return u;
} // returns the list of tree centers
vector<int> tree_centers(int r, vector<int> adj[]) {
--- int size = 0;
--- for (int u=bfs(bfs(r, adj), adj); u!=-1; u=pre[u])
---- path[size++] = u;
--- vector<int> med(1, path[size/2]);
--- if (size % 2 == 0) med.push_back(path[size/2-1]);
--- return med;
} // returns "unique hashcode" for tree with root u
LL rootcode(int u, vector<int> adj[], int p=-1, int d=15){
--- vector<LL> k; int nd = (d + 1) % primes;
--- for (int i = 0; i < adj[u].size(); ++i)
---- if (adj[u][i] != p)
----- k.push_back(rootcode(adj[u][i], adj, u, nd));
--- sort(k.begin(), k.end());
--- LL h = k.size() + 1;
--- for (int i = 0; i < k.size(); ++i)
---- h = h * pr[d] + k[i];
--- return h;
} // returns "unique hashcode" for the whole tree
LL treecode(int root, vector<int> adj[]) {
--- vector<int> c = tree_centers(root, adj);
--- if (c.size()==1)
----- return (rootcode(c[0], adj) << 1) | 1;
--- return (rootcode(c[0], adj)*rootcode(c[1], adj))<<1;
} // checks if two trees are isomorphic
bool isomorphic(int r1, vector<int> adj1[], int r2,
--- vector<int> adj2[], bool rooted = false) {
--- if (rooted)
----- return rootcode(r1, adj1) == rootcode(r2, adj2);
--- return treecode(r1, adj1) == treecode(r2, adj2); }
```

5. MATH I - ALGEBRA

5.1. Generating Function Manager.

```
const int DEPTH = 19;
const int ARR_DEPTH = (1<<DEPTH); //approx 5x10^5
const int SZ = 12;
ll temp[SZ][ARR_DEPTH+1];
const ll MOD = 998244353;
struct GF_Manager {
- int tC = 0;
- std::stack<int> to_be_freed;
- const static ll DEPTH = 23;
- ll prim[DEPTH+1], prim_inv[DEPTH+1], two_inv[DEPTH+1];
- ll mod_pow(ll base, ll exp) {
--- if(exp==0) return 1;
--- if(exp&1) return (base*mod_pow(base,exp-1))%MOD;
--- else return mod_pow((base*base)%MOD, exp/2); }
- void set_up_primitives() {
--- prim[DEPTH] = 31;
```



```

--- prim_inv[DEPTH] = mod_pow(prim[DEPTH], MOD-2); -----
--- two_inv[DEPTH] = mod_pow(1<<DEPTH,MOD-2); -----
--- for(int n = DEPTH-1; n >= 0; n--) { -----
----- prim[n] = (prim[n+1]*prim[n+1])%MOD; -----
----- prim_inv[n] = mod_pow(prim[n],MOD-2); -----
----- two_inv[n] = mod_pow(1<<n,MOD-2); } } -----
GF_Manager(){ set_up_primitives(); } -----
- void start_claiming(){ to_be_freed.push(0); } -----
- ll* claim(){ -----
  ++to_be_freed.top(); assert(tC < SZ); return temp[tC++]; }
- void end_claiming(){tC-=to_be_freed.top(); to_be_freed.pop();}
- void NTT(ll A[], int n, ll t[], -----
  bool is_inverse=false, int offset=0) { -----
- if (n==0) return; -----
- //Put the evens first, then the odds -----
- for (int i = 0; i < (1<<(n-1)); i++) { -----
  t[i] = A[offset+2*i]; -----
  t[i+(1<<(n-1))] = A[offset+2*i+1]; } -----
- for(int i = 0; i < (1<<n); i++) -----
  A[offset+i] = t[i]; -----
- NTT(A, n-1, t, is_inverse, offset); -----
- NTT(A, n-1, t, is_inverse, offset+(1<<(n-1))); -----
- ll w1 = (is_inverse ? prim_inv[n] : prim[n]), w = 1; -----
- for (int i = 0; i < (1<<(n-1)); i++, w=(w*w1)%MOD) { -----
  t[i] = (A[offset+i] + w*A[offset+(1<<(n-1))+i])%MOD; -----
  t[i+(1<<(n-1))] = (A[offset+i] -----
  w*A[offset+(1<<(n-1))+i])%MOD; } -----
- for (int i = 0; i < (1<<n); i++) A[offset+i] = t[i]; -----
- } -----
- int add(ll A[], int an, ll B[], int bn, ll C[]) { -----
- int cn = 0; -----
- for(int i = 0; i < max(an,bn); i++) { -----
  C[i] = A[i]+B[i]; -----
  if(C[i] <= -MOD) C[i] += MOD; -----
  if(MOD <= C[i]) C[i] -= MOD; -----
  if(C[i]!=0) cn = i; } -----
- return cn; } -----
- int subtract(ll A[], int an, ll B[], int bn, ll C[]) { -----
- int cn = 0; -----
- for(int i = 0; i < max(an,bn); i++) { -----
  C[i] = A[i]-B[i]; -----
  if(C[i] <= -MOD) C[i] += MOD; -----
  if(MOD <= C[i]) C[i] -= MOD; -----
  if(C[i]!=0) cn = i; } -----
- return cn+1; } -----
- int scalar_mult(ll v, ll A[], int an, ll C[]) { -----
- for(int i = 0; i < an; i++) C[i] = (v*A[i])%MOD; -----
- return v==0 ? 0 : an; } -----
- int mult(ll A[], int an, ll B[], int bn, ll C[]) { -----
- start_claiming(); -----
- // make sure you've called setup prim first -----
- // note: an and bn refer to the *number of items in -----
- // each array*, NOT the degree of the largest term -----
- int n, degree = an+bn-1; -----
- for(n=0; (1<<n) < degree; n++); -----
- ll *tA = claim(), *tB = claim(), *t = claim(); -----
  copy(A,A+an,tA); fill(tA+an,tA+(1<<n),0); -----
  copy(B,B+bn,tB); fill(tB+bn,tB+(1<<n),0); -----
  NTT(tA,n,t); -----
  NTT(tB,n,t); -----
  for(int i = 0; i < (1<<n); i++) -----
    tA[i] = (tA[i]*tB[i])%MOD; -----
  NTT(tA,n,t,true); -----
  scalar_mult(two_inv[n],tA,degree,C); -----
  end_claiming(); -----
  return degree; } -----
- int reciprocal(ll F[], int fn, ll R[]) { -----
- start_claiming(); -----
- ll *tR = claim(), *tempR = claim(); -----
- int n; for(n=0; (1<<n) < fn; n++); -----
- fill(tempR,tempR+(1<<n),0); -----
- tempR[0] = mod_pow(F[0],MOD-2); -----
- for (int i = 1; i <= n; i++) { -----
  mult(tempR,1<<i,F,1<<i,tR); -----
  tR[0] -= 2; -----
  scalar_mult(-1,tR,1<<i,tR); -----
  mult(tempR,1<<i,tR,1<<i,tempR); } -----
- copy(tempR,tempR+fn,R); -----
- end_claiming(); -----
- return n; } -----
- int quotient(ll F[], int fn, ll G[], int gn, ll Q[]) { -----
- start_claiming(); -----
- ll* revF = claim(); -----
- ll* revG = claim(); -----
- ll* tempQ = claim(); -----
- copy(F,F+fn,revF); reverse(revF,revF+fn); -----
- copy(G,G+gn,revG); reverse(revG,revG+gn); -----
- int qn = fn-gn+1; -----
- reciprocal(revG,qn,revG); -----
- mult(revF,qn,revG,qn,tempQ); -----
- reverse(tempQ,tempQ+qn); -----
- copy(tempQ,tempQ+qn,Q); -----
- end_claiming(); -----
- return qn; } -----
- int mod(ll F[], int fn, ll G[], int gn, ll R[]) { -----
- start_claiming(); -----
- ll *Q = claim(), *GQ = claim(); -----
- int qn = quotient(F, fn, G, gn, Q); -----
- int gqn = mult(G, gn, Q, qn, GQ); -----
- int rn = subtract(F, fn, GQ, gqn, R); -----
- end_claiming(); -----
- return rn; } -----
- ll horners(ll F[], int fn, ll xi) { -----
- ll ans = 0; -----
- for(int i = fn-1; i >= 0; i--) -----
  ans = (ans*xi+F[i]) % MOD; -----
- return ans; } }; -----
GF_Manager gfManager; -----
ll split[DEPTH+1][2*(ARR_DEPTH)+1]; -----
ll Fi[DEPTH+1][2*(ARR_DEPTH)+1]; -----
int bin_splitting(ll a[], int l, int r, int s=0, int offset=0) { -----
- if(l == r) { -----
  split[s][offset] = -a[l]; //x^0 -----
  split[s][offset+1] = 1; //x^1 -----
  return 2; } -----
- int m = (l+r)/2; -----
- int sz = m-l+1; -----
- int da = bin_splitting(a, l, m, s+1, offset); -----
- int db = bin_splitting(a, m+1, r, s+1, offset+(sz<<1)); -----
- return gfManager.mult(split[s+1]+offset, da, -----
  split[s+1]+offset+(sz<<1), db, split[s]+offset); } -----
void multipoint_eval(ll a[], int l, int r, ll F[], int fn, -----
ll ans[], int s=0, int offset=0) { -----
- if(l == r) { -----
  ans[l] = gfManager.horners(F,fn,a[l]); -----
  return; } -----
- int m = (l+r)/2; -----
- int sz = m-l+1; -----
- int da = gfManager.mod(F, fn, split[s+1]+offset, -----
  sz+1, Fi[s]+offset); -----
- int db = gfManager.mod(F, fn, split[s+1]+offset+(sz<<1), -----
  r-m+1, Fi[s]+offset+(sz<<1)); -----
- multipoint_eval(a,l,m,Fi[s]+offset,da,ans,s+1,offset); -----
- multipoint_eval(a,m+1,r,Fi[s]+offset+(sz<<1), -----
  db,ans,s+1,offset+(sz<<1)); -----
- } -----
} -----

5.2. Fast Fourier Transform. Compute the Discrete Fourier Trans-
form (DFT) of a polynomial in  $O(n \log n)$  time.

struct poly { -----
- double a, b; -----
- poly(double a=0, double b=0): a(a), b(b) {} -----
- poly operator+(const poly& p) const { -----
  return poly(a + p.a, b + p.b);} -----
- poly operator-(const poly& p) const { -----
  return poly(a - p.a, b - p.b);} -----
- poly operator*(const poly& p) const { -----
  return poly(a*p.a - b*p.b, a*p.b + b*p.a);} -----
}; -----
void fft(poly in[], poly p[], int n, int s) { -----
- if (n < 1) return; -----
- if (n == 1) {p[0] = in[0]; return;} -----
- n >>= 1; fft(in, p, n, s << 1); -----
- fft(in + s, p + n, n, s << 1); -----
- poly w(1), wn(cos(M_PI/n), sin(M_PI/n)); -----
- for (int i = 0; i < n; ++i) { -----
  poly even = p[i], odd = p[i + n]; -----
  p[i] = even + w * odd; -----
  p[i + n] = even - w * odd; -----
  w = w * wn; -----
} -----
} -----
void fft(poly p[], int n) { -----
- poly *f = new poly[n]; fft(p, f, n, 1); -----
- copy(f, f + n, p); delete[] f; -----
} -----
void inverse_fft(poly p[], int n) { -----
- for(int i=0; i<n; i++) {p[i].b *= -1; fft(p, n); -----

```

```
--- for(int i=0; i<n; i++) {p[i].a/=n; p[i].b/= -1*n;} -----
} -----
```

5.3. **FFT Polynomial Multiplication.** Multiply integer polynomials a, b of size an, bn using FFT in $O(n \log n)$. Stores answer in an array c , rounded to the nearest integer (or double).

```
// note: c[] should have size of at least (an+bn)
int mult(int a[],int an,int b[],int bn,int c[]) {
  int n, degree = an + bn - 1;
  for (n = 1; n < degree; n <= 1); // power of 2
  poly *A = new poly[n], *B = new poly[n];
  copy(a, a + an, A); fill(A + an, A + n, 0);
  copy(b, b + bn, B); fill(B + bn, B + n, 0);
  fft(A, n); fft(B, n);
  for (int i = 0; i < n; i++) A[i] = A[i] * B[i];
  inverse_fft(A, n);
  for (int i = 0; i < degree; i++)
    c[i] = int(A[i].a + 0.5); // same as round(A[i].a)
  delete[] A, B; return degree;
}
```

5.4. **Number Theoretic Transform.** Other possible moduli: $2^{11}3929217(2^{25})$, $2013265920268435457(2^{28}, withg = 5)$

```
#include "../mathematics/primitive_root.cpp"
int mod = 998244353, g = primitive_root(mod),
  ginv = mod_pow<ll>(g, mod-2, mod),
  inv2 = mod_pow<ll>(2, mod-2, mod);
#define MAXN (1<<22)
struct Num {
  int x;
  Num(ll _x=0) { x = (_x%mod+mod)%mod; }
  Num operator +(const Num &b) { return x + b.x; }
  Num operator -(const Num &b) const { return x - b.x; }
  Num operator *(const Num &b) const { return (ll)x * b.x; }
  Num operator /(const Num &b) const {
    return (ll)x * b.inv().x; }
  Num inv() const { return mod_pow<ll>((ll)x, mod-2, mod); }
  Num pow(int p) const { return mod_pow<ll>((ll)x, p, mod); }
} T1[MAXN], T2[MAXN];
void ntt(Num x[], int n, bool inv = false) {
  Num z = inv ? ginv : g;
  z = z.pow((mod - 1) / n);
  for (ll i = 0, j = 0; i < n; i++) {
    if (i < j) swap(x[i], x[j]);
    ll k = n>>1;
    while (1 <= k && k <= j) j -= k, k >=> 1;
    j += k; }
  for (int mx = 1, p = n/2; mx < n; mx <= 1, p >=> 1) {
    Num wp = z.pow(p), w = 1;
    for (int k = 0; k < mx; k++, w = w*wp) {
      for (int i = k; i < n; i += mx << 1) {
        Num t = x[i + mx] * w;
        x[i + mx] = x[i] - t;
        x[i] = x[i] + t; } } }
  if (inv) {
    Num ni = Num(n).inv();
    rep(i,0,n) { x[i] = x[i] * ni; } }
```

```
void inv(Num x[], Num y[], int l) {
  if (l == 1) { y[0] = x[0].inv(); return; }
  inv(x, y, l>>1);
  // NOTE: maybe l<<2 instead of l<<1
  rep(i,l>>1,l<<1) T1[i] = y[i] = 0;
  rep(i,0,l) T1[i] = x[i];
  ntt(T1, l<<1); ntt(y, l<<1);
  rep(i,0,l<<1) y[i] = y[i]*2 - T1[i] * y[i] * y[i];
  ntt(y, l<<1, true); }
void sqrt(Num x[], Num y[], int l) {
  if (l == 1) { assert(x[0].x == 1); y[0] = 1; return; }
  sqrt(x, y, l>>1);
  inv(y, T2, l>>1);
  rep(i,l>>1,l<<1) T1[i] = T2[i] = 0;
  rep(i,0,l) T1[i] = x[i];
  ntt(T2, l<<1); ntt(T1, l<<1);
  rep(i,0,l<<1) T2[i] = T1[i] * T2[i];
  ntt(T2, l<<1, true);
  rep(i,0,l) y[i] = (y[i] + T2[i]) * inv2; }
// vim: cc=60 ts=2 sts=2 sw=2:
```

5.5. **Polynomial Long Division.** Divide two polynomials A and B to get Q and R , where $\frac{A}{B} = Q + \frac{R}{B}$.

```
typedef vector<double> Poly;
Poly Q, R; // quotient and remainder
void trim(Poly& A) { // remove trailing zeroes
  while (!A.empty() && abs(A.back()) < EPS)
    A.pop_back(); }
void divide(Poly A, Poly B) {
  if (B.size() == 0) throw exception();
  if (A.size() < B.size()) {Q.clear(); R=A; return;}
  Q.assign(A.size() - B.size() + 1, 0);
  Poly part;
  while (A.size() >= B.size()) {
    int As = A.size(), Bs = B.size();
    part.assign(As, 0);
    for (int i = 0; i < Bs; i++)
      part[As-Bs+i] = B[i];
    double scale = Q[As-Bs] = A[As-1] / part[As-1];
    for (int i = 0; i < As; i++)
      A[i] -= part[i] * scale;
    trim(A);
  } R = A; trim(Q); }
```

5.6. **Matrix Multiplication.** Multiplies matrices $A_{p \times q}$ and $B_{q \times r}$ in $O(n^3)$ time, modulo MOD.

```
long[][] multiply(long A[][], long B[][]) {
  int p = A.length, q = A[0].length, r = B[0].length;
  // if(q != B.length) throw new Exception("((");
  long AB[][] = new long[p][r];
  for (int i = 0; i < p; i++)
    for (int j = 0; j < q; j++)
      for (int k = 0; k < r; k++)
        (AB[i][k] += A[i][j] * B[j][k]) %= MOD;
  return AB; }
```

5.7. **Matrix Power.** Computes for B^e in $O(n^3 \log e)$ time. Refer to Matrix Multiplication.

```
long[][] power(long B[][], long e) {
  int n = B.length;
  long ans[][] = new long[n][n];
  for (int i = 0; i < n; i++) ans[i][i] = 1;
  while (e > 0) {
    if (e % 2 == 1) ans = multiply(ans, B);
    B = multiply(B, B); e /= 2;
  } return ans; }
```

5.8. **Fibonacci Matrix.** Fast computation for n th Fibonacci $\{F_1, F_2, \dots, F_n\}$ in $O(\log n)$:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \times \begin{bmatrix} F_2 \\ F_1 \end{bmatrix}$$

5.9. **Gauss-Jordan/Matrix Determinant.** Row reduce matrix A in $O(n^3)$ time. Returns true if a solution exists.

```
boolean gaussJordan(double A[][]) {
  int n = A.length, m = A[0].length;
  boolean singular = false;
  // double determinant = 1;
  for (int i=0, p=0; i<n && p<m; i++, p++) {
    for (int k = i + 1; k < n; k++) {
      if (Math.abs(A[k][p]) > EPS) { // swap
        // determinant *= -1;
        double t[] = A[i]; A[i] = A[k]; A[k] = t;
        break;
      }
    }
    // determinant *= A[i][p];
    if (Math.abs(A[i][p]) < EPS)
      { singular = true; i--; continue; }
    for (int j = m-1; j >= p; j--) A[i][j] /= A[i][p];
    for (int k = 0; k < n; k++) {
      if (i == k) continue;
      for (int j = m-1; j >= p; j--)
        A[k][j] -= A[k][p] * A[i][j];
    }
  } return !singular; }
```

6. MATH II - COMBINATORICS

6.1. **Lucas Theorem.** Compute $\binom{n}{k} \bmod p$ in $O(p + \log_p n)$ time, where p is a prime.

```
LL f[P], lid; // P: biggest prime
LL lucas(LL n, LL k, int p) {
  if (k == 0) return 1;
  if (n < p && k < p) {
    if (lid != p) {
      lid = p; f[0] = 1;
      for (int i = 0; i < p; ++i) f[i] = f[i-1]*i%p;
    }
    return f[n] * modpow(f[n-k]*f[k]%p, p-2, p) % p;
  } return lucas(n/p,k/p,p) * lucas(n%p,k%p,p) % p; }
```

6.2. **Granville's Theorem.** Compute $\binom{n}{k} \bmod m$ (for any m) in $O(m^2 \log^2 n)$ time.

```
def fprime(n, p): -----
--- # counts the number of prime divisors of n! -----
--- pk, ans = p, 0 -----
--- while pk <= n:
---     ans += n // pk
---     pk *= p
--- return ans
-----

def granville(n, k, p, E):
--- # n choose k (mod p^E) -----
--- prime_pow = fprime(n, p) - fprime(k, p) - fprime(n - k, p)
--- if prime_pow >= E:
---     return 0
--- e = E - prime_pow
--- pe = p**e
--- r, f = n - k, [1] * pe
--- for i in range(1, pe):
---     x = i
---     if x % p == 0:
---         x = 1
---         f[i] = f[i - 1] * x % pe
---     numer, denom, negate, ptr = 1, 1, 0, 0
---     while n:
---         if f[-1] != 1 and ptr >= e:
---             negate ^= (n & 1) ^ (k & 1) ^ (r & 1)
---             numer = numer * f[n % pe] % pe
---             denom = denom * f[k % pe] % pe * f[r % pe] % pe
---             n, k, r = n // p, k // p, r // p
---             ptr += 1
---         ans = numer * modinv(denom, pe) % pe
---         if negate and (p != 2 or e < 3):
---             ans = (pe - ans) % pe
---     return mod(ans * p**prime_pow, p**E)
-----
```

```
def choose(n, k, m): # generalized (n choose k) mod m -----
--- factors, x, p = [], m, 2 -----
--- while p * p <= x:
---     e = 0
---     while x % p == 0:
---         e += 1
---         x //= p
---     if e:
---         factors.append((p, e))
---     p += 1
--- if x > 1:
---     factors.append((x, 1))
--- crt_array = [granville(n, k, p, e) for p, e in factors]
--- mod_array = [p**e for p, e in factors]
--- return chinese_remainder(crt_array, mod_array)[0] -----
```

6.3. **Derangements.** Compute the number of permutations with n elements such that no element is at their original position:

$$D(n) = (n - 1)(D(n - 1) + D(n - 2)) = nD(n - 1) + (-1)^n$$

6.4. **Factoradics.** Convert a permutation of n items to factoradics and vice versa in $O(n \log n)$.

```
// use fenwick tree add, sum, and low code -----
typedef long long LL;
void factoradic(int arr[], int n) { // 0 to n-1 -----
--- for (int i = 0; i <= n; i++) fen[i] = 0; -----
--- for (int i = 1; i < n; i++) add(i, 1); -----
--- for (int i = 0; i < n; i++) { -----
---     int s = sum(arr[i]); -----
---     add(arr[i], -1); arr[i] = s; -----
--- } -----
void permute(int arr[], int n) { // factoradic to perm -----
--- for (int i = 0; i <= n; i++) fen[i] = 0; -----
--- for (int i = 1; i < n; i++) add(i, 1); -----
--- for (int i = 0; i < n; i++) { -----
---     arr[i] = low(arr[i] - 1); -----
---     add(arr[i], -1); -----
--- } -----
```

6.5. **kth Permutation.** Get the next k th permutation of n items, if exists, using factoradics. All values should be from 0 to $n - 1$. Use factoradics methods as discussed above.

```
std::vector<int> nth_permutation(int cnt, int n) { -----
- std::vector<int> idx(cnt), per(cnt), fac(cnt); -----
- for (int i = 0; i < cnt; ++i) idx[i] = i; -----
- for (int i = 1; i < cnt+1; ++i) fac[i - 1] = n % i, n /= i;
- for (int i = cnt - 1; i >= 0; --i) -----
-     per[cnt - i - 1] = idx[fac[i]], -----
-     idx.erase(idx.begin() + fac[i]);
- return per; } -----
```

6.6. **Catalan Numbers.**

$$C_n = \frac{1}{n + 1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n + 1}$$

- (1) The number of non-crossing partitions of an n -element set
- (2) The number of expressions with n pairs of parentheses
- (3) The number of ways $n + 1$ factors can be parenthesized
- (4) The number of full binary trees with $n + 1$ leaves
- (5) The number of monotonic lattice paths of an $n \times n$ grid (5-SAT problem)
- (6) The number of triangulations of a convex polygon with $n + 2$ sides (non-rotational)
- (7) The number of permutations $\{1, \dots, n\}$ without a 3-term increasing subsequence
- (8) The number of ways to form a mountain range with n ups and n downs

6.7. **Stirling Numbers.** s_1 : Count the number of permutations of n elements with k disjoint cycles

s_2 : Count the ways to partition a set of n elements into k nonempty subsets

$$s_1(n, k) = \begin{cases} 1 & n = k = 0 \\ s_1(n - 1, k - 1) - (n - 1)s_1(n - 1, k) & n, k > 0 \\ 0 & \text{elsewhere} \end{cases}$$

$$s_2(n, k) = \begin{cases} 1 & n = k = 0 \\ s_2(n - 1, k - 1) + ks_2(n - 1, k) & n, k > 0 \\ 0 & \text{elsewhere} \end{cases}$$

6.8. **Partition Function.** Pregenerate the number of partitions of positive integer n with n positive addends.

$$p(n, k) = \begin{cases} 1 & n = k = 0 \\ 0 & n < k \\ p(n - 1, k - 1) + p(n - k, k) & n \geq k \end{cases}$$

7. MATH III - NUMBER THEORY

7.1. **Linear Prime Sieve.**

```
std::bitset<N> isc; // #include <bitset> -----
std::vector<int> p; -----
void sieve() { -----
- for (int i = 2; i < N; i++) { -----
-     if (!isc[i]) p.push_back(i); -----
-     for (int j = 0; j < p.size() && i*p[j] < N; j++) { -----
-         isc[i*p[j]] = 1; -----
-         if (i%p[j] == 0) break; } } } -----
```

7.2. **Miller Rabin.**

```
bool sieve[LIM+1]; // remember to sieve up to some LIM -----
vector<ll> primes; -----
//deterministic up to 2^64 -----
ll good_bases[] = {2,325,9375,28178,450775,9780504,1795265022};
// remember mod_pow and mod_mult -----
bool witness(ll a, ll n) { -----
- ll t = 0, u = n-1; -----
- while (u % 2 == 0) { -----
-     u >>= 1; -----
-     t += 1; } -----
- ll xp = 1, xi = mod_pow(a, u, n); -----
- for (int i = 0; i < t; i++) { -----
-     xp = xi; -----
-     xi = mod_mult(xi, xi, n); -----
-     if (xi == 1 && !(xp == 1 || xp == n-1)) return true; } ---
- return xi != 1; } -----
bool miller_rabin(ll n) { -----
- if (n <= 1) return false; -----
- if (n == 2) return true; -----
- if (n%2 == 0) return false; -----
- if (n <= LIM) return sieve[n]; -----
- for (const ll& x : good_bases) { -----
-     ll a = x % n; -----
-     if (a == 0) return true; -----
-     if (witness(a, n)) return false; } -----
- return true; } -----
```

7.3. **Number/Sum of Divisors.** If a number n is prime factorized where $n = p_1^{e_1} \times p_2^{e_2} \times \cdots \times p_k^{e_k}$, where σ_0 is the number of divisors while σ_1 is the sum of divisors:

$$\sum_{d|n} d^k = \sigma_k(n) = \prod \frac{p_i^{k(e_i)+1} - 1}{p_i - 1}$$

Product: $\prod_{d|n} d = n^{\frac{\sigma_1(n)}{2}}$

7.4. **Möbius Sieve.** The Möbius function μ is the Möbius inverse of e such that $e(n) = \sum_{d|n} \mu(d)$.

```
std::bitset<N> is; int mu[N];
void mobiusSieve() {
- for (int i = 1; i < N; ++i) mu[i] = 1;
- for (int i = 2; i < N; ++i) if (!is[i]) {
--- for (int j = i; j < N; j += i) is[j] = 1, mu[j] *= -1;
--- for (ll j = 1LL*i*i; j < N; j += i*i) mu[j] = 0; }
```

7.5. **Möbius Inversion.** Given arithmetic functions f and g :

$$g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

7.6. **GCD Subset Counting.** Count number of subsets $S \subseteq A$ such that $\gcd(S) = g$ (modifiable).

```
int f[MX+1]; // MX is maximum number of array
long long gcnt[MX+1]; // gcnt[G]: answer when gcd==G
long long C(int f) {return (1ll << f) - 1;}
// f: frequency count
// C(f): # of subsets of f elements (YOU CAN EDIT)
// Usage: int subsets_with_gcd_1 = gcnt[1];
void gcd_counter(int a[], int n) {
- memset(f, 0, sizeof f);
- memset(gcnt, 0, sizeof gcnt);
- int mx = 0;
- for (int i = 0; i < n; ++i) {
--- f[a[i]] += 1;
---- mx = max(mx, a[i]); }
- for (int i = mx; i >= 1; --i) {
--- int add = f[i];
--- long long sub = 0;
--- for (int j = 2*i; j <= mx; j += i) {
----- add += f[j];
----- sub += gcnt[j]; }
--- gcnt[i] = C(add) - sub; }
```

7.7. **Euler Totient.** Counts all integers from 1 to n that are relatively prime to n in $O(\sqrt{n})$ time.

```
ll totient(ll n) {
- if (n <= 1) return 1;
- ll tot = n;
- for (int i = 2; i * i <= n; i++) {
--- if (n % i == 0) tot -= tot / i;
--- while (n % i == 0) n /= i; }
- if (n > 1) tot -= tot / n;
- return tot; }
```

7.8. **Extended Euclidean.** Assigns x, y such that $ax + by = \gcd(a, b)$ and returns $\gcd(a, b)$.

```
ll mod(ll x, ll m) { // use this instead of x % m
- if (m == 0) return 0;
- if (m < 0) m *= -1;
- return (x%m + m) % m; // always nonnegative
}
ll extended_euclid(ll a, ll b, ll &x, ll &y) {
- if (b==0) {x = 1; y = 0; return a;}
- ll g = extended_euclid(b, a%b, x, y);
- ll z = x - a/b*y;
- x = y; y = z; return g;
}
```

7.9. **Modular Exponentiation.** Find $b^e \pmod m$ in $O(\log e)$ time.

```
template <class T>
T mod_pow(T b, T e, T m) {
- T res = T(1);
- while (e) {
--- if (e & T(1)) res = smod(res * b, m);
--- b = smod(b * b, m), e >>= T(1); }
- return res; }
```

7.10. **Modular Inverse.** Find unique x such that $ax \equiv 1 \pmod m$. Returns 0 if no unique solution is found. Please use modulo solver for the non-unique case.

```
ll modinv(ll a, ll m) {
- ll x, y; ll g = extended_euclid(a, m, x, y);
- if (g == 1 || g == -1) return mod(x * g, m);
- return 0; // 0 if invalid }
```

7.11. **Modulo Solver.** Solve for values of x for $ax \equiv b \pmod m$. Returns $(-1, -1)$ if there is no solution. Returns a pair (x, M) where solution is $x \pmod M$.

```
p ll modsolver(ll a, ll b, ll m) {
- ll x, y; ll g = extended_euclid(a, m, x, y);
- if (b % g != 0) return {-1, -1};
- return {mod(x*b/g, m/g), abs(m/g)}; }
```

7.12. **Linear Diophantine.** Computes integers x and y such that $ax + by = c$, returns $(-1, -1)$ if no solution. Tries to return positive integer answers for x and y if possible.

```
p ll null(-1, -1); // needs extended euclidean
p ll diophantine(ll a, ll b, ll c) {
- if (!a && !b) return c ? null : {0, 0};
- if (!a) return c % b ? null : {0, c / b};
- if (!b) return c % a ? null : {c / a, 0};
- ll x, y; ll g = extended_euclid(a, b, x, y);
- if (c % g) return null;
- y = mod(y * (c/g), a/g);
- if (y == 0) y += abs(a/g); // prefer positive sol.
- return {(c - b*y)/a, y}; }
```

7.13. **Chinese Remainder Theorem.** Solves linear congruence $x \equiv b_i \pmod{m_i}$. Returns $(-1, -1)$ if there is no solution. Returns a pair (x, M) where solution is $x \pmod M$.

```
p ll chinese(ll b1, ll m1, ll b2, ll m2) {
- ll x, y; ll g = extended_euclid(m1, m2, x, y);
- if (b1 % g != b2 % g) return ii(-1, -1);
- ll M = abs(m1 / g * m2);
- return {mod(mod(x*b2*m1+y*b1*m2, M*g)/g,M), M}; }
ii chinese_remainder(ll b[], ll m[], int n) {
- ii ans(0, 1);
- for (int i = 0; i < n; ++i) {
--- ans = chinese(b[i],m[i],ans.first,ans.second);
--- if (ans.second == -1) break; }
- return ans; }
```

7.13.1. *Super Chinese Remainder.* Solves linear congruence $a_i x \equiv b_i \pmod{m_i}$. Returns $(-1, -1)$ if there is no solution.

```
p ll super_chinese(ll a[], ll b[], ll m[], int n) {
- p ll ans(0, 1);
- for (int i = 0; i < n; ++i) {
--- p ll two = modsolver(a[i], b[i], m[i]);
--- if (two.second == -1) return two;
--- ans = chinese(ans.first, ans.second,
--- two.first, two.second);
--- if (ans.second == -1) break; }
- return ans; }
```

7.14. **Primitive Root.**

```
#include "mod_pow.cpp"
ll primitive_root(ll m) {
- std::vector<ll> div;
- for (ll i = 1; i*i <= m-1; i++) {
--- if ((m-1) % i == 0) {
----- if (i < m) div.push_back(i);
----- if (m/i < m) div.push_back(m/i); } }
- for (int x = 2; x < m; ++x) {
--- bool ok = true;
--- for (int d : div) if (mod_pow<ll>(x, d, m) == 1) {
----- ok = false; break; }
--- if (ok) return x; }
- return -1; }
```

7.15. **Josephus.** Last man standing out of n if every k th is killed. Zero-based, and does not kill 0 on first pass.

```
int J(int n, int k) {
- if (n == 1) return 0;
- if (k == 1) return n-1;
- if (n < k) return (J(n-1,k)+k)%n;
- int np = n - n/k;
- return k*((J(np,k)+np-n*k*np)%np) / (k-1); }
```

7.16. **Number of Integer Points under a Lines.** Count the number of integer solutions to $Ax + By \leq C$, $0 \leq x \leq n$, $0 \leq y$. In other words, evaluate the sum $\sum_{x=0}^n \left\lfloor \frac{C-Ax}{B} + 1 \right\rfloor$. To count all solutions, let $n = \left\lfloor \frac{c}{a} \right\rfloor$. In any case, it must hold that $C - nA \geq 0$. Be very careful about overflows.

8. MATH IV - NUMERICAL METHODS

8.1. **Fast Square Testing.** An optimized test for square integers.

```
long long M;
void init_is_square() {
- for (int i = 0; i < 64; ++i) M |= 1ULL << (63-(i*i)%64); }
inline bool is_square(ll x) {
- if (x == 0) return true; // XXX
- if ((M << x) >= 0) return false;
- int c = std::__builtin_ctz(x);
- if (c & 1) return false;
- x >>= c;
- if ((x&7) - 1) return false;
- ll r = std::sqrt(x);
- return r*r == x; }
```

8.2. **Simpson Integration.** Use to numerically calculate integrals

```
const int N = 1000 * 1000; // number of steps
double simpson_integration(double a, double b){
- double h = (b - a) / N;
- double s = f(a) + f(b); // a = x_0 and b = x_2n
- for (int i = 1; i <= N - 1; ++i) {
-- double x = a + h * i;
-- s += f(x) * ((i & 1) ? 4 : 2); }
- s *= h / 3;
- return s; }
```

9. STRINGS

9.1. **Knuth-Morris-Pratt** . Count and find all matches of string f in string s in $O(n)$ time.

```
int par[N]; // parent table
void buildKMP(string& f) {
- par[0] = -1, par[1] = 0;
- int i = 2, j = 0;
- while (i <= f.length()) {
-- if (f[i-1] == f[j]) par[i++] = ++j;
-- else if (j > 0) j = par[j];
-- else par[i++] = 0; } }
std::vector<int> KMP(string& s, string& f) {
- buildKMP(f); // call once if f is the same
- int i = 0, j = 0; vector<int> ans;
- while (i + j < s.length()) {
-- if (s[i + j] == f[j]) {
--- if (++j == f.length()) {
---- ans.push_back(i);
---- i += j - par[j];
---- if (j > 0) j = par[j]; }
-- } else {
-- i += j - par[j];
-- if (j > 0) j = par[j]; }
- } return ans; }
```

9.2. **Trie.**

```
template <class T>
struct trie {
- struct node {
-- map<T, node*> children;
```

```
-- int prefixes, words;
-- node() { prefixes = words = 0; } };
node* root;
trie() : root(new node()) { }
template <class I>
void insert(I begin, I end) {
- node* cur = root;
- while (true) {
-- cur->prefixes++;
-- if (begin == end) { cur->words++; break; }
-- else {
--- T head = *begin;
--- typename map<T, node*>::const_iterator it;
--- it = cur->children.find(head);
--- if (it == cur->children.end()) {
---- pair<T, node*> nw(head, new node());
---- it = cur->children.insert(nw).first;
--- } begin++, cur = it->second; } } }
template<class I>
int countMatches(I begin, I end) {
- node* cur = root;
- while (true) {
-- if (begin == end) return cur->words;
-- else {
--- T head = *begin;
--- typename map<T, node*>::const_iterator it;
--- it = cur->children.find(head);
--- if (it == cur->children.end()) return 0;
--- begin++, cur = it->second; } } }
template<class I>
int countPrefixes(I begin, I end) {
- node* cur = root;
- while (true) {
-- if (begin == end) return cur->prefixes;
-- else {
--- T head = *begin;
--- typename map<T, node*>::const_iterator it;
--- it = cur->children.find(head);
--- if (it == cur->children.end()) return 0;
--- begin++, cur = it->second; } } };
```

9.2.1. *Persistent Trie.*

```
const int MAX_KIDS = 2;
const char BASE = '0'; // 'a' or 'A'
struct trie {
- int val, cnt;
- std::vector<trie*> kids;
- trie () : val(-1), cnt(0), kids(MAX_KIDS, NULL) {}
- trie (int val) : val(val), cnt(0), kids(MAX_KIDS, NULL) {}
- trie (int val, int cnt, std::vector<trie*> &n_kids) :
-- val(val), cnt(cnt), kids(n_kids) {}
- trie *insert(std::string &s, int i, int n) {
-- trie *n_node = new trie(val, cnt+1, kids);
-- if (i == n) return n_node;
-- if (!n_node->kids[s[i]-BASE])
--- n_node->kids[s[i]-BASE] = new trie(s[i]);
```

```
-- n_node->kids[s[i]-BASE] =
--- n_node->kids[s[i]-BASE]->insert(s, i+1, n);
-- return n_node; } };
// max xor on a binary trie from version 'a+1' to 'b' (b > a):
int get_max_xor(trie *a, trie *b, int x) {
- int ans = 0;
- for (int i = MAX_BITS; i >= 0; --i) {
-- // don't flip the bit for min xor
-- int u = ((x & (1 << i)) > 0) ^ 1;
-- int res_cnt = (b and b->kids[u] ? b->kids[u]->cnt : 0) -
-- (a and a->kids[u] ? a->kids[u]->cnt : 0);
-- if (res_cnt == 0) u ^= 1;
-- ans ^= (u << i);
-- if (a) a = a->kids[u];
-- if (b) b = b->kids[u]; }
- return ans; }
```

9.3. **Suffix Array** . Construct a sorted catalog of all substrings of s in $O(n \log n)$ time using counting sort.

```
int n, equiv[N+1], suffix[N+1];
ii equiv_pair[N+1];
string T;
void make_suffix_array(string& s) {
- if (s.back()!='$') s += '$';
- n = s.length();
- for (int i = 0; i < n; i++)
-- suffix[i] = i;
- sort(suffix, suffix+n, [&s](int i, int j){return s[i] < s[j];});
- int sz = 0;
- for(int i = 0; i < n; i++){
-- if(i==0 || s[suffix[i]]!=s[suffix[i-1]])
--- ++sz;
-- equiv[suffix[i]] = sz; }
- for (int t = 1; t < n; t<=1) {
-- for (int i = 0; i < n; i++)
--- equiv_pair[i] = {equiv[i],equiv[(i+t)%n]};
-- sort(suffix, suffix+n, [](int i, int j) {
--- return equiv_pair[i] < equiv_pair[j];});
- int sz = 0;
- for (int i = 0; i < n; i++) {
-- if(i==0 || equiv_pair[suffix[i]]!=equiv_pair[suffix[i-1]])
--- ++sz;
-- equiv[suffix[i]] = sz; } } }
int count_occurences(string& G) { // in string T
- int L = 0, R = n-1;
- for (int i = 0; i < G.length(); i++){
-- // lower/upper = first/last time G[i] is
-- // the ith character in suffixes from [L,R]
-- std::tie(L,R) = {lower(G[i],i,L,R), upper(G[i],i,L,R)};
-- if (L==-1 && R==-1) return 0; }
- return R-L+1; }
```

9.4. **Longest Common Prefix** . Find the length of the longest common prefix for every substring in $O(n)$.

```
int lcp[N]; // lcp[i] = LCP(s[sa[i]:], s[sa[i+1]:])
void buildLCP(std::string s) { // build suffix array first
```



```
- for (int i = 0, k = 0; i < n; i++) { -----
- - if (pos[i] != n - 1) { -----
- - - for(int j = sa[pos[i]+1]; s[i+k]==s[j+k];k++); -----
- - - lcp[pos[i]] = k; if (k > 0) k--; -----
- } else { lcp[pos[i]] = 0; } } }
```

9.5. **Aho-Corasick Trie**. Find all multiple pattern matches in $O(n)$ time. This is KMP for multiple strings.

```
class Node { -----
- HashMap<Character, Node> next = new HashMap<>(); -----
- Node fail = null; -----
- long count = 0; -----
- public void add(String s) { // adds string to trie -----
- - Node node = this; -----
- - for (char c : s.toCharArray()) { -----
- - - if (!node.contains(c)) -----
- - - - node.next.put(c, new Node()); -----
- - - node = node.get(c); -----
- - } node.count++; } -----
- public void prepare() { -----
- - // prepares fail links of Aho-Corasick Trie -----
- - Node root = this; root.fail = null; -----
- - Queue<Node> q = new ArrayDeque<Node>(); -----
- - for (Node child : next.values()) // BFS -----
- - - { child.fail = root; q.offer(child); } -----
- - while (!q.isEmpty()) { -----
- - - Node head = q.poll(); -----
- - - for (Character letter : head.next.keySet()) { -----
- - - - // traverse upwards to get nearest fail link -----
- - - - Node p = head; -----
- - - - Node nextNode = head.get(letter); -----
- - - - do { p = p.fail; } -----
- - - - while(p != root && !p.contains(letter)); -----
- - - - if (p.contains(letter)) { // fail link found -----
- - - - - p = p.get(letter); -----
- - - - - nextNode.fail = p; -----
- - - - - nextNode.count += p.count; -----
- - - - } else { nextNode.fail = root; } -----
- - - q.offer(nextNode); } } } -----
- public BigInteger search(String s) { -----
- - // counts the words added in trie present in s -----
- - Node root = this, p = this; -----
- - BigInteger ans = BigInteger.ZERO; -----
- - for (char c : s.toCharArray()) { -----
- - - while (p != root && !p.contains(c)) p = p.fail; -----
- - - if (p.contains(c)) { -----
- - - - p = p.get(c); -----
- - - - ans = ans.add(BigInteger.valueOf(p.count)); } -----
- - } return ans; } -----
- private Node get(char c) { return next.get(c); } -----
- private boolean contains(char c) { -----
- - return next.containsKey(c); } -----
// Usage: Node trie = new Node(); -----
// for (String s : dictionary) trie.add(s); -----
// trie.prepare(); BigInteger m = trie.search(str); -----
```

9.6. Palindromes.

9.6.1. **Palindromic Tree**. Find lengths and frequencies of all palindromic substrings of a string in $O(n)$ time.

Theorem: there can only be up to n unique palindromic substrings for any string.

```
int par[N*2+1], child[N*2+1][128]; -----
int len[N*2+1], node[N*2+1], cs[N*2+1], size; -----
long long cnt[N + 2]; // count can be very large -----
int newNode(int p = -1) { -----
- cnt[size] = 0; par[size] = p; -----
- len[size] = (p == -1 ? 0 : len[p] + 2); -----
- memset(child[size], -1, sizeof child[size]); -----
- return size++; } -----
int get(int i, char c) { -----
- if (child[i][c] == -1) child[i][c] = newNode(i); -----
- return child[i][c]; } -----
void manachers(char s[]) { -----
- int n = strlen(s), cn = n * 2 + 1; -----
- for (int i = 0; i < n; i++) { -----
- - cs[i * 2] = -1; cs[i * 2 + 1] = s[i]; } -----
- size = n * 2; -----
- int odd = newNode(), even = newNode(); -----
- int cen = 0, rad = 0, L = 0, R = 0; -----
- size = 0; len[odd] = -1; -----
- for (int i = 0; i < cn; i++) -----
- - node[i] = (i % 2 == 0 ? even : get(odd, cs[i])); -----
- for (int i = 1; i < cn; i++) { -----
- - if (i > rad) { L = i - 1; R = i + 1; } -----
- - else { -----
- - - int M = cen * 2 - i; // retrieve from mirror -----
- - - node[i] = node[M]; -----
- - - if (len[node[M]] < rad - i) L = -1; -----
- - - else { -----
- - - - R = rad + 1; L = i * 2 - R; -----
- - - - while (len[node[i]] > rad - i) -----
- - - - - node[i] = par[node[i]]; } } // expand palindrome -----
- - while (L >= 0 && R < cn && cs[L] == cs[R]) { -----
- - - if (cs[L] != -1) node[i] = get(node[i], cs[L]); -----
- - - L--, R++; } -----
- - cnt[node[i]]++; -----
- - if (i + len[node[i]] > rad) { -----
- - - rad = i + len[node[i]]; cen = i; } } -----
- for (int i = size - 1; i >= 0; --i) -----
- - cnt[par[i]] += cnt[i]; // update parent count } -----
int countUniquePalindromes(char s[]) { -----
- manachers(s); return size; } -----
int countAllPalindromes(char s[]) { -----
- manachers(s); int total = 0; -----
- for (int i = 0; i < size; i++) total += cnt[i]; -----
- return total; } -----
// longest palindrome substring of s -----
std::string longestPalindrome(char s[]) { -----
- manachers(s); -----
- int n = strlen(s), cn = n * 2 + 1, mx = 0; -----
- for (int i = 1; i < cn; i++) -----
- - if (len[node[mx]] < len[node[i]]) -----
- - - mx = i; -----
```

```
- int pos = (mx - len[node[mx]]) / 2; -----
- return std::string(s + pos, s + pos + len[node[mx]]); } -----
```

9.6.2. Eertree.

```
struct node { -----
- int start, end, len, back_edge, *adj; -----
- node() { -----
- - adj = new int[26]; -----
- - for (int i = 0; i < 26; ++i) adj[i] = 0; } -----
- node(int start, int end, int len, int back_edge) : -----
- - start(start), end(end), len(len), back_edge(back_edge) { -----
- - adj = new int[26]; -----
- - for (int i = 0; i < 26; ++i) adj[i] = 0; } }; -----
struct eertree { -----
- int ptr, cur_node; -----
- std::vector<node> tree; -----
- eertree () { -----
- - tree.push_back(node()); -----
- - tree.push_back(node(0, 0, -1, 1)); -----
- - tree.push_back(node(0, 0, 0, 1)); -----
- - cur_node = 1; -----
- - ptr = 2; } -----
- int get_link(int temp, std::string &s, int i) { -----
- - while (true) { -----
- - - int cur_len = tree[temp].len; -----
- - - // don't return immediately if you want to -----
- - - // get all palindromes; not recommended -----
- - - if (i - cur_len - 1 >= 0 and s[i] == s[i - cur_len - 1]) -----
- - - - return temp; -----
- - - temp = tree[temp].back_edge; } -----
- - return temp; } -----
- void insert(std::string &s, int i) { -----
- - int temp = cur_node; -----
- - temp = get_link(temp, s, i); -----
- - if (tree[temp].adj[s[i] - 'a'] != 0) { -----
- - - cur_node = tree[temp].adj[s[i] - 'a']; -----
- - - return; } -----
- - ptr++; -----
- - tree[temp].adj[s[i] - 'a'] = ptr; -----
- - int len = tree[temp].len + 2; -----
- - tree.push_back(node(i - len + 1, i, len, 0)); -----
- - temp = tree[temp].back_edge; -----
- - cur_node = ptr; -----
- - if (tree[cur_node].len == 1) { -----
- - - tree[cur_node].back_edge = 2; -----
- - - return; } -----
- - temp = get_link(temp, s, i); -----
- - tree[cur_node].back_edge = tree[temp].adj[s[i] - 'a']; } -----
- void insert(std::string &s) { -----
- - for (int i = 0; i < s.size(); ++i) -----
- - - insert(s, i); } };
```

9.7. **Z Algorithm**. Find the longest common prefix of all substrings of s with itself in $O(n)$ time.

```
int z[N]; // z[i] = lcp(s, s[i:]) -----
void compute_z(string s) { -----
- int n = s.length(); z[0] = 0; -----
```

```
- for (int i = 1, L = 0, R = 0; i < n; i++) { -----
- if (i <= R) z[i] = min(R-i+1, z[i-L]);
- while (i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++;
- if (i+z[i]-1 > R) L = i, R = i+z[i]-1; }
- z[0] = n; }
```

9.8. **Booth's Minimum String Rotation** . Booth's Algo: Find the index of the lexicographically least string rotation in $O(n)$ time.

```
int f[N * 2]; -----
int booth(string S) {
- S.append(S); // concatenate itself -----
- int n = S.length(), i, j, k = 0;
- memset(f, -1, sizeof(int) * n);
- for (j = 1; j < n; j++) { -----
- i = f[j-k-1];
- while (i != -1 && S[j] != S[k + i + 1]) { -----
- if (S[j] < S[k + i + 1]) k = j - i - 1;
- i = f[i];
- } if (i == -1 && S[j] != S[k + i + 1]) { -----
- if (S[j] < S[k + i + 1]) k = j;
- f[j - k] = -1;
- } else f[j - k] = i + 1;
- } return k; }
```

9.9. Hashing.

9.9.1. Rolling Hash.

```
int MAXN = 1e5+1, MOD = 1e9+7; -----
struct hasher { -----
- int n;
- std::vector<ll> *p_pow, *h_ans;
- hash(vi &s, vi primes) : n(primes.size()) { -----
- p_pow = new std::vector<ll>[n];
- h_ans = new std::vector<ll>[n];
- for (int i = 0; i < n; ++i) { -----
- p_pow[i] = std::vector<ll>(MAXN);
- p_pow[i][0] = 1;
- for (int j = 0; j+1 < MAXN; ++j) -----
- p_pow[i][j+1] = (p_pow[i][j] * primes[i]) % MOD;
- h_ans[i] = std::vector<ll>(MAXN);
- h_ans[i][0] = 0;
- for (int j = 0; j < s.size(); ++j) -----
- h_ans[i][j+1] = (h_ans[i][j] + -----
- s[j] * p_pow[i][j]) % MOD; } } };
```

10. OTHER ALGORITHMS

10.1. **2SAT**. Build the implication graph of the input by converting ORs $A \vee B$ to $!A \rightarrow B$ and $!B \rightarrow A$. This forms a bipartite graph. If there exists X such that both X and $!X$ are in the same strongly connected component, then there is no solution. Otherwise, iterate through the literals, arbitrarily assign a truth value to unassigned literals and propagate the values to its neighbors.

10.2. **DPLL Algorithm**. A SAT solver that can solve a random 1000-variable SAT instance within a second.

```
#define IDX(x) ((abs(x)-1)*2+((x)>0)) -----
struct SAT { -----
- int n;
- vi cl, head, tail, val;
- vii log; vvi w, loc;
- SAT() : n(0) { }
- int var() { return ++n; }
- void clause(vi vars) { -----
- set<int> seen; iter(it,vars) { -----
- if (seen.find(IDX(*it)^1) != seen.end()) return;
- seen.insert(IDX(*it)); }
- head.push_back(cl.size());
- iter(it,seen) cl.push_back(*it);
- tail.push_back((int)cl.size() - 2); }
- bool assume(int x) { -----
- if (val[x^1]) return false;
- if (val[x]) return true;
- val[x] = true; log.push_back(ii(-1, x));
- rep(i,0,w[x^1].size()) { -----
- int at = w[x^1][i], h = head[at], t = tail[at];
- log.push_back(ii(at, h));
- if (cl[t+1] != (x^1)) swap(cl[t], cl[t+1]);
- while (h < t && val[cl[h]^1]) h++;
- if ((head[at] = h) < t) { -----
- w[cl[h]].push_back(w[x^1][i]);
- swap(w[x^1][i--], w[x^1].back());
- w[x^1].pop_back();
- swap(cl[head[at]++], cl[t+1]);
- } else if (!assume(cl[t])) return false; }
- return true; }
- bool bt() { -----
- int v = log.size(), x; ll b = -1;
- rep(i,0,n) if (val[2*i] == val[2*i+1]) { -----
- ll s = 0, t = 0;
- rep(j,0,2) { iter(it,loc[2*i+j]) -----
- s+=1LL<<max(0,40-tail[*it]+head[*it]); swap(s,t); }
- if (max(s,t) >= b) b = max(s,t), x = 2*i + (t>=s); }
- if (b == -1 || (assume(x) && bt())) return true;
- while (log.size() != v) { -----
- int p = log.back().first, q = log.back().second;
- if (p == -1) val[q] = false; else head[p] = q;
- log.pop_back(); }
- return assume(x^1) && bt(); }
- bool solve() { -----
- val.assign(2*n+1, false);
- w.assign(2*n+1, vi()); loc.assign(2*n+1, vi());
- rep(i,0,head.size()) { -----
- if (head[i] == tail[i]+2) return false;
- rep(at,head[i],tail[i]+2) loc[cl[at]].push_back(i); }
- rep(i,0,head.size()) if (head[i] < tail[i]+1) rep(t,0,2) -----
- w[cl[tail[i]+t]].push_back(i);
- rep(i,0,head.size()) if (head[i] == tail[i]+1) -----
- if (!assume(cl[head[i]])) return false;
- return bt(); }
- bool get_value(int x) { return val[IDX(x)]; } };
```

10.3. **Stable Marriage**. The Gale-Shapley algorithm for solving the stable marriage problem.

```
vi stable_marriage(int n, vvi &m, vvi &w) { -----
- std::queue<int> q;
- vi at(n, 0), eng(n, -1), res(n, -1); vvi inv(n, vi(n));
- for (int i = 0; i < n; ++i) { -----
- for (int j = 0; j < n; ++j) -----
- inv[i][w[i][j]] = j;
- q.push(i); }
- while (!q.empty()) { -----
- int curm = q.front(); q.pop();
- for (int &i = at[curm]; i < n; i++) { -----
- int curw = m[curm][i];
- if (eng[curw] == -1) { }
- else if (inv[curw][curm] < inv[curw][eng[curw]]) -----
- q.push(eng[curw]);
- else continue;
- res[eng[curw] = curm] = curw, ++i; break; } }
- return res; }
```

10.4. **Cycle-Finding**. An implementation of Floyd's Cycle-Finding algorithm.

```
ii find_cycle(int x0, int (*f)(int)) { -----
- int t = f(x0), h = f(t), mu = 0, lam = 1;
- while (t != h) t = f(t), h = f(f(h));
- h = x0;
- while (t != h) t = f(t), h = f(h), mu++;
- h = f(t);
- while (t != h) h = f(h), lam++;
- return ii(mu, lam); }
```

10.5. Longest Increasing Subsequence.

```
vi lis(vi &arr) { -----
- if (arr.empty()) return vi();
- vi seq, back(arr.size()), ans;
- for (int i = 0; i < arr.size()) { -----
- int res = 0, lo = 1, hi = seq.size();
- while (lo <= hi) { -----
- int mid = (lo + hi) / 2;
- if (arr[seq[mid-1]] < arr[i]) res = mid, lo = mid + 1;
- else hi = mid - 1; }
- if (res < seq.size()) seq[res] = i;
- else seq.push_back(i);
- back[i] = res == 0 ? -1 : seq[res-1]; }
- int at = seq.back();
- while (at != -1) ans.push_back(at), at = back[at];
- std::reverse(ans.begin(), ans.end());
- return ans; }
```

```
10.6. Dates. Functions to simplify date calculations.
int intToDay(int jd) { return jd % 7; } -----
int dateToInt(int y, int m, int d) { -----
- return 1461 * (y + 4800 + (m - 14) / 12) / 4 + -----
- 367 * (m - 2 - (m - 14) / 12 * 12) / 12 - -----
- 3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 + -----
- d - 32075; }
void intToDate(int jd, int &y, int &m, int &d) { -----
```

```
- int x, n, i, j; ----- // maximize c^T x
- x = jd + 68569; ----- // subject to Ax <= b
- n = 4 * x / 146097; ----- // x >= 0
- x -= (146097 * n + 3) / 4; ----- // INPUT: A -- an m x n matrix
- i = (4000 * (x + 1)) / 1461001; ----- // b -- an m-dimensional vector
- x -= 1461 * i / 4 - 31; ----- // c -- an n-dimensional vector
- j = 80 * x / 2447; ----- // x -- a vector where the optimal solution will be
- d = x - 2447 * j / 80; ----- // stored
- x = j / 11; ----- // OUTPUT: value of the optimal solution (infinity if
- m = j + 2 - 12 * x; ----- // unbounded above, nan if infeasible)
- y = 100 * (n - 49) + i + x; } ----- // To use this code, create an LPSolver object with A, b,
// and c as arguments. Then, call Solve(x).
```

10.7. **Simulated Annealing.** An example use of Simulated Annealing to find a permutation of length n that maximizes $\sum_{i=1}^{n-1} |p_i - p_{i+1}|$.

```
double curtime() {
- return static_cast<double>(clock()) / CLOCKS_PER_SEC; }
int simulated_annealing(int n, double seconds) {
- default_random_engine rng;
- uniform_real_distribution<double> randfloat(0.0, 1.0);
- uniform_int_distribution<int> randint(0, n - 2);
- // random initial solution
- vi sol(n);
- for (int i = 0; i < n; ++i) sol[i] = i + 1;
- std::random_shuffle(sol.begin(), sol.end());
- // initialize score
- int score = 0;
- for (int i = 1; i < n; ++i)
- score += std::abs(sol[i] - sol[i-1]);
- int iters = 0;
- double T0 = 100.0, T1 = 0.001,
- progress = 0, temp = T0,
- starttime = curtime();
- while (true) {
- if (!(iters & ((1 << 4) - 1))) {
- progress = (curtime() - starttime) / seconds;
- temp = T0 * std::pow(T1 / T0, progress);
- if (progress > 1.0) break; }
- // random mutation
- int a = std::randint(rng);
- // compute delta for mutation
- int delta = 0;
- if (a > 0) delta += std::abs(sol[a+1] - sol[a-1])
- std::abs(sol[a] - sol[a-1]);
- if (a+2 < n) delta += std::abs(sol[a] - sol[a+2])
- std::abs(sol[a+1] - sol[a+2]);
- // maybe apply mutation
- if (delta >= 0 || randfloat(rng) < exp(delta / temp)) {
- std::swap(sol[a], sol[a+1]);
- score += delta;
- // if (score >= target) return;
- }
- iters++; }
- return score; }
```

10.8. **Simplex.**

// Two-phase simplex algorithm for solving linear programs
// of the form

```
typedef long double DOUBLE;
typedef vector<DOUBLE> VD;
typedef vector<VD> VVD;
typedef vector<int> vi;
const DOUBLE EPS = 1e-9;
struct LPSolver {
- int m, n;
- vi B, N;
- VVD D;
- LPSolver(const VVD &A, const VD &b, const VD &c) :
- m(b.size()), n(c.size()),
- N(n + 1), B(m), D(m + 2, VD(n + 2)) {
- for (int i = 0; i < m; i++) for (int j = 0; j < n; j++)
- D[i][j] = A[i][j];
- for (int i = 0; i < m; i++) { B[i] = n + i; D[i][n] = -1;
- D[i][n + 1] = b[i]; }
- for (int j = 0; j < n; j++) { N[j] = j; D[m][j] = -c[j]; }
- N[n] = -1; D[m + 1][n] = 1; }
- void Pivot(int r, int s) {
- double inv = 1.0 / D[r][s];
- for (int i = 0; i < m + 2; i++) if (i != r)
- for (int j = 0; j < n + 2; j++) if (j != s)
- D[i][j] -= D[r][j] * D[i][s] * inv;
- for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] *= inv;
- for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] *= -inv;
- D[r][s] = inv;
- swap(B[r], N[s]); }
- bool Simplex(int phase) {
- int x = phase == 1 ? m + 1 : m;
- while (true) {
- int s = -1;
- for (int j = 0; j <= n; j++) {
- if (phase == 2 && N[j] == -1) continue;
- if (s == -1 || D[x][j] < D[x][s] ||
- D[x][j] == D[x][s] && N[j] < N[s]) s = j; }
- if (D[x][s] > -EPS) return true;
- int r = -1;
- for (int i = 0; i < m; i++) {
- if (D[i][s] < EPS) continue;
- if (r == -1 || D[i][n + 1] / D[i][s] < D[r][n + 1] /
- D[r][s] || (D[i][n + 1] / D[i][s]) == (D[r][n + 1] /
- D[r][s]) && B[i] < B[r]) r = i; }
- if (r == -1) return false;
```

```
-- Pivot(r, s); } }
DOUBLE Solve(VD &x) {
- int r = 0;
- for (int i = 1; i < m; i++) if (D[i][n + 1] < D[r][n + 1])
- r = i;
- if (D[r][n + 1] < -EPS) {
- Pivot(r, n);
- if (!Simplex(1) || D[m + 1][n + 1] < -EPS)
- return -numeric_limits<DOUBLE>::infinity();
- for (int i = 0; i < m; i++) if (B[i] == -1) {
- int s = -1;
- for (int j = 0; j <= n; j++)
- if (s == -1 || D[i][j] < D[i][s] ||
- D[i][j] == D[i][s] && N[j] < N[s])
- s = j;
- Pivot(i, s); } }
- if (!Simplex(2)) return numeric_limits<DOUBLE>::infinity();
- x = VD(n);
- for (int i = 0; i < m; i++) if (B[i] < n)
- x[B[i]] = D[i][n + 1];
- return D[m][n + 1]; } }
```

10.9. **Fast Input Reading.** If input or output is huge, sometimes it is beneficial to optimize the input reading/output writing. This can be achieved by reading all input in at once (using fread), and then parsing it manually. Output can also be stored in an output buffer and then dumped once in the end (using fwrite). A simpler, but still effective, way to achieve speed is to use the following input reading method.

```
void readn(register int *n) {
- int sign = 1;
- register char c;
- *n = 0;
- while((c = getc_unlocked(stdin)) != '\n') {
- switch(c) {
- case '-': sign = -1; break;
- case ' ': goto hell;
- case '\n': goto hell;
- default: *n *= 10; *n += c - '0'; break; } }
hell:
- *n *= sign; }
```

10.10. **128-bit Integer.** GCC has a 128-bit integer data type named `__int128`. Useful if doing multiplication of 64-bit integers, or something needing a little more than 64-bits to represent. There's also `__float128`.

10.11. **Bit Hacks.**

```
int snoob(int x) {
- int y = x & -x, z = x + y;
- return z | ((x ^ z) >> 2) / y; }
```

11. Misc

11.1. **Debugging Tips.**

- Stack overflow? Recursive DFS on tree that is actually a long path?
- Floating-point numbers
 - Getting NaN? Make sure `acos` etc. are not getting values out of their range (perhaps `1+eps`).
 - Rounding negative numbers?

- Outputting in scientific notation?
- Wrong Answer?
 - Read the problem statement again!
 - Are multiple test cases being handled correctly? Try repeating the same test case many times.
 - Integer overflow?
 - Think very carefully about boundaries of all input parameters
 - Try out possible edge cases:
 - * $n = 0, n = -1, n = 1, n = 2^{31} - 1$ or $n = -2^{31}$
 - * List is empty, or contains a single element
 - * n is even, n is odd
 - * Graph is empty, or contains a single vertex
 - * Graph is a multigraph (loops or multiple edges)
 - * Polygon is concave or non-simple
 - Is initial condition wrong for small cases?
 - Are you sure the algorithm is correct?
 - Explain your solution to someone.
 - Are you using any functions that you don't completely understand? Maybe STL functions?
 - Maybe you (or someone else) should rewrite the solution?
 - Can the input line be empty?
- Run-Time Error?
 - Is it actually Memory Limit Exceeded?

11.2. Solution Ideas.

- Dynamic Programming
 - Parsing CFGs: CYK Algorithm
 - Drop a parameter, recover from others
 - Swap answer and a parameter
 - When grouping: try splitting in two
 - 2^k trick
 - When optimizing, see Section 2 (DP) if you can use anything there
 - * Convex hull optimization
 - * Divide and conquer optimization
 - * Knuth optimization
- Greedy
- Randomized
- Optimizations
 - Use bitset (/64)
 - Switch order of loops (cache locality)
- Process queries offline
 - Mo's algorithm
- Square-root decomposition
- Precomputation
- Efficient simulation
 - Mo's algorithm
 - Sqrt decomposition
 - Store 2^k jump pointers
- Data structure techniques
 - Sqrt buckets
 - Store 2^k jump pointers
 - 2^k merging trick
- Counting
 - Inclusion-exclusion principle
 - Generating functions

- Graphs
 - Can we model the problem as a graph?
 - Can we use any properties of the graph?
 - Strongly connected components
 - Cycles (or odd cycles)
 - Bipartite (no odd cycles)
 - * Bipartite matching
 - * Hall's marriage theorem
 - * Stable Marriage
 - Cut vertex/bridge
 - Biconnected components
 - Degrees of vertices (odd/even)
 - Trees
 - * Heavy-light decomposition
 - * Centroid decomposition
 - * Least common ancestor
 - * Centers of the tree
 - Eulerian path/circuit
 - Chinese postman problem
 - Topological sort
 - (Min-Cost) Max Flow
 - Min Cut
 - * Maximum Density Subgraph
 - Huffman Coding
 - Min-Cost Arborescence
 - Steiner Tree
 - Kirchoff's matrix tree theorem
 - Prüfer sequences
 - Lovász Toggle
 - Look at the DFS tree (which has no cross-edges)
 - Is the graph a DFA or NFA?
 - * Is it the Synchronizing word problem?
- Mathematics
 - Is the function multiplicative?
 - Look for a pattern
 - Permutations
 - * Consider the cycles of the permutation
 - Functions
 - * Sum of piecewise-linear functions is a piecewise-linear function
 - * Sum of convex (concave) functions is convex (concave)
 - Modular arithmetic
 - * Chinese Remainder Theorem
 - * Linear Congruence
 - Sieve
 - System of linear equations
 - Values too big to represent?
 - * Compute using the logarithm
 - * Divide everything by some large value
 - Linear programming
 - * Is the dual problem easier to solve?
 - Can the problem be modeled as a different combinatorial problem? Does that simplify calculations?
- Logic
 - 2-SAT
 - XOR-SAT (Gauss elimination or Bipartite matching)

- Meet in the middle
- Only work with the smaller half ($\log(n)$)
- Strings
 - Trie (maybe over something weird, like bits)
 - Suffix array
 - Suffix automaton (+DP?)
 - Aho-Corasick
 - eerTree
 - Work with $S + S$
- Hashing
- Euler tour, tree to array
- Segment trees
 - Lazy propagation
 - Persistent
 - Implicit
 - Segment tree of X
- Geometry
 - Minkowski sum (of convex sets)
 - Rotating calipers
 - Sweep line (horizontally or vertically?)
 - Sweep angle
 - Convex hull
- Fix a parameter (possibly the answer).
- Are there few distinct values?
- Binary search
- Sliding Window (+ Monotonic Queue)
- Computing a Convolution? Fast Fourier Transform
- Computing a 2D Convolution? FFT on each row, and then on each column
- Exact Cover (+ Algorithm X)
- Cycle-Finding
- What is the smallest set of values that identify the solution? The cycle structure of the permutation? The powers of primes in the factorization?
- Look at the complement problem
 - Minimize something instead of maximizing
- Immediately enforce necessary conditions. (All values greater than 0? Initialize them all to 1)
- Add large constant to negative numbers to make them positive
- Counting/Bucket sort

12. FORMULAS

- **Legendre symbol:** $\left(\frac{a}{b}\right) = a^{(b-1)/2} \pmod{b}$, b odd prime.
- **Heron's formula:** A triangle with side lengths a, b, c has area $\sqrt{s(s-a)(s-b)(s-c)}$ where $s = \frac{a+b+c}{2}$.
- **Pick's theorem:** A polygon on an integer grid strictly containing i lattice points and having b lattice points on the boundary has area $i + \frac{b}{2} - 1$. (Nothing similar in higher dimensions)
- **Euler's totient:** The number of integers less than n that are coprime to n are $n \prod_{p|n} \left(1 - \frac{1}{p}\right)$ where each p is a distinct prime factor of n .
- **König's theorem:** In any bipartite graph $G = (L \cup R, E)$, the number of edges in a maximum matching is equal to the number of vertices in a minimum vertex cover. Let U be the set of unmatched vertices in L , and Z be the set of vertices that are either in U or are connected to U

- by an alternating path. Then $K = (L \setminus Z) \cup (R \cap Z)$ is the minimum vertex cover.
- A minimum Steiner tree for n vertices requires at most $n-2$ additional Steiner vertices.
- The number of vertices of a graph is equal to its minimum vertex cover number plus the size of a maximum independent set.
- Lagrange polynomial** through points $(x_0, y_0), \dots, (x_k, y_k)$ is $L(x) = \sum_{j=0}^k y_j \prod_{\substack{0 \leq m \leq k \\ m \neq j}} \frac{x-x_m}{x_j-x_m}$
- Hook length formula:** If λ is a Young diagram and $h_\lambda(i, j)$ is the hook-length of cell (i, j) , then then the number of Young tableaux $d_\lambda = n! / \prod h_\lambda(i, j)$.
- Möbius inversion formula:** If $f(n) = \sum_{d|n} g(d)$, then $g(n) = \sum_{d|n} \mu(d) f(n/d)$. If $f(n) = \sum_{m=1}^n g(\lfloor n/m \rfloor)$, then $g(n) = \sum_{m=1}^n \mu(m) f(\lfloor \frac{n}{m} \rfloor)$.
- #primitive pythagorean triples with hypotenuse $< n$ approx $n/(2\pi)$.
- Frobenius Number:** largest number which can't be expressed as a linear combination of numbers $a_1, \dots, a_n$ with non-negative coefficients. $g(a_1, a_2) = a_1 a_2 - a_1 - a_2$, $N(a_1, a_2) = (a_1 - 1)(a_2 - 1)/2$. $g(d \cdot a_1, d \cdot a_2, a_3) = d \cdot g(a_1, a_2, a_3) + a_3(d - 1)$. An integer $x > (\max_i a_i)^2$ can be expressed in such a way iff. $x \mid \gcd(a_1, \dots, a_n)$.

12.1. Physics.

- Snell's law:** $\frac{\sin \theta_1}{v_1} = \frac{\sin \theta_2}{v_2}$

12.2. **Markov Chains.** A Markov Chain can be represented as a weighted directed graph of states, where the weight of an edge represents the probability of transitioning over that edge in one timestep. Let $P^{(m)} = (p_{ij}^{(m)})$ be the probability matrix of transitioning from state i to state j in m timesteps, and note that $P^{(1)}$ is the adjacency matrix of the graph. **Chapman-Kolmogorov:** $p_{ij}^{(m+n)} = \sum_k p_{ik}^{(m)} p_{kj}^{(n)}$. It follows that $P^{(m+n)} = P^{(m)} P^{(n)}$ and $P^{(m)} = P^m$. If $p^{(0)}$ is the initial probability distribution (a vector), then $p^{(0)} P^{(m)}$ is the probability distribution after m timesteps.

The return times of a state i is $R_i = \{m \mid p_{ii}^{(m)} > 0\}$, and i is *aperiodic* if $\gcd(R_i) = 1$. A MC is aperiodic if any of its vertices is aperiodic. A MC is *irreducible* if the corresponding graph is strongly connected.

A distribution π is stationary if $\pi P = \pi$. If MC is irreducible then $\pi_i = 1/\mathbb{E}[T_i]$, where T_i is the expected time between two visits at i . π_j/π_i is the expected number of visits at j in between two consecutive visits at i . A MC is *ergodic* if $\lim_{m \rightarrow \infty} p^{(0)} P^m = \pi$. A MC is ergodic iff. it is irreducible and aperiodic.

A MC for a random walk in an undirected weighted graph (unweighted graph can be made weighted by adding 1-weights) has $p_{uv} = w_{uv} / \sum_x w_{ux}$. If the graph is connected, then $\pi_u = \sum_x w_{ux} / \sum_v \sum_x w_{vx}$. Such a random walk is aperiodic iff. the graph is not bipartite.

An *absorbing* MC is of the form $P = \begin{pmatrix} Q & R \\ 0 & I_r \end{pmatrix}$. Let $N = \sum_{m=0}^\infty Q^m = (I_t - Q)^{-1}$. Then, if starting in state i , the expected number of steps till absorption is the i -th entry in $N1$. If starting in state i , the probability of being absorbed in state j is the (i, j) -th entry of NR .

Many problems on MC can be formulated in terms of a system of recurrence relations, and then solved using Gaussian elimination.

12.3. **Burnside's Lemma.** Let G be a finite group that acts on a set X . For each g in G let X^g denote the set of elements in X that are fixed by g . Then the number of orbits

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

$$Z(S_n) = \frac{1}{n} \sum_{l=1}^n a_l Z(S_{n-l})$$

12.4. **Bézout's identity.** If (x, y) is any solution to $ax + by = d$ (e.g. found by the Extended Euclidean Algorithm), then all solutions are given by

$$\left(x + k \frac{b}{\gcd(a, b)}, y - k \frac{a}{\gcd(a, b)}\right)$$

12.5. Misc.

12.5.1. Determinants and PM.

$$\det(A) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{i, \sigma(i)}$$

$$\text{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i, \sigma(i)}$$

$$\begin{aligned} pf(A) &= \frac{1}{2^n n!} \sum_{\sigma \in S_{2n}} \text{sgn}(\sigma) \prod_{i=1}^n a_{\sigma(2i-1), \sigma(2i)} \\ &= \sum_{M \in \text{PM}(n)} \text{sgn}(M) \prod_{(i, j) \in M} a_{i, j} \end{aligned}$$

12.5.2. *BEST Theorem.* Count directed Eulerian cycles. Number of OST given by Kirchoff's Theorem (remove r/c with root) $\# \text{OST}(G, r) \cdot \prod_v (d_v - 1)!$

12.5.3. *Primitive Roots.* Only exists when n is $2, 4, p^k, 2p^k$, where p odd prime. Assume n prime. Number of primitive roots $\phi(\phi(n))$ Let g be primitive root. All primitive roots are of the form g^k where $k, \phi(p)$ are coprime.

k -roots: $g^{i \cdot \phi(n)/k}$ for $0 \leq i < k$

12.5.4. Sum of primes.

$$S(n, p) = S(n, p-1) - f(p) \cdot (S(n/p, p-1) - S(p-1, p-1))$$

12.5.5. Floor.

$$\begin{aligned} \lfloor \lfloor x/y \rfloor / z \rfloor &= \lfloor x/(yz) \rfloor \\ x \% y &= x - y \lfloor x/y \rfloor \end{aligned}$$

12.5.6. *Large Primes.* 100894373, 103893941, 999999937, 1000000007, 16208191877

13. OTHER COMBINATORICS STUFF

Catalan	$C_0 = 1, C_n = \frac{1}{n+1} \binom{2n}{n} = \sum_{i=0}^{n-1} C_i C_{n-i-1} = \frac{4n-2}{n+1} C_{n-1}$
Stirling 1st kind	$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1, \begin{bmatrix} n \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ n \end{bmatrix} = 0, \begin{bmatrix} n \\ k \end{bmatrix} = (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}$
Stirling 2nd kind	$\left\{ \begin{smallmatrix} n \\ 1 \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n \\ n \end{smallmatrix} \right\} = 1, \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\} + \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\}$
Euler	$\left\langle \begin{smallmatrix} n \\ 0 \end{smallmatrix} \right\rangle = \left\langle \begin{smallmatrix} n \\ n-1 \end{smallmatrix} \right\rangle = 1, \left\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \right\rangle = (k+1) \left\langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\rangle + (n-k) \left\langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\rangle$
Euler 2nd Order	$\left\langle\!\left\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \right\rangle\!\right\rangle = (k+1) \left\langle\!\left\langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\rangle\!\right\rangle + (2n-k-1) \left\langle\!\left\langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\rangle\!\right\rangle$
Bell	$B_1 = 1, B_n = \sum_{k=0}^{n-1} B_k \binom{n-1}{k} = \sum_{k=0}^n \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$

#labeled rooted trees	n^{n-1}
#labeled unrooted trees	n^{n-2}
#forests of k rooted trees	$\sum_{i=1}^k \binom{n}{i} n^{n-i}$
$\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$	$\sum_{i=1}^n i^3 = n^2(n+1)^2/4$
$!n = n \times! (n-1) + (-1)^n$	$!n = (n-1)!(n-1) +! (n)$
$\sum_{i=1}^n \binom{n}{i} F_i = F_{2n}$	$\sum_i \binom{n-i}{i} = F_{n+1}$
$\sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}$	$x^k = \sum_{i=0}^k i! \left\{ \begin{smallmatrix} k \\ i \end{smallmatrix} \right\} \binom{x}{i} = \sum_{i=0}^k \text{Stirling}(k, i) x^i$
$a \equiv b \pmod{x, y} \Rightarrow a \equiv b \pmod{\text{lcm}(x, y)}$	$\sum_{d n} \phi(d) = n$
$ac \equiv bc \pmod{m} \Rightarrow a \equiv b \pmod{\frac{m}{\gcd(c, m)}}$	$(\sum_{d n} \sigma_0(d))^2 = \sum_{d n} \sigma_0(d)$
p prime $\Leftrightarrow (p-1)! \equiv -1 \pmod{p}$	$\gcd(n^a - 1, n^b - 1) = n^{\gcd(a, b)}$
$\sigma_x(n) = \prod_{i=0}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$	$\sigma_0(n) = \prod_{i=0}^r (a_i + 1)$
$\sum_{k=0}^m (-1)^k \binom{n}{k} = (-1)^m \binom{n-1}{m}$	$\sum_{i=1}^n 2^{\omega(i)} = O(n \log n)$
$2^{\omega(n)} = O(\sqrt{n})$	$v_f^2 = v_i^2 + 2ad$
$d = v_i t + \frac{1}{2} a t^2$	$d = \frac{v_i + v_f}{2} t$
$v_f = v_i + at$	

13.1. **The Twelffold Way.** Putting n balls into k boxes.

Balls	same	distinct	same	distinct	
Boxes	same	same	distinct	distinct	Remarks
-	$p_k(n)$	$\sum_{i=0}^k \left\{ \begin{smallmatrix} n \\ i \end{smallmatrix} \right\}$	$\binom{n+k-1}{k-1}$	k^n	$p_k(n)$: #partitions of n into k boxes
size ≥ 1	$p(n, k)$	$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	$\binom{n-1}{k-1}$	$k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	$p(n, k)$: #partitions of n into k nonempty boxes
size ≤ 1	$[n \leq k]$	$[n \leq k]$	$\binom{k}{n}$	$n! \binom{k}{n}$	$[cond]$: 1 if $cond = \text{true}$